THE UNIVERSITY OF QUEENSLAND

AUSTRALIA

Enabling Quantum Machine Learning Simulations via Direct Search

Jovana Gojkovic
Physics Honours Student

Under the supervision of
Dr. Riddhi Gupta

A thesis submitted to the University of Queensland
in partial fulfilment of the degree of Bachelor of Science with Honours
School of Mathematics and Physics
November 2025

© Jovana Gojkovic, 2025.

Typeset in LaTeX 2_ε.

The work presented in this Thesis is, to the best of my knowledge and belief original, except as acknowledged in the text, and has not been submitted either in whole or in part, for a degree at this or any other university.

Jovana Gojkovic

Abstract

Quantum machine learning (QML) and variational quantum algorithms (VQAs) are broad families of quantum algorithms that rely on classical optimisation to solve learning tasks. These quantum algorithms typically use gradient-based optimisers such as Adam or stochastic gradient descent (SGD) to tune free parameters in quantum computation. Gradient-based optimisers perform well in small, noiseless systems but struggle in realistic simulations with a large number of quantum bits (qubits), especially when gradients become unreliable to estimate. In this project, we explore gradient-free methods as an alternative approach for performing black-box optimisation on quantum learning problems. Specifically, we apply insights from a recent direct search algorithm, which employs random subspace projections to reduce the dimensionality of the parameter search space. Our results show that this direct search algorithm outperforms Adam for modest qubit numbers when finite-sampling errors are negligible. We additionally show that this algorithm can theoretically accommodate a finite number of quantum measurements, i.e., when sampling errors dominate. Building on this, we propose a doubly stochastic direct search approach that combines subspace sampling with finite-shot estimation, offering a scalable and error-resilient optimisation strategy for future QML and variational protocols.

Acknowledgements

I would like to thank my supervisor, Dr Riddhi Gupta, for the guidance and support throughout my project, and for encouraging me to seek out opportunities and continue improving as a researcher.

I would also like to thank Connor van Rossum for helping me get started with my project and for the helpful discussions. Thank you as well to the QML theory group for the discussions, practice talks and general support. Being able to talk through ideas and see what others are working on really helped me learn more about the field.

Finally, I would like to thank my friends and family who have supported me from the start, and have encouraged me to push forward throughout my Honours year.

Contents

Abstract **v**

Acknowledgements **vii**

List of Figures **xi**

List of Tables **xiii**

List of Abbreviations and Symbols **xv**

1 Introduction **1**

2 Background **5**
2.1 Quantum Information 5
2.1.1 Qubit 5
2.1.2 Gates 7
2.1.3 Multi-qubit Systems 9
2.1.4 Observables 10
2.1.5 Measurements 10
2.2 Classical Machine Learning and Optimisation Algorithms 12
2.2.1 Supervised Learning 13
2.2.2 Optimisation Landscapes 13
2.2.3 Classical Algorithms for Optimisation 14
2.3 Quantum Machine Learning 15

3 Literature Review **17**
3.1 Optimisation Challenges in VQAs 18
3.1.1 Supervised Learning in QML 18
3.1.2 Barren Plateaus 20
3.1.3 Exponentially Many Local Minima 21
3.1.4 Existing Mitigation Strategies 23
3.2 Optimisation Approaches in VQAs 24
3.2.1 Gradient-Based Methods 24
3.2.2 Gradient-Free Methods 26
3.3 Stochastic Optimisation Relevant to this Thesis 28
3.3.1 Quantum Stochastic Gradient Descent (QSGD) 28

3.3.2 Stochastic Sampling for Gradient-Free Simplex Methods 29

4 Testbed for QML Simulations 31
4.1 Data Re-uploading Circuits (DRCs) 31
4.2 Fourier Series as a Learnable Target 33
4.3 Usage of Adam as a Gradient-Based Optimiser 34
4.4 Characterising Learning Scalability of QML Simulations 36
4.4.1 Model Accuracy and Convergence Behaviour 36
4.4.2 Computational Resource Scaling 38
4.5 Towards Scalable Gradient-Free Optimisation 43

5 Direct Search for Variational Quantum Models 45
5.1 Reduced Subspace Direct Search Algorithm 45
5.1.1 Default Parameters and Notation 46
5.1.2 Algorithm Design 47
5.1.3 Convergence Criteria 49
5.2 Direct-Search for Quantum Models in the Finite-Shot Limit 53
5.2.1 Loss functions for quantum models 53
5.2.2 Direct search in finite-shot limit 54

6 Direct Search Simulations for Data Re-uploading Circuits 57
6.1 Methodology for Configuring Algorithm 1 58
6.1.1 Subspace dimension 59
6.1.2 Maximum evaluations 61
6.2 Empirical Evaluation of Hyper-parameters 62
6.2.1 Effect of Subspace Dimension on Learning Behaviour 62
6.2.2 Scaling with Maximum Evaluations 63
6.2.3 Computational Costs 65
6.3 Comparison of Gradient-free and Gradient-based Methods 67
6.4 Finite Shots and Doubly Stochastic Optimisation 71
6.4.1 Doubly Stochastic Direct Search 72
6.4.2 Limitations of Gradient-Based Optimisers Under Shot Noise 74
6.5 Does Algorithm 1 Overcome QML Trainability Issues? 75

7 Conclusions and Next Steps 77

References 79

List of Figures

2.1 Bloch sphere representation of a qubit 7
2.2 Bell state circuit diagram, showing a $|\Phi^+\rangle$ state 10
2.3 Measurement operation in quantum circuits 11
2.4 Basic quantum circuit diagram 15

3.1 Supervised learning circuit in QML 19
3.2 Exponentially many local minima in a loss landscape 22
3.3 Local cost/loss function mitigating barren plateaus 23
3.4 Comparison of gradient-free and gradient descent methods by total shot cost 27

4.1 Structure of a data re-uploading circuit (DRC) 33
4.2 Training layer with CNOT ladder structure 34
4.3 Example Fourier series target function 35
4.4 Trained DRC matching target Fourier function 37
4.5 Scaling of DRCs learning accuracy with qubit count 38
4.6 Loss convergence behaviour in DRCs 39
4.7 Runtime scaling of DRCs with qubit count and circuit depth 40
4.8 High qubit DRC simulation results and CPU usage 41
4.9 Memory and parameter scaling in DRC simulations 42
4.10 Parallelisation impact on DRC simulation runtime 43

5.1 Visualisation of standard direct search optimisation 46

6.1 Parameter space reduction via subspace projection 61
6.2 Direct search performance across reduced subspace sizes 63
6.3 Effect of evaluation budget on reduced subspace direct search performance . 64
6.4 Hyper-parameter heat maps showing direct search convergence regions . . . 66
6.5 Hyper-parameter heat maps of loss function evaluations 67
6.6 Comparison of direct search and Adam performance across qubit sizes 69
6.7 Shot noise robustness for direct search 72
6.8 Effect of using low numbers of shots on direct search learning 73
6.9 Comparison of direct search and Adam under finite shots 74

List of Tables

3.1 Untrainability of VQAs 24

6.1 Direct Search and Adam optimiser settings versus qubit count 68
6.2 Average best-step performance values and configurations of direct search and Adam optimisers 70

List of Abbreviations and Symbols

Abbreviations

Adam ..... Adaptive moment estimation

BO ..... Bayesian optimisation

CNOT ..... controlled-NOT

COBYLA ... Constrained by linear approximation

CPU ..... Central processing unit

DRC ..... Data re-uploading circuit

DSGD ..... Doubly stochastic gradient descent

GPU ..... Graphics processing unit

MSE ..... Mean squared error

NISQ ..... Noisy intermediate-scale quantum

PSO ..... Particle swarm optimisation

PSS ..... Positive spanning set

QAOA ..... Quantum approximate optimisation algorithm

QML ..... Quantum machine learning

QNN ..... Quantum neural network

QSGD ..... Quantum stochastic gradient descent

Qubit ..... Quantum bit

SEM ..... Standard error of the mean

SGD ..... Stochastic gradient descent

SLURM ..... Simple Linux utility for resource management

VQA..... Variational quantum algorithm

VQC..... Variational quantum circuit

VQE..... Variational quantum eigensolver

WHRF..... Wishart hypertoroidal random field

Symbols

$|\psi\rangle$..... quantum state

$\mathcal{H}$..... Hilbert space

ρ..... density matrix

O..... observable (Hermitian operator)

$\langle O\rangle$..... expectation value of observable O

$\dagger$..... conjugate transpose (dagger)

$*$..... complex conjugate

H..... Hadamard gate

$R_x(\cdot), R_y(\cdot), R_z(\cdot)$ single-qubit rotation gates (X, Y, Z axes)

X, Y, Z..... Pauli gates

S..... phase gate

n..... number of qubits

N..... number of data re-uploading training blocks

x..... classical input data

θ..... trainable circuit parameter

S..... number of measurement shots

y..... target (label) data

$g(x)$..... target function (Fourier series)

d..... degree of Fourier series

$\mathcal{L}$..... loss function

R^2..... coefficient of determination

η..... learning-rate

p ..... number of parameters / full parameter-space dimension

r ..... reduced subspace dimension

E ..... number of maximum evaluations

P_k ..... random projection matrix

d_k ..... search direction vector

$\mathcal{D}_k$ ..... polling direction set

α ..... optimisation step size

$\alpha_{\max}, \alpha_{\min}$ ... max/min step-size bounds

$\gamma_{\text{inc}}, \gamma_{\text{dec}}$ ..... step-size scaling factors

c ..... sufficient-decrease constant

$\tilde{o}$ ..... estimator of observable expectation value

$\mathbb{E}$ ..... expectation operator

$\mathbb{P}$ ..... probability

$\|\cdot\|$ ..... Euclidean norm

∇ ..... gradient operator

$\mathcal{O}$ ..... asymptotic (Big-O) notation

ϵ ..... stationary point tolerance

$\mathcal{F}$ ..... algorithm history

$\text{cm}(\cdot,\cdot)$ ..... cosine measure

ν ..... alignment constant in well-aligned property

σ ..... singular-value bound

q_w ..... probability of well-aligned property

p_d ..... probability of probabilistic descent

1 Introduction

Quantum machine learning (QML) is an emerging field of physics situated at the intersection of quantum computing and classical machine learning. It explores how principles of quantum information can be utilised to represent, process and learn from data in fundamentally new ways compared to classical computation methods. Central to this field are variational quantum algorithms (VQAs), which combine quantum circuits with classical optimisation routines to train parameterised models. These quantum circuits are composed of qubits – quantum information units – which are manipulated by a sequence of quantum gates and then measured to extract classical outcomes. In VQAs, a classical optimiser adjusts the free parameters of these gates in order to map classical data inputs to predictions by minimising a loss function.

VQAs are used for two major applications: to iteratively find ground states in quantum simulations [1], and to solve learning tasks in QML. Focusing on the latter case, they perform supervised learning tasks, such as classification or regression [2]. While quantum machine learning has not been proven yet to be capable of outperforming classical models [1], it demonstrates comparable performance in tasks like image classification and other small scale tasks [3, 4].

The practical development of quantum machine learning is currently constrained by two closely linked challenges: optimisation difficulty and computational cost. VQAs are particularly susceptible to these limitations as the quantum system size increases. These trainability issues often arise in complex optimisation landscapes, where models struggle to learn efficiently. Trainability refers to the model's ability to minimise a loss function of a specific task. Difficulty in training a quantum model may arise due to the complex structure and properties of the loss landscape [5, 6], such as highly non-convex structure or vanishing gradients, especially in high-dimensional parameter spaces [7, 8]. These unfavourable loss landscapes cause the classical optimiser to struggle to find useful parameter updates. The training challenge is further exacerbated by scaling the quantum circuit to large qubit counts

or deeper circuits [5], this typically limits quantum variational techniques to smaller scale tasks [9]. While some of the trainability challenges above can be reduced or amplified by the design of the quantum machine learning problem, the choice of an optimiser may affect how non-ideal features of the loss landscape impact training.

Gradient-based optimisation methods currently form the foundation of most QML research and are considered the industry standard for training variational quantum circuits (VQCs). Their efficiency and compatibility with QML training problems make them highly effective, and the preferred choice for most situations. However, because these algorithms rely on gradient information, they can fail in unfavourable loss landscapes where gradients become unreliable – either vanishing exponentially or indicating incorrect descent directions. Under these regimes, gradient-based approaches struggle to make meaningful parameter updates, limiting the trainability and scalability of QML models. These issues will be revisited in more detail in the literature review in Chapter 3.

Despite these pitfalls, gradient-free optimisation methods – which do not depend on gradient information – may offer advantages in such challenging landscapes. While gradient-free optimisation has been explored extensively in classical settings, this class of algorithms remains under-explored in QML, limited to a few preliminary studies with little systematic analysis of its scalability or robustness [10]. As a result, we lack a clear understanding of when and how gradient-free optimisation can provide advantages for training VQCs. In this thesis, we address this gap by reviewing existing literature, conducting direct empirical comparisons, and introducing a new gradient-free optimisation approach that has not previously been applied in QML.

The central comparison in this thesis lies between gradient-based and gradient-free optimisation methods for training VQCs. Both these approaches play the role of the classical optimiser in a VQA, sharing the same objective of minimising the loss function. However, they differ fundamentally in how they explore a given optimisation landscape. By empirically simulating these optimisers on a QML task, we benchmark their behaviour across increasing quantum circuit sizes in terms of learning accuracy and computational costs as well as their robustness to noise effects such as finite sampling (shot noise). Through these comparisons, we identify the regimes in which gradient-free methods can match or even outperform gradient-based approaches, providing insight into which optimisation strategies are most robust to the trainability challenges present in QML.

By making this comparison between gradient-based and gradient-free methods, this thesis seeks to clarify how different optimisation strategies respond to the unique challenges of quantum loss landscapes. Understanding their behaviours and differences can reveal when gradient-free methods offer advantages, guiding the development of more robust algorithms that learn accurately and efficiently. Importantly, our work addresses gaps in current knowledge of gradient-free methods in QML, introducing a method that has not previously been explored and demonstrating its effectiveness. Our findings contribute to the field by providing a foundation for future research, enabling further exploration of the role of classical optimisation in QML.

In this thesis, I led the code-based development, performed all simulations, analysed all data, generated all figures unless stated otherwise, and wrote the thesis in its entirety.

This thesis is organised as follows: Chapter 2 introduces the theoretical foundations of

quantum machine learning, including key concepts of quantum information and classical machine learning.

Chapter 3 reviews prior work analysing the main trainability issues in VQAs and existing gradient-based and gradient-free optimisation strategies.

The simulation framework developed for this project is described in Chapter 4, where the data re-uploading circuit (DRC) architecture is implemented using a gradient-based optimiser and provides a benchmark to study both performance and computational scaling.

This chapter uses unpublished code for simulating data re-uploading circuits from Ref. [11], which was originally developed in Python/Qiskit by a fellow Ph.D student, Connor van Rossum. My novel contributions have been to extend this Python/Qiskit code base from single-qubit simulations to an arbitrary number of qubits, while also investigating how entanglement should be generated based on publicly available information in Ref. [11].

In addition, standard versions of Adam in Python/Qiskit were slightly modified by Connor van Rossum to support his work in Ref. [12]. I have used the same implementation of Adam throughout this thesis for consistency between projects.

Chapter 5 presents new optimisation techniques that combine elements of stochastic and gradient-free methods to propose an approach with reduced computational cost. My main original contribution in this chapter is the application of direct search to VQA loss functions evaluated using sampled quantum measurements.

Chapter 6 presents original and as-yet unpublished results, where I empirically evaluate both existing and newly proposed gradient-free methods, providing the first systematic study of direct search for VQAs, including a detailed review of hyper-parameters that impact operational performance. We compare the performance of direct search against the baseline results for Adam from Chapter 4 to assess efficiency and robustness. This allows us to identify computational bottlenecks in runtime and memory usage when quantum circuits are scaled to larger numbers of qubits.

We also investigate my new proposed protocol for doubly stochastic direct search in reduced subspaces using finite numbers of quantum measurements. I altered the code base to allow for shot-based measurements, as opposed to density matrix simulations, and performed novel empirical tests to show convergence properties of my proof-of-concept algorithm.

Finally, Chapter 7 concludes with a summary of key findings, discussing future directions for scalable and efficient optimisation in QML.

Quantum machine learning remains a rapidly evolving field, with the potential to transform optimisation and learning tasks through quantum computation. By providing a structured simulation framework for evaluating optimisation algorithms, along with new gradient-free approaches, this thesis contributes to the broad effort to understand current challenges in the scalability and robustness of QML models.

2
Background

This chapter provides the necessary background required to understand the concepts and methods explored throughout this thesis. We begin this chapter by introducing central and fundamental principles of quantum information theory, followed by a high-level overview of machine learning in both classical and quantum contexts. These topics collectively form the theoretical basis of modern quantum machine learning, linking quantum information techniques with machine learning frameworks. The purpose of this background is to equip the reader with sufficient theoretical foundations to follow the subsequent literature review in Chapter 3, where we focus on optimisation and training challenges encountered in quantum machine learning (QML).

2.1 Quantum Information

In this section, we introduce fundamental concepts of quantum information relevant to this thesis. We limit our discussion to include concepts necessary for understanding the core quantum model we focused on, the data re-uploading circuit (DRC). These topics include the mathematical representation of qubits, basic quantum operators in the form of gates, multi-qubit systems, observables and measurements. More advanced concepts of quantum information theory are omitted as they fall outside the scope of this work. Below we present a concise foundation of quantum information, that supports understanding quantum circuit architectures analysed in later chapters.

2.1.1 Qubit

At the foundation of all computation lies the bit, the fundamental building block of all modern classical computers, existing in a state of on or off, 0 or 1. This classical bit, restricted to one of two discrete states, underpins all computation in modern computers, from simple

arithmetic to complex algorithms. Combining these bits together to high degrees through modern computer technology yields remarkable processing power through deterministic sequential logic.

Similarly, quantum computing incorporates the quantum equivalent of this, the quantum bit or the qubit. Qubits complete the foundation for a quantum system, with qubits capable of exploiting quantum mechanical properties with superposition and entanglement. Unlike the classical bit, the qubit can exist in coherent superposition of both 0 and 1 simultaneously. The state of superposition is an intrinsic property of qubits, existing in a state in between 0 and 1, however, this all changes when a qubit is observed collapsing to either a 0 or 1 state, probabilistically [13].

A quantum state describes the complete information of a quantum system and can be represented as a column vector in a complex linear vector space. We can describe this column vector state of a quantum bit written in Dirac notation as a ket $|\psi\rangle$. For every ket, there is exists a corresponding bra denoted as $\langle\psi|$, which is the complex conjugate row vector of the ket. A bra maps a ket to a complex scalar through the inner product, $\langle\phi|\psi\rangle \in \mathbb{C}$. In this thesis, we only consider finite-dimensional vector spaces with such an inner product, known as Hilbert spaces $\mathcal{H}$. All quantum states satisfy the normalisation condition,

$$\langle\phi|\psi\rangle = 1, \tag{2.1}$$

if and only if $\phi = \psi$. The inner product also satisfies the conjugate symmetry property,

$$\langle\phi|\psi\rangle = \langle\psi|\phi\rangle^*. \tag{2.2}$$

A quantum state can also exist as a superposition. In this context, a superposition of quantum states refers to a linear combination of basis states with complex coefficients, where the total probability $|\langle\phi|\psi\rangle|^2 = 1$. For a single qubit, this is written as,

$$|\psi\rangle = \alpha\,|0\rangle + \beta\,|1\rangle\,, \tag{2.3}$$

where $|0\rangle$ and $|1\rangle$ form the orthonormal basis and equate to the classical 0 and 1, where α and β are the complex probability amplitudes $|\alpha|^2 + |\beta|^2 = 1$. The states are $|0\rangle$ and $|1\rangle$ are called the computational basis states.

Quantum states can also be represented as a matrix in Hilbert space. This is called the density matrix, denoted ρ. A density matrix is a Hermitian matrix, meaning it is equal to its conjugate transpose; positive semidefinite, meaning it's eigenvalues are all non-negative; and has unit trace, meaning the sum of diagonal elements equals one. Density matrices are useful as they provide a unified way to describe both ideal quantum systems and noisy quantum systems.

In an ideal quantum system, the state of the system can be described as a pure state $|\psi\rangle$. This can be written in the density matrix formalism as,

$$\rho = |\psi\rangle\langle\psi|. \tag{2.4}$$

This is mathematically equivalent to the state vector of the system but allows for a more consistent description of measurements [13–15]. In realistic cases, the quantum state can

be noisy or uncertain, which can be represented by a mixed state, defined as a statistical mixture of pure states,

$$\rho = \sum_i c_i |\psi_i\rangle\langle\psi_i|, \tag{2.5}$$

where the coefficients c_i are real, non-negative, and sum to one. By representing both pure and mixed states as density matrices, we can utilise the same formalism to calculate expectation values or measurement outcomes.

While the density matrix formalism provides a useful algebraic representation of a qubit's state, the Bloch sphere offers a geometric perspective. A single qubit can be visualised as a point on or inside the Bloch sphere, as shown in Figure 2.1. Pure states ($Tr(\rho^2) = 1$) lie on the surface of the sphere, while mixed states ($Tr(\rho^2) < 1$) occupy points inside the sphere. Each state of a single qubit can be described as a location within the sphere or alternatively using spherical coordinates with θ and ϕ angles [13, 14].

To perform computation we must be able to transform and controls these quantum states. This is possible through using quantum gates, the quantum equivalent of classical logic gates. Quantum gates are vital to manipulating qubits to form the fundamental operations of quantum algorithms.

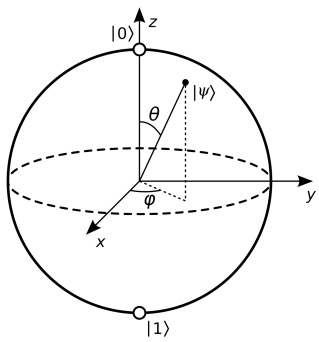

Figure 2.1: Bloch sphere representation of a qubit [16].

2.1.2 Gates

Quantum gates are unitary operations that transform qubit states ensuring that quantum coherence is preserved. Gates allow individual qubit manipulation and generate entanglement between qubits.

Single-qubit Gates

The most basic gate is the Hadamard gate, which introduces superposition into a circuit by performing a rotation on the Bloch sphere placing the qubit in equal probability of collapsing to 0 or 1.

$$H = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \tag{2.6}$$

This simple gate is important for initialising qubits in a state of superposition allowing for quantum algorithms to exploit parallelism.

Rotation gates are another type of single qubit gate, applying a continuous transformation to the qubit state via rotation on the Bloch sphere about the X, Y or Z axes. The rotation gate applies an angle θ of rotation on the chosen axis, this variable being utilised as a trainable parameter in quantum machine learning algorithms.

$$R_x(\theta) = \begin{bmatrix} \cos(\frac{\theta}{2}) & -i\sin(\frac{\theta}{2}) \\ -i\sin(\frac{\theta}{2}) & \cos(\frac{\theta}{2}) \end{bmatrix} \tag{2.7}$$

$$R_y(\theta) = \begin{bmatrix} \cos(\frac{\theta}{2}) & -\sin(\frac{\theta}{2}) \\ \sin(\frac{\theta}{2}) & \cos(\frac{\theta}{2}) \end{bmatrix} \tag{2.8}$$

$$R_z(\theta) = \begin{bmatrix} e^{\frac{-i\theta}{2}} & 0 \\ 0 & e^{\frac{i\theta}{2}} \end{bmatrix} \tag{2.9}$$

Indeed, the Pauli gates are an important family of single qubit gates and are the limiting cases of the above rotation gates. The Pauli X, Y, and Z gate matrices are given by,

$$X := \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \tag{2.10}$$

$$Y := \begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix} \tag{2.11}$$

$$Z := \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix} \tag{2.12}$$

These correspond to setting $\theta = \pi$ in R_x, R_y, R_z respectively, up to a global phase factor of $-i$.

Finally, the S gate, otherwise known as the phase gate, applies a $\pi/2$ phase shift:

$$S = \begin{bmatrix} 1 & 0 \\ 0 & i \end{bmatrix}. \tag{2.13}$$

Pauli operators are the standard choice of observables in quantum circuits. Measuring in the Z basis corresponds to directly measuring the qubit. However, to measure in the X or Y basis, we must first rotate the qubit using a Hadamard gate H or a combination of a conjugate phase gate and a Hadamard gate $S^\dagger H$ before measurement [17]. This demonstrates that single qubit gates are not only used to manipulate the state of qubits, but also to transform the measurement basis for observables.

Two-qubit Gates

A controlled-NOT gate or CNOT gate is a two-qubit gate used to create entanglement between qubits. A two-qubit gate requires two input qubits - one to be the control and the other to be the target. If the control qubit is in state 1, then the target is flipped, otherwise the target is left unchanged [13]. By coupling qubits with CNOT gates, entanglement can

be generated within a quantum circuit, correlating qubits together in a way not possible in classical computing.

$$|c\rangle\,|t\rangle \rightarrow |c\rangle\,|t \oplus c\rangle \tag{2.14}$$

$$CNOT = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix} \tag{2.15}$$

By combining these quantum logic gates together within a sequence, we can form a quantum circuit. Multiple gates can be grouped together to form a layer, often symbolised and defined for specific logical functionality.

2.1.3 Multi-qubit Systems

Multi-qubit systems are an example of a composite quantum system. For n qubits, a 2^n dimensional Hilbert space is generated by using a tensor product of subsystems. For two qubits with states,

$$|\psi_1\rangle = \alpha\,|0\rangle + \beta\,|1\rangle\,, \quad |\psi_2\rangle = \gamma\,|0\rangle + \delta\,|1\rangle\,, \tag{2.16}$$

yields a combined state of

$$|\psi\rangle = |\psi_1\rangle \otimes |\psi_2\rangle = \alpha\gamma\,|00\rangle + \ \alpha\delta\,|01\rangle + \beta\gamma\,|10\rangle + \beta\delta\,|11\rangle\,. \tag{2.17}$$

In this two-qubit system, the possible states grow to $2^2 = 4$ with states $|00\rangle, |01\rangle, |10\rangle$, and $|11\rangle$ [13]. This exponential scaling allows for complex computation with a relatively low number of qubits.

Scaling is an important factor in why you would use a multi-qubit system, but another important feature is entanglement – the unique quantum mechanical property where the state of each qubit becomes linked with others. Entanglement is one of the key properties of quantum systems that does not have a classical correlation as entangled states cannot be written as a product of individual qubit states. One of the Bell states is a classic representation of entanglement:

$$\left|\Phi^+\right\rangle = \frac{|00\rangle + |11\rangle}{\sqrt{2}}. \tag{2.18}$$

This is easily constructed in a quantum circuit via a Hadamard gate and CNOT gate as displayed in Figure 2.2.

In this Bell state, the qubits are in an entangled state, which has the property that measuring one qubit immediately determines the outcome of the other. An entangled state can be correlated – measuring one means that the other has the same result – or anti-correlated – measuring one means that the other has the opposite result. Quantum entanglement creates non-classical correlations that cannot be replicated by classical methods. To understand how information is extracted from these quantum states, we turn to the concept of observables and measurements.

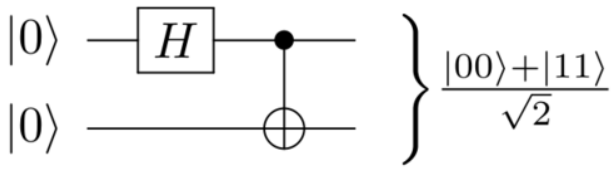

FIGURE 2.2: Bell state circuit diagram, showing a $|\Phi^+\rangle$ state [18].

2.1.4 Observables

Before considering the measurement of quantum circuits, we must first understand the core principle of observables. In quantum mechanics, an observable is a physical quantity or value that can be extracted from a quantum system. In the context of quantum information, this could be something simple like asking whether a qubit is in state 0 or 1, or more generally, evaluating the output of a quantum circuit.

Observables are represented mathematically as Hermitian (self-adjoint) operators, usually denoted O, acting on the quantum state $|\psi\rangle$. Hermitian operators have the important property that their eigenvalues are real, which ensures that measurement outcomes correspond to real physical quantities. When we perform a measurement, we are effectively evaluating the observable for the current state, and the expectation value $\langle\psi|O|\psi\rangle$ is always real due O being Hermitian [13–15].

Expectation Values

The expectation value of an observable O with respect to the quantum state communicates the average outcome of many measurements of that observable. For a pure state $|\psi\rangle$, the expectation value is the inner product of the O with respect to the pure state:

$$\langle O\rangle = \langle\psi|O|\psi\rangle \tag{2.19}$$

This expectation value can also be represented using the density matrix formalism:

$$\langle O\rangle = \mathrm{Tr}(\rho O), \tag{2.20}$$

where $\rho = |\psi\rangle\langle\psi|$ for a pure state, and $\mathrm{Tr}(\cdot)$ denotes the trace operation (sum of diagonal matrix elements). This form is useful as it allows for both pure and mixed states to be measured [14, 15].

2.1.5 Measurements

At the end of a quantum circuit, the final quantum state must be extracted from the system, this process is known as measurement. This measurement records the final quantum state by collapsing the superposition to yield a classical numerical result. Quantum circuits must record measurements only at the end as collapsing a qubit fundamentally alters the system, unlike classical systems where the state of a bit can be directly accessed without disruption.

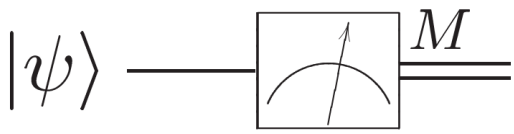

Figure 2.3: Measurement operation in quantum circuits [13].

The measurement operation connects a single qubit state to a probabilistic classical bit, the probabilities $|\alpha|^2$ and $|\beta|^2$ for the state 0 or 1 on the classical bit. The qubit's superposition is collapsed to the observed basis state and in this process the qubit is forced into a definite state. Quantum states cannot be cloned or reused following measurement, this being due to the no-cloning theorem (impossible to identically copy unknown quantum state) as each measurement irreversibly collapses the qubit [13]. This property is important for quantum machine learning because extracting classical data from a quantum circuits requires estimating expectation values, which can only be obtained by averaging over repeated measurements of the same circuit. Measurement in quantum circuits allows for quantum computation to be used in a classical context.

Once a quantum circuit has been measured, the individual outcomes (0 or 1) are not a meaningful value on their own, instead what is important is the average value of the observable of the circuit.

In practice, calculating expectation values using the full density matrix $\mathrm{Tr}(\rho O)$ can be computationally costly, and on real quantum hardware it is not directly available. Since a single quantum measurement only provides one probabilistic outcome, expectation values must instead be estimated statistically. This is done by executing the circuit multiple times, with each execution being referred to as a shot. By repeating the circuit for many measurement shots and averaging the outcomes, we can approximate the true expectation value. This process introduces sampling error, which is important to consider in quantum machine learning [14].

We now describe how to obtain expectation values of observables using two different approaches. For this thesis, we only need to consider observables that are given by a Pauli operators on each qubit.

If a Pauli observable on a qubit is I or Z, we directly measure the qubit. If a desired Pauli observable is X, we apply a H before measurement. If a desired Pauli observable is Y, we apply $S^\dagger H$ before measurement.

As introduced later in Chapter 4, our observable of interest is always the Pauli Z operator on the first qubit with identity Pauli operators applied to all other qubits. There are two possible methods of computing the expectation value of the Z operator on the first qubit, $\langle Z \rangle$: shot-based estimation and the exact density matrix method.

Shot-based Estimation

Using finite shots, each measurement gives us a classical outcome of 0 or 1, and we build a histogram over multiple shots. The final expectation value using shots is calculated as,

$$\langle Z \rangle \approx \frac{S_0 - S_1}{S}, \tag{2.21}$$

where S_0 is the number of 0 outcomes, and S_1 is the number of 1 outcomes, and S is the total number of shots.

Exact Density Matrix Method

Using exact simulation, we define the reduced density matrix in an n-qubit system as,

$$\rho_{\text{red}} = \text{Tr}_{2,\ldots,n}(\rho), \tag{2.22}$$

where $\text{Tr}_{2,\ldots,n}(\rho)$ denotes the partial trace, tracing out all qubits except for the first, and ρ is the full density matrix. This operation effectively marginalises out the other qubits, yielding the state of the first qubit alone. The expectation value of the observable $\langle Z \rangle$ on the first qubit can then be written as,

$$\langle Z \rangle = \text{Tr}(\rho_{\text{red}} Z). \tag{2.23}$$

This method provides the exact expectation value of the circuit with no sampling error, with the density matrix being of size $2^n \times 2^n$.

Increasing the number of shots improves the accuracy of the shot-based estimate of the expectation value, but this also increases computational costs. In quantum machine learning, this method can be considered costly, especially for small qubit count circuits, due to the small density matrix (of $2^n \times 2^n$ size). However, with larger qubit circuits, the exact density matrix method of calculating expectation values can be more costly due to the exponential scaling size of the density matrix, and shot-based estimation becomes more practical.

The concepts introduced in this section form one half of the theoretical foundation for quantum machine learning. In the following section, we introduce classical machine learning concepts, which form the second half of this foundation [14].

2.2 Classical Machine Learning and Optimisation Algorithms

Machine learning is a method for which an algorithm can slowly learns defined patterns or makes decisions based on data without being explicitly programmed to do so. This field is a subset of artificial intelligence, focusing specifically on improving from experience with a set of data and usually a target. Machine learning methods are broadly categorised into three general groups with supervised, unsupervised and reinforcement learning [19]. Depending on the nature of the data and the desired function of the algorithm, each method has their own use cases. However, in the context of quantum machine learning the most common method is supervised learning due to its compatibility with noisy intermediate-scale quantum (NISQ) computing devices [14]. A high level description of classical supervised learning and optimisation provided below, to be expanded upon in later chapters.

2.2.1 Supervised Learning

As mentioned above, supervised learning is one of the most widely used forms of machine learning and not just in quantum computing. Supervised learning trains with the goal of mapping a set of data inputs x to known outputs y based on labelled examples. The model uses data in the form of feature-label pairs, aiming to learn accurate predictions for unseen data by generalising results from the training examples. Supervised learning is most effectively used for regression tasks – predicting continuous values for a function – and classification tasks – predicting discrete trends [20].

Training In both classical and quantum machine learning, training is an important step in the machine learning process, as it forms the basis for the algorithm's learning and decision making. The model is given a set of training data with labelled input-output pairs and attempts to learn patterns by adjusting internal variables to best match the data correctly. The accuracy of the algorithms pattern training is determined by the loss function. The quality of learning is quantified through a loss function, which measures how close the models' predictions are to the true labels. During training, the loss function is minimised algorithmically by iteratively updating parameters to reduce loss, moving toward a parameter configuration that fits the data best. A common method for this is gradient descent, where updates are made in the direction of the steepest decrease of the loss function [19].

Testing Following this training step, the model's accuracy is further evaluated on a test set – this set being a separate set of data that was not included in the training set. This tests the model's accuracy on data beyond examples it has seen. If the model's performance on the training data is accurate with a low loss function, but then poorly predicts the test data, the model can be defined as over-fit – meaning that the algorithm just memorised training data instead of learning patterns or rules.

Generalisation The key measure of the success of a supervised learning algorithm is the generalisation of the model. Generalisation is the model's ability to accurately predict patterns for unseen data, this being the goal of machine learning algorithms, that your algorithm has learnt the nature of the data you trained it on. Good generalisation is characterised by balancing over-fitting – learning patterns too specific to the training data (memorising) – and under-fitting – being unable to portray underlying patterns. There are various techniques that are used to improve generalisation such as cross-validation, regularisation and early stopping [20] and in the context of quantum machine learning, circuit design and choice of optimisation algorithms can also be used [14].

2.2.2 Optimisation Landscapes

Minimising the loss function is central to training a machine learning model, with the loss function serving to measure the discrepancy between the model's learned prediction and the true output labels. By changing internal parameters, the loss function value changes, forming an optimisation landscape parameter space. The landscape describes how the loss

function varies with respect to the parameters, and determines how easily an optimiser can navigate the space towards an optimal solution.

Imagining the optimisation landscape as a physical terrain, features such as deep valleys, broad hills, or flat plateaus can influence a model's convergence and generalisation [19]. Ideally, the optimiser navigates towards a broad low loss region of the landscape - a region that correlates to minimal error but stable generalisation to unseen data. However, in optimisation landscapes with difficult areas such as flat regions, the optimiser's progress can stall as the loss provides little directional information for parameter updates.

Mathematically, in classical machine learning, an ideal optimisation landscape is referred to as a convex loss landscape, guaranteeing the existence of a single global minima where gradients vanish. These idealised situations allow for optimisation to be straightforward and predictable, always converging to the global solution [21]. On the other hand, most practical models, including quantum machine learning circuits, result in highly non-convex landscapes usually including many local minima [22]. These challenges in the context of quantum algorithms are discussed in Section 3.1.

2.2.3 Classical Algorithms for Optimisation

Once the optimisation landscape is defined by the loss function, it must be navigated using optimisation algorithms. These algorithms iteratively update the model parameters to reduce loss and ideally reach the optimal solution. Optimisation algorithms form the core logic for the learning process, determining how efficiently and accurately a model can be trained.

Generally, these optimisation algorithms can be grouped into two broad categories: gradient-based and gradient-free methods. Gradient-based methods, such as gradient descent methods, rely on evaluating the gradient of the loss function to guide the next iterative parameter update towards the solution [19, 23]. These methods are widely used within classical and quantum machine learning due to their efficiency.

In contrast, gradient-free methods do not require explicit gradient information, primarily exploring the optimisation landscape using geometric search methods to evaluate the change in loss between points [24]. These methods tend to be under-utilised. While often slower in classical settings, their key strength comes from their robustness to noise and their ability to operate in landscapes where gradients unreliable or vanish entirely.

The choice between gradient-based and gradient-free approaches depends on the problem difficulty and complexity, as well as the characteristics of the underlying optimisation landscape.

In the next chapter, we will introduce quantum machine learning for classical gradient-based methods described in Chapter 4 with the specific use of Adam [25], described in detail in Section 4.3. Gradient-free methods are first described in Section 3.2 and compared to gradient-based methods in Section 6.3.

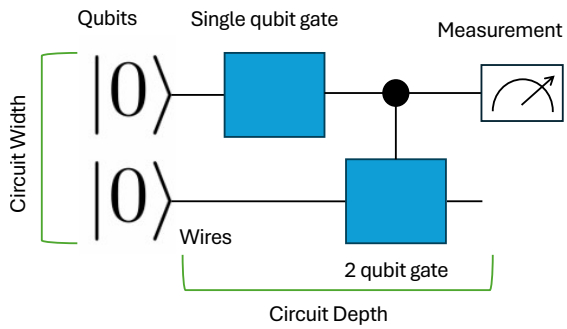

Figure 2.4: Diagram of a basic quantum circuit, with key features indicated. Qubits are represented by the notation $|0\rangle$, wires extend horizontally, single qubit gates are depicted as boxes, and two-qubit gates are shown as connected boxes across wires. Measurement is represented by a meter icon at the end of the circuit. The diagram uses brackets to denote the circuit's depth and width.

2.3 Quantum Machine Learning

Quantum machine learning (QML) refers to the broad topic of utilising quantum mechanical principles to enhance or enable machine learning. By representing and processing data through quantum states, QML models leverage quantum mechanics to potentially achieve computational advantages in learning and optimisation. Currently, most approaches to QML are based on a more general class of variational quantum algorithms (VQAs) [1]. VQAs have become a core framework for most noisy intermediate-scale quantum (NISQ) computing devices and include variational quantum eigensolvers (VQEs), quantum approximate optimisation algorithms (QAOAs), and other quantum machine learning models [1, 14].

Variational quantum algorithms (VQAs) utilise a hybrid quantum-classical approach where internal degrees of freedom in quantum circuits are tuned using classical optimisers. As discussed in detail in Chapter 3, single-qubit gates in a quantum circuit are parameterised by classical variables. These parameters are either optimised by a classical optimisation algorithm [1, 26] and/or encode classical data into a quantum Hilbert space in quantum machine learning applications [11] (Figure 3.1). In the former case, the role of the classical optimiser is to find parameter values that minimise a loss function. During training, the quantum circuit encodes data through the parameterised quantum states, while the classical optimiser iteratively adjusts these parameters to improve the circuit output. This learning loop is repeated ideally until the loss is minimised or convergence is reached. This hybrid approach combines two paradigms: quantum circuits can represent data through transformations in a high-dimensional Hilbert space, while classical optimisers efficiently update parameters using information from quantum measurements. This framework is particularly well fitting for quantum machine learning applications using classical data.

In QML applications, the quantum circuit component of a VQA is known as a variational quantum circuit (VQC), whose structure can be defined and controlled in terms of its architecture, depth, width, and parameterisation. A basic diagram of a quantum circuit is shown in Figure 2.4, where the features of depth and width are explicitly labelled. Other features

such as architecture and parameterisation are discussed in detail within Chapter 4 (Figure 4.1). These design choices influence both the expressivity of a circuit and its trainability. Variational quantum circuits (VQCs) have four distinct features:

- Architecture: The structure or layout of the quantum circuit, such as the logical sequence of gates and qubit inter-connectivity.
- Depth: The number of sequential gate layers in the circuit.
- Width: The number of qubits used in the circuit.
- Parameterisation: The specific choice of which gates are tunable and how parameters are initialised and scaled.

The architecture of a circuit, determines the circuit's expressivity and trainability [14]. Here, expressivity is a measure of the circuit's potential representational power, which is directly linked to its depth, width and the types of quantum gates used. Circuit architecture defines the core structure forming the logic of the circuit, with architectures often designed to be modular and scalable to a desired depth or width.

The choice of depth and width tailors a VQC for a specific problem. Wider circuits (more qubits) can represent higher dimensional data due to the increased Hilbert space. Deeper circuits (more gates/layers) enable more complex data transformations. Both these factors can increase the circuits expressivity, but they also influence trainability introducing issues that can make optimisation more challenging [27]. These design choices provide trade-offs between expressibility and trainability [27–29]. This is an important consideration for the specific function and purpose of the QML algorithm.

Quantum circuits are usually designed for a specific task, capable of suiting many types of learning paradigms like supervised (classification, regression) [9, 30, 31], unsupervised (generative modelling) [1, 32] or reinforcement (value or policy based) [33–35] learning. These learning tasks define the objective, but the specific circuit architecture constructed for the problem determines how it is solved. These circuit architectures are more broadly referred to as quantum circuit ansatzes [36] as the specific circuit choices designed with an structural bias to solve these learning tasks. Among recently developed ansatzes, the data re-uploading circuit (DRC) has emerged as an asymptotically universal circuit architecture specifically designed for supervised learning tasks such as classification or regression [31]. In particular, DRCs can approximate any function that has a Fourier series transform or decomposition, making them a flexible and theoretically universal testbed for comparing optimisation algorithms, discussed in detail in Chapter 4.

All variational quantum algorithms rely on a classical optimiser to tune free parameters of the circuit. For variational quantum circuits, the optimisation landscape can exhibit regions that are exponentially flat or have many sharp minima [5, 6]. These types of landscapes make training models inherently difficult due to the quality of loss information, motivating the study of optimisation strategies to mitigate these unwanted effects on optimisation. These topics will be discussed in the Literature Review in Chapter 3.

3
Literature Review

In the previous chapter, we introduced the fundamental principles of quantum information and machine learning, establishing how these two fields intersect to form quantum machine learning (QML). Building on this foundation, this chapter focuses on understanding the trainability of variational quantum algorithms. Our review of trainability in current literature centres on two aspects: challenges in training QML models, and approaches to optimisation in QML.

In Section 3.1, two limiting cases of these loss landscape problems are introduced. These are known as barren plateaus and exponentially many local minima. These challenges arise at two extremes of parameterised quantum circuits, with overly deep or expressive circuits [5, 27, 28] resulting in barren plateaus, and too shallow or under-parameterised circuits [28, 37] generating exponentially many local minima. These two problems become more pronounced in systems with a large number of qubits as the number of parameters affects the trainability of the quantum model.

In Section 3.2, a detailed review of classical gradient and gradient-free methods is provided, including their susceptibility to quantum machine learning challenges discussed above. Current literature primarily focuses on gradient-based optimisers such as stochastic gradient descent (SGD) [37–39] and Adam (Adaptive Moment Estimation) [11, 28, 37, 39] to train these models. These methods are widespread for use in variational quantum algorithms due to their simplicity and compatibility with classical optimisers. However, these gradient-based optimisers exhibit notable issues regarding the optimisation challenges. As gradient-based optimisers rely on derivative information, in difficult loss landscapes, gradients can vanish exponentially [5] and the optimiser is more likely to fall into trap minima [37]. This makes gradient-based optimisers more susceptible to these challenges associated with difficult optimisation landscapes, thus slowing convergence to the optimal solution [5, 37]. Meanwhile, gradient-free methods refer to a range of different optimisation algorithms, characterised by the fact that they do not rely on derivative information to perform updates. Direct search

methods are a common class of gradient-free optimisers that search for minima by evaluating the objective function directly. Examples of these include simplex-based methods like Nelder-Mead [40], and COBYLA (Constrained BY Linear Approximation) [41].

Currently, there is considerable debate about whether gradient-free methods could be more advantageous compared to gradient-based methods for addressing the types of trainability issues for quantum models. On the one hand, there is evidence that gradient-free methods are susceptible to similar effects of vanishing loss differences in barren plateaus [10]. On the other hand, gradient-free methods are known to be better at avoiding classical local minima [42, 43] and are the standard tool for optimisation problems when the loss function represents a black-box physical system [44, 45]. In Chapter 6, we will investigate the properties of gradient-free methods, and compare their performance to gradient-based methods for optimising quantum models of increasing qubit count.

Finally, in Section 3.3 we review insights from two techniques that we incorporate in Chapter 5 to propose a modified gradient-free method. The first technique originates from stochastic gradient descent (SGD), as a stochastic sampling gradient-based variant that has been adapted for VQAs and utilises finite-shot measurements. The second technique originates from state-of-the-art literature on modern gradient-free methods and proposes stochastic sampling strategies to modify existing gradient-free methods. Both techniques aim to reduce the resource costs for the overall optimisation procedure and are combined in this thesis to propose a new protocol in Chapter 5.

3.1 Optimisation Challenges in VQAs

In this section, we will review common challenges in optimising variational quantum circuits in supervised learning problems in hybrid quantum-classical machine learning. These challenges arise from the structure of the quantum circuit and the measurement process, rather than the choice of classical optimiser. We will present barren plateaus and exponentially many local minima as two limiting cases that make optimisation difficult, and we will also discuss existing mitigation strategies in the literature.

3.1.1 Supervised Learning in QML

Building on the general principles of supervised learning introduced in Section 2.2.1, QML follows the same core objective of training models to learn mappings between data inputs and known outputs. However, in QML, this process is implemented through a parameterised quantum circuit rather than a classical model. Although the optimiser remains classical, optimisation is performed on the parameterised quantum gates in order to learn from data to fit a target function [9, 46] across an often high-dimensional and complex loss landscape. Specifically, the parameterised quantum circuit is defined by a set of tunable parameters θ encoded within its quantum gates, with the goal of minimising the loss function $\mathcal{L}(\theta)$, which serves as a measure of how close the predicted model is to the target function. This is illustrated in Figure 3.1. Since the loss function is defined by the parameter space of the quantum circuit, the optimisation landscape is often high-dimensional and can be non-convex, which presents unique challenges during training.

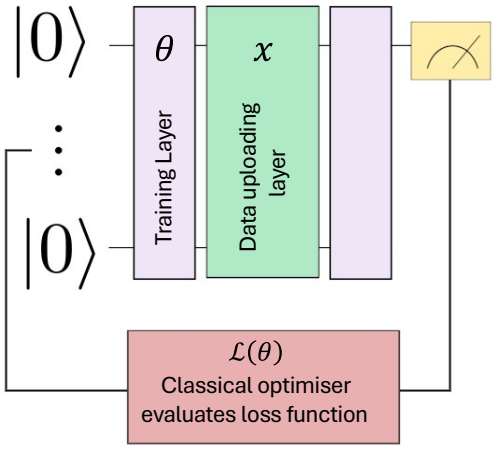

Figure 3.1: An example supervised learning circuit in quantum machine learning (QML), where classical data x is encoded into a parameterised quantum gates with trainable parameters θ. Measurements are used to compute the loss function $\mathcal{L}(\theta)$, which is minimised by a classical optimiser.

In classical supervised learning problems, the optimisation landscape is usually convex, meaning there is a single global minimum and no local minima, indicating that the optimiser can reliably converge to the most optimal solution [21]. However, in quantum systems, the loss function is generally non-convex due to the quantum model prediction on which the loss is based. A non-convex optimisation landscape implies the presence of local minima, saddle points or flat regions [22, 47, 48]. These characteristics of the loss function arise from the complex nature of quantum circuit models and can significantly affect optimiser performance and model trainability, remaining a central cause of the challenges faced in variational quantum models [49, 50].

In supervised learning problems using variational quantum circuits, the objective is to optimise the loss function through the circuit's trainable parameters θ. The quantum prediction is calculated through the expectation value of quantum observables - which is obtained by measuring the final output of the circuit. For the input x and trainable parameters θ, a simple loss function can be constructed as such [38]:

$$\mathcal{L}(\theta) = \langle O \rangle_\theta, x \tag{3.1}$$

This loss function is essentially the measurement of the quantum circuit, with the observable O being measured in the circuit. A commonly used loss function in supervised learning, including in quantum machine learning, is the mean squared error (MSE),

$$\mathcal{L}(\theta) = \frac{1}{M} \sum_{j=1}^{M} (\langle O \rangle_{\theta, x_j} - y_j)^2 \tag{3.2}$$

where M is the size of the data set, and y refers to the target data [38]. Mean squared error is a convex loss function with respect to its input, as it is a quadratic function. However, in

a quantum setting, the input to the loss function - the expectation value $\langle O \rangle$ of the circuit which depends on θ and x - is non-linear with respect to x and θ. This expectation value is not linearly correlated to θ or x, as its dependence is governed by the structure of the parameterised quantum circuit - particularly circuit depth and structure. Despite MSE being a convex loss function, the resulting loss landscape $\mathcal{L}(\theta)$ is generally non-convex due to the complex relationship between the quantum observable O and trainable parameters θ as well as the input data x.

The structure of the optimisation landscape can affect different optimisers in a multitude of ways. In this review, we focus on two groups of optimisers: gradient-based and gradient-free defined in Section 3.2. Gradient-based optimisers use the gradient of the loss function with respect to the circuit parameters θ to provide the direction and magnitude of change to update parameters where the loss would decrease. In contrast, gradient-free optimisers will use only loss values to search and update parameters such that the loss function decreases. However, both the evaluation of loss values or estimating its gradients are determined by the structure of the quantum circuits. Irrespective of the choice of optimisers, poor design of loss functions could lead to difficulties in optimisation. The next sections will centre on the two limiting cases, where optimising the loss function becomes difficult: with barren plateaus and exponentially many local minima.

3.1.2 Barren Plateaus

A well-known problem in quantum optimisation has been the emergence of barren plateaus in the loss function of variational quantum circuits (VQCs). The term barren plateau [5] refers to a region in the parameter space where the gradient of the loss function vanishes exponentially with the number of qubits. This exponential decay makes meaningful parameter updates for the optimiser extremely difficult [5, 29], stalling optimisation. For a loss function $\mathcal{L}(\boldsymbol{\theta})$ depending on a parameter vector $\boldsymbol{\theta} = (\theta_1, ..., \theta_M)$, let $\partial_\mu \mathcal{L}(\boldsymbol{\theta}) = \frac{\partial \mathcal{L}(\boldsymbol{\theta})}{\partial \theta_\mu}$ denote the partial derivative of the function with respect to $\theta_\mu \in \boldsymbol{\theta}$. The literature [10] characterises barren plateaus satisfying multiple equivalent conditions including,

$$E_\theta[\partial_\mu \mathcal{L}(\boldsymbol{\theta})] = 0, \tag{3.3}$$

and exponentially vanishing variance of the derivatives with the number of qubits n,

$$\mathrm{Var}[\partial_\mu \mathcal{L}(\boldsymbol{\theta})] \leq F(n), \quad \text{with} \quad F(n) \in \mathcal{O}\left(\frac{1}{b^n}\right). \tag{3.4}$$

In the above, the variance is specifically upper bounded to a function $F(n)$, which decays as $\mathcal{O}\left(\frac{1}{b^n}\right)$ with some constant $b > 1$. The term $\mathcal{O}\left(\frac{1}{b^n}\right)$ explicitly encodes exponential decay in n, since b^n grows exponentially, with $1/b^n$ decaying exponentially. This implies that the landscape has become effectively flat, since gradients vanish as n increases [51].

Barren plateaus occur in VQCs that are deep or highly expressive - where expressibility is defined as the circuit's capacity to generate diverse quantum states through parameterised gates and entanglement operations [8, 28]. A quantum model's expressive power depends on both its number of trainable parameters and its ability to generate diverse quantum states

through gate operations [8]. More parameters can increase expressibility, but the circuit's entangling gates and gate arrangement are equally important [28].

This increased expressiveness is specifically true for many types of layered circuit architectures commonly used in VQCs used for QML applications. In these settings, randomly initialised parameters $\boldsymbol{\theta}$ in deep circuits with many parameterised gates lead to the following traits characteristic of barren plateaus [5, 29, 52] as number of qubits n increases:

1. The mean of the loss function approaches a fixed value ($\mathbb{E}[\mathcal{L}(\boldsymbol{\theta})]$ to c (constant value)).

2. The variance of the loss function approaches zero ($\mathrm{Var}[\mathcal{L}(\boldsymbol{\theta})]$to 0).

3. The mean and variance of the loss function gradient both approach zero ($\mathbb{E}[\partial_\mu \mathcal{L}] = 0, \mathrm{Var}[\partial_\mu \mathcal{L}] \in \mathcal{O}(1/b^n))$.

This occurs because the resulting VQC behaves like an approximate 2-design, which is a distribution that mimics the statistical properties of Haar-random unitaries [5, 27, 52]. In these circuits, the large number of gates and qubits causes output states to become indistinguishable from random quantum states [5, 29]. As a result, the values of the measured observables centre around a constant average value, with little variance [53]. This implies that, despite changing the input data, the quantum model begins to predict the same value, and the gradients of the loss function lack useful information for training [52].

Since the optimisation landscape is functionally flat and featureless, there is little directional information available for optimisers, consequently making optimisers struggle to escape these regions [53]. Empirically, this has been shown to require an exponential increase in the number of measurements or circuit evaluations required to accurately estimate gradients, which can make convergence extremely slow or even impossible [10, 53]. The scaling nature of this problem is fundamentally important for the practical success of quantum machine learning and is uniquely challenging to handle, with many studies focusing on understanding its causes and the development of mitigation strategies [5, 10, 27].

3.1.3 Exponentially Many Local Minima

In contrast to barren plateaus (which occur in deep circuits), VQCs face an opposite limiting case, where shallow circuits exhibit exponentially many local minima in the loss landscape. This issue is present in shallow or under-parameterised circuits which avoid the flat gradient problems seen in barren plateaus, instead exhibiting highly non-convex loss landscapes with a large number of suboptimal local minima. In this regime (under-parameterised), only a superpolynomially small fraction ($1/n^k$) of the local minima lie near the global minimum [6, 54, 55]. Since most of these exponentially many local minima are far from the true solution, the bumpy landscape is swamped with poor quality traps [6]. This happens because these shallow circuits have a lower number of gates and can only explore a limited portion of the parameter space. As a result, each gate in the circuit has a larger effect on the output, creating interference that manifest as many small dips and bumps in the landscape [37]. This problem can be thought of as the other extreme case with barren plateaus when considering optimisation in quantum settings.

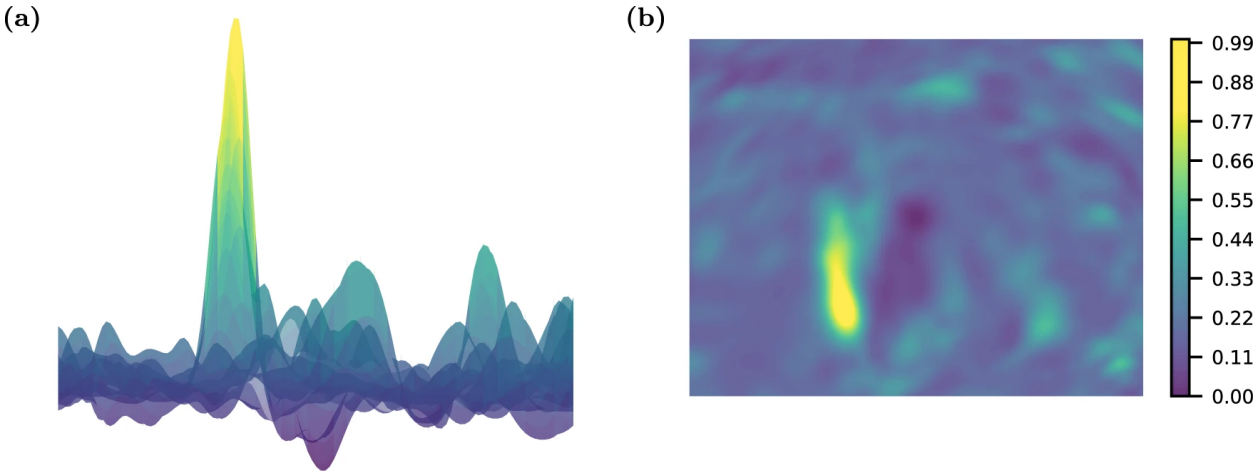

FIGURE 3.2: Bumpy loss landscape showing exponentially many local minima in under-parameterised regime (from Anschuetz and Kiani's Fig. 1 [6])

It is demonstrated that some classes of quantum neural networks (QNNs) can be constructed in a way that the loss function contains an exponential number of local minima in the number of parameters [37]. You and Wu [37] showed that for certain types of quantum neural networks, the number of local minima in the loss landscape can grow exponentially with the number of parameters. Specifically, for generic QNN architectures - which have no built-in structures for specific problems - it is possible that the optimisation landscape has 2^p local minima, with p as the number of parameters. With all these local minima, all but one $2^p - 1$ are spurious, meaning that they are not the global minimum and give worse performance for optimisation by creating trap minima [6, 37].

In small parameter spaces, the optimisation algorithm must navigate a large amount of local traps where only one is the global minimum. This result shows that having exponentially many local minima in the optimisation landscape can still be difficult to navigate - as the classical optimiser can easily get stuck in one of the many bad minima [6, 37]. The high number of spurious solutions implies that both gradient-based and gradient-free optimisers may become stuck in poor minima, which can prevent the optimiser from converging. Anschuetz and Kiani specifically refer to this as "swamped with traps," finding that these shallow VQC circuits often exhibit highly non-convex landscapes overcome with poor minima pitfall traps [54].

Due to this, the optimiser can encounter many possible suboptimal solutions, where each solution is a different poor fit of the target data, leading to exponentially many spurious local minima [37, 54].

Recent literature has unified the two landscape challenges of barren plateaus and exponentially many local minima through a theoretical framework. Researchers use different random process approximations to study statistical properties used for analysing the loss function of high-dimensional quantum models. The Wishart Hypertoroidal Random Field (WHRF) is one such process over a periodic parameter space, and is used to model the loss

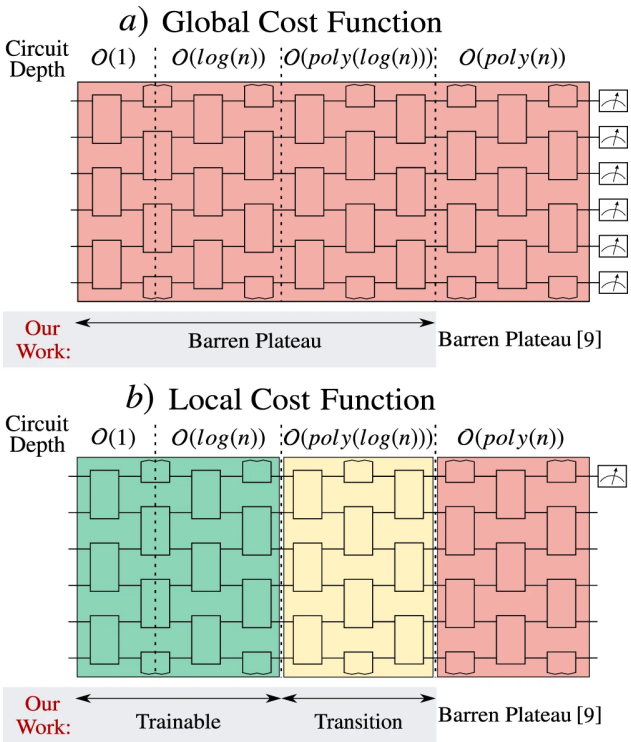

Figure 3.3: Local cost/loss function mitigation strategies for barren plateaus (from Cerezo et al. Fig 1. [29])

function of a variational quantum circuit. This Wishart process is capable of describing landscapes of flatness in deep, expressive circuits as well as the bumpy, trap-filled nature of shallow, under-parameterised circuits within the same statistical model. The theoretical framework utilises random matrix theory with Wishart distributions in order to approximate the structure of the loss function in these high-dimensional quantum models [55].

3.1.4 Existing Mitigation Strategies

Various methods exist for mitigating the effects of quantum optimisation problems, such as barren plateaus or exponentially many local minima.

For barren plateaus, the main strategies for mitigation centre around the preservation of gradients as the number of qubits increases. This is manifested through local loss functions [29] to avoid global flat gradients, specific circuit architectures to lower expressiveness [27, 56, 57] and even layer wise training to mitigate the effects of deep circuit training [58, 59].

Mitigating exponentially many local minima often revolves around increasing circuit expressivity, since shallow circuits tend to over-fit data [6, 54]. A technique called loss smoothing can be employed to help reshape the landscape [60], while multiple restarts can increase the chance of the optimiser finding improved minima [61, 62].

Circuit architecture and problem specific design is also a valid strategy to reduce optimisation difficulty. A summary of different circuit architectures, studying the performance of

each one when faced with barren plateaus or poor minima, is summarised in Table 3.1 [6].

TABLE 3.1: The untrainability of variational quantum algorithms (from Anschuetz and Kiani Table 1. [6])

Result	Dimension	Locality	Depth	Worst case?	Barren plateaus?	Poor minima?
Ref. [5]	d	2	$\Omega(n^{\frac{1}{d}})$	✗	✓	?
Ref. [29]	1	2	$\omega(\log(n))$	✗	✓	?
Ref.[63]	d	2	$\omega(\log(n)^{\frac{1}{d}})$	✗	✓	?
Ref. [64]	N/A	n	$\Omega(1)$	✗	✓/✗	✓/✗
Ref. [37]	d	2	$\Omega(1)$	✓	?	✓
Their [6] results	d	2	$\Omega(1)$	✗	✗	✓

Thus far, we have talked about challenges in optimising quantum circuits. In the next section, we focus on the classical optimisers that are used in this procedure, and how different optimisation strategies - particularly gradient-based and gradient-free methods - respond to the unique structure of variational quantum landscapes.

3.2 Optimisation Approaches in VQAs

In this section, we will review two major families of optimisation algorithms: gradient vs. gradient-free. While the literature mainly relies on gradient-based methods, we additionally review properties of gradient-free methods that could help towards building better optimisation protocols for QML.

3.2.1 Gradient-Based Methods

Various types of gradient-based methods exist for optimising variational quantum circuits, but the most common type is gradient descent algorithms. These methods are deterministic where the optimisation of variational quantum circuits depends on analytic or numerically approximated gradients of the loss function, producing the same update given the same initial parameters. Gradient descent minimises loss functions by iteratively updating parameters θ in the direction of steepest descent. Within each of these steps t, it calculates the gradient indicating the direction of steepest ascent $\nabla\mathcal{L}$ and updates parameters via $\theta_{t+1} = \theta_t + \nabla\mathcal{L}$, where step size (learning rate) is controlled [19]. Deterministic versions use exact gradients from all data, while stochastic variants (SGD, Adam) estimate gradients from random subsets for efficiency.

In quantum systems, the parameter shift rule allows for the exact calculation of gradients for loss functions which depend on expectation values by evaluating the circuit at shifted parameters $\theta \pm s$ (where s is fixed and often $\frac{\pi}{2}$ for Pauli rotations). The parameter shift rule leverages principles of quantum circuits, where the partial derivative of the quantum model can be expressed as a linear combination of quantum model functions,

$$\hat{\partial}_\mu \mathcal{L}(\theta) = \frac{\mathcal{L}(\theta_j + \pi/2) - \mathcal{L}(\theta_j - \pi/2)}{2} \tag{3.5}$$

In this example, the shift is $s = \pi/2$ and the loss function evaluations $\mathcal{L}$ are from the same circuit at previous parameters θ_j [31, 65, 66]. The other approach is the finite-difference method, which estimates gradients numerically by evaluating the circuit at parameters adjusted by a finite step size, with common methods being central-difference or forward difference. The central difference version computes the gradient for each parameter by perturbing its value by a finite number h. The simplicity of this method comes at a loss of its accuracy due to quantum shot noise and step size trade-offs, making parameter-shift rules almost always preferred in QML as this is an estimate of analytic gradients [65]. These methods provide clear deterministic updates to gradient-based optimisation, and are ideal for convex loss landscapes. Convergence is guaranteed here, since traversal of a loss landscape in a convex system is akin to rolling a marble in a bowl - where the global solution is located at the bottom. However, this convexity is rarely seen in variational quantum circuits, limiting deterministic convergence guarantees, but motivating the use of more complex descent optimisers that introduce stochasticity.

Stochastic gradient descent (SGD) methods are widely used in variational quantum circuit learning, being particularly useful when full expectation value gradients become computationally unreasonable. Unlike deterministic gradient descent methods, stochastic gradient descent methods estimate the true gradient by computing partial derivatives on random subsets of the training data (mini-batches) rather than the full dataset [23]. This method introduces stochasticity through data sampling, benefiting optimisation due to computational efficiency, faster convergence and improved resilience to local minima [67]. A popular extension of SGD is the Adam (Adaptive Moment Estimation) optimiser [25], which uses adaptive learning rates and momentum to improve model convergence. Adam combines momentum terms - exponentially decaying averages of gradient and squared gradient - to accelerate convergence in directions with constant improvement, with normalised updates on gradients per parameter [25]. The adaptive nature of this method helps in navigation especially in complex quantum landscapes and dampens noisy regions. Both SGD and Adam heavily rely on the learning rate chosen for a given problem, as values too small can cause stagnant learning, but values too large can overshoot solutions [38, 68].

Unlike deterministic gradient descent methods, stochastic quantum gradient descent methods estimate the gradient using measurement shots - repeated, stochastic quantum measurements that approximate expectation values, which introduces stochasticity into the optimisation process [38]. This process is beneficial as it avoids the loss of calculating all expectation values, averages the effects of shot noise, and matches quantum hardware as measurements in a quantum system are inherently stochastic.

One of the largest limitations of gradient-based methods is the computational loss of calculating gradients when the number of circuit parameters and circuit depth grows [38, 65]. This can become increasingly expensive, with many circuit evaluations (shots) being needed to estimate gradients with sufficient precision - with even more needed in noisy settings. Gradient-based methods struggle with navigating barren plateaus and fail to converge effectively due to poor parameter updates [29]. Some landscapes can also exhibit exponentially many local minima, as discussed in the section above, where gradient-based methods can get stuck within suboptimal minima due to a low learning rate [6, 37]. These issues all contribute to the effectiveness of gradient-based optimisers in QML, particularly when circuits

scale with qubit count.

3.2.2 Gradient-Free Methods

Gradient-free optimisers are an alternative method of optimisation which does not rely on derivative information. These methods are used when gradient calculations cannot be trusted; when loss functions are not analytic and gradients cannot be computed, or to deal with loss functions that are pathological for gradient-based counterparts [69]. Gradient-free optimisers encompass a broad range of methods, including direct search like Nelder-Mead [40], evolutionary/population like particle swarm optimisation (PSO) [70] or sequential model-based like Bayesian optimisation (BO) [71, 72]. For variational quantum circuits, there is some debate on whether problems like barren plateaus or exponentially many local minima that affect gradient-based methods are also an issue for gradient-free methods.

In this section, we focus on direct search, gradient-free optimisers. Direct search methods are a class of gradient-free optimisers that rely on loss function evaluations to explore the optimisation landscape - as they directly search for the optimal solution. These gradient-free methods are practical for use in quantum variational circuits as they perform well in systems where the objective function is non-differentiable, noisy or computationally expensive [69, 73]. The independence from gradient information makes direct search methods appealing for difficult optimisation landscapes such as barren plateaus and exponentially many local minima. Examples of direct-search include simplex methods such as Nelder-Mead [40], Powell's method [74] and COBYLA [41], as they are built on simple, geometric search theory.

The three direct search optimisers mentioned above align with optimisers that have been specifically used in quantum machine learning [10, 75]. Nelder-Mead is a simplex-based method that uses a geometric shape (simplex) made up of points in the loss landscape, it traverses the landscape through operation updates, which are reflection, expansion, contraction or shrinking [40, 76, 77]. This allows the simplex to explore the landscape without any gradient information, updating points via the following rule [41]:

$$\mathbf{x}^{\text{new}} = -\theta \mathbf{x}^{(l)} + (1+\theta)\frac{1}{n}\sum_{j \neq l} \mathbf{x}^{(j)} \tag{3.6}$$

Where $\mathbf{x}^{(l)}$ is the worst performing point within the convex hull of the simplex. Other gradient-free optimisers such as Powell's method take a different approach as a direction-set method, where it explores along line vectors without building a model of the loss function; instead, it moves directionally to identify minima [74, 78]. COBYLA (Constrained Optimisation BY Linear Approximation) is somewhat of a combination of the last two gradient-free methods as it is a simplex-based method, it then replaces the worst performing point by extrapolating and travelling along the line in the direction of the slope. It steps along this direction and stays within the trust region maintaining a linear approximation of the objective function - then the next trust region is created around the new point and the process is repeated, ensuring convergence [10, 41, 79].

These three optimisers were directly compared against each other and a basic gradient descent algorithm in the work done by Arrasmith et al. [10] on gradient-free optimisers in barren plateaus. Their empirical findings showed that despite their independence from

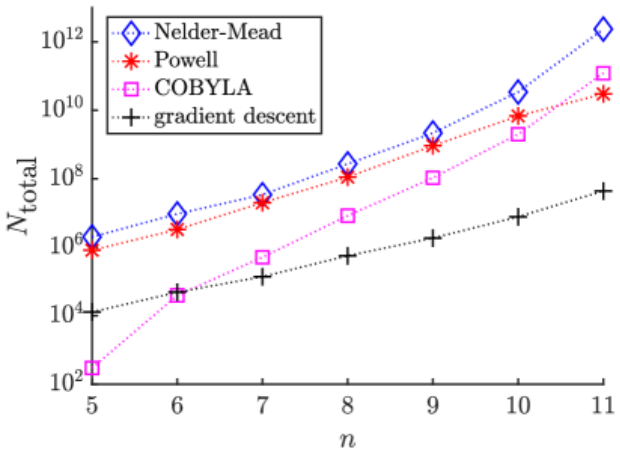

Figure 3.4: Comparison of gradient-free and gradient descent methods for the total shots required to reach a threshold 0.4 loss function value as the number of qubits increases (from Arrasmith et al. Fig 3. [10])

gradients, the gradient-free algorithms discussed above all suffered from the same exponential scaling issues that gradient-based optimisers do, this is seen in Figure 3.4. Numerical simulations conducted on the same circuit showed that the loss difference precision vanishes in systems with barren plateaus as the number of qubits increases. The metric on which this was evaluated was the median-total shot number necessary to reach a loss threshold in each system, which showed that with increasing system size, the number of shots increases exponentially and exceeds that required by standard gradient descent. This is directly linked to the number of circuit evaluations required by default in gradient-free methods, as a simplex must evaluate all the points of the convex hull, whereas a gradient-based method must only calculate one.

Traditional approaches like Nelder-Mead have been extended into general direct search methods that evaluate loss functions in multiple directions without relying on geometric concepts. Classical gradient-free algorithms, such as Nelder-Mead, COBYLA, Powell used in Arrasmith et al. [10], represent approaches where search directions are based on geometric operations such as reflection and contraction. In contrast, contemporary algorithms for direct search use sophisticated techniques to construct these search directions. When multiple search directions are evaluated simultaneously, at least one search direction is guaranteed to be close to the true descent direction [69]. These modern direct search methods can be contrasted with gradient-based methods, which only search along one direction by definition. Under Lipschitz continuity and stochastic sampling, where convergence has been formally proved [69, 80, 81]. Despite this provable convergence, the main drawback for gradient-free algorithms is that they may require more loss function evaluations and their operational robustness has not been well-characterised compared to gradient-descent methods.

Thus far, we have reviewed gradient-based optimisers that are typically used to train quantum circuits and discussed potential advantages and disadvantages of gradient-free alternatives. In the next section, we will introduce techniques that could help build a more efficient gradient-free optimisation protocol for variational quantum circuit applications by

reducing the cost of evaluation of the loss function.

3.3 Stochastic Optimisation Relevant to this Thesis

In this section, we focus on two techniques that aim to reduce the resource cost of optimisation for QML. The first technique argues that every gradient calculation in QML is stochastic, and therefore represents a way to reduce circuit evaluations. The second technique combines stochastic sampling with subspace methods. We will use both of these techniques to propose an efficient gradient-free protocol for QML in Section 4.5.

3.3.1 Quantum Stochastic Gradient Descent (QSGD)

An inherent property of quantum systems is that every expectation value calculated from the quantum circuit is stochastic. Each evaluation of the loss function or its corresponding gradient is affected by quantum measurement noise, which creates variance in the circuit outputs used in optimisation. This is different from classical algorithms - where measurement and gradients are usually deterministic [29].

Sweke et al. [38] formalises this idea by developing a method that uses stochastic gradient descent with a gradient estimator. They show that in a quantum setting, QSGD utilises this stochasticity to perform optimisation, even when gradients are estimated from a small number of measurements. A requirement of this method is that the gradient estimator must be unbiased. An unbiased gradient estimator means that the expectation value of the estimated gradient must equal the true gradient,

$$\mathbb{E}[g^{(t)}(\boldsymbol{\theta})] = \nabla\mathcal{L}(\boldsymbol{\theta}). \tag{3.7}$$

This property allows the optimiser to move in the correct direction towards the solution, despite having noise in each optimisation step. The key finding from this paper was that even a single shot gradient estimate can usefully train the model, this discovery significantly reduces the resource loss per iteration of the optimiser [38].

Since gradients in quantum systems are estimated through repeated circuit evaluations, each measurement introduces inherent stochasticity into the optimisation process due to quantum noise. Sweke et al. [38] demonstrated that meaningful optimisation can be achieved with even a single-shot gradient estimate, provided that the estimator is unbiased. This logic follows from convergence theory from classical stochastic optimisation, which states unbiased gradient estimates are sufficient for convergence - even when estimates are noisy or have high variance [82]. In practical usages of QML, it has been shown to significantly reduce the number of quantum measurements that are needed for each optimisation step [65], this allowing VQCs to be more computationally efficient.

Sweke et al. [38] also introduces the concept of doubly stochastic gradient descent (DSGD). Here, a second layer of stochasticity can be introduced over the shot noise, by sampling over different terms in the loss function. This can be particularly useful in quantum systems where the loss function is a sum of the expectation values of the quantum observables [1, 66]. Randomly sampling terms from the loss function sum for each iteration, as well as

using a small number of shots to estimate gradients, the overall loss per update will be significantly reduced. Convergence is still guaranteed under unbiased conditions, despite the added noise. This sampling approach allows for a more scalable quantum optimisation solution, as the computational resources can be reduced per training step.

Currently, stochastic sampling for gradient-free methods has no robust framework like QSGD, especially when considering quantum systems. The success of stochastic quantum gradient descent estimators suggests a promising direction to build quantum gradient-free methods, as pursued in Section 4.5.

3.3.2 Stochastic Sampling for Gradient-Free Simplex Methods

Traditional gradient-free simplex optimisers like Nelder-Mead [40] and COBYLA [41] are by construction deterministic methods, with predefined geometric rules for searching. As discussed earlier, Nelder-Mead uses simplex geometry transformations with reflection, expansion, contraction, and shrinkage [76], while COBYLA builds linear approximations using deterministic updates constrained within the defined trust region [79]. However, contemporary simplex-inspired methods have been introduced where random sampling of subspaces is utilised for direction selection. These algorithms utilise stochasticity through these smaller random subspaces at each iterative step to determine the next search direction [83, 84]. This enables more diverse exploration of the optimisation landscape, aiding in preventing the optimiser from getting stuck in poor regions due to limited directional coverage. These directions are chosen from lower dimension subspaces, enabling efficient exploration without full dimensional simplex updates [83, 84]. Unlike deterministic methods like Nelder-Mead and COBYLA, this stochastic approach to direction selection forms a foundation for a more scalable and effective search algorithm, especially in non-convex loss landscapes.

In high-dimensional parameter spaces, these contemporary methods offer a reduction in computational cost of directional search from random sampling. In deterministic simplex methods, the number of function evaluations per iterative step can grow with the parameter space dimensionality, as usually the simplex spans over the whole parameter space [40]. Stochastic simplex methods overcome this by restricting their simplex updates to random lower dimensional subspaces, only evaluating function values on a small number of selected directions per step [83]. The improvement on deterministic models also inherently improves computational efficiency, as these stochastic direction selections help break cyclic behaviours - the randomisation preventing the optimiser from repeating poor decisions [84]. In the context of QML, these properties are desirable, as fewer function evaluations translate to a lower number of shots - this approach lowering quantum resource usage. This reduced subspace technique has been shown to benefit both computational resource usage and convergence criteria.

Unlike Nelder-Mead and COBYLA, contemporary stochastic simplex methods have been proven to have formal convergence guarantees [85], even in non-convex and high-dimensional landscapes. As discussed in the section above, the traditional deterministic simplex methods lack formal convergence guarantees for higher dimensional landscapes, with theoretical convergence only possible in smooth and convex landscapes [73, 80, 86] By introducing stochastic direction selection via random subspace sampling, these modern direct search methods ensure

probabilistic descent, guaranteeing convergence. The concept of probabilistic descent guarantees convergence because it ensures with some minimum probability, each step decreases the loss function sufficiently. Essentially, the stochastic property of the method ensures that the algorithm will adapt and eventually make progress, even when steps fail [85, 87].
The success of stochastic sampling in both gradient-free optimisation and quantum gradient estimation in QSGD encourages a united strategy for proposing a new direct search optimisation technique that reduces computational cost in variational quantum circuits. In Chapter 5, we explore how direct search with random subspace sampling can use quantum expectation values in the finite-shot limit, as inspired by [38]. We will determine if our protocol retains convergence guarantees while lowering the number of quantum circuit evaluations in empirical benchmarking against gradient methods. With this resource reduction, we anticipate that our gradient-free method may overcome the disadvantage of gradient-free methods empirically outlined within Arrasmith et al. [10].

4
Testbed for QML Simulations

Prior to comparing optimisation algorithms for QML, we first require a well-understood problem to serve as a reliable testbed. In this chapter, we focus on data re-uploading circuits (DRCs) as our quantum machine learning model of interest. The motivation for choosing DRCs is that they are an asymptotical universal model for classification and regression [11]. By asymptotically universal, we mean that DRCs can approximate any continuous function, allowing for perfect learning in noiseless regression tasks. This property makes DRCs ideal for controlled benchmarking, allowing us to isolate errors that arise from the optimisation process or scalability, rather than model expressivity. By model expressivity, one loosely means the capacity of a model to represent or approximate a chosen target function, given sufficient parameters and circuit depth. These guarantees are not available in other QML settings.

In this chapter, I developed the framework used to simulate data re-uploading circuits. I extended Connor van Rossum's code base from single-qubit to multi-qubit circuits, and developed an approach for entangling qubits based on Ref. [11]. I carried out all simulations using the UQ high performance computing (HPC) Bunya cluster [88], analysed data, and proposed methods to diagnose and characterise performance including the use of the coefficient of determination (R^2), loss monitoring and convergence checking. I also introduced parallelisation and created SLURM (job scheduler) scripts to enable efficient large scale simulation.

4.1 Data Re-uploading Circuits (DRCs)

Data re-uploading circuits (DRCs) are a class of variational quantum circuits designed for quantum machine learning, being most suited for classification and regression learning tasks [31]. Their defining feature are the alternating training and data encoding layers, where each layer consists of parameterised quantum gates and two-qubit entangling gates [11].

This layer-wise structure allows for the circuit to repeatedly encode the same classical data – this being the concept of data re-uploading.

By uploading input data multiple times throughout the circuit, DRCs can represent increasingly complex non-linear relationships between inputs and outputs, which enhances expressivity without requiring overly deep or wide circuits [11, 31]. These DRCs are flexible for model expressivity, where circuit depth and width are both easily tuned. These properties make DRCs particularly effective for modelling continuous target functions, and therefore well suited for regression QML tasks.

This data re-uploading circuit (DRC) architecture forms the core quantum circuit model used in this work for learning regression tasks [31]. In practice, data re-uploading can be implemented using either multiple qubits or a single qubit with multiple gate layers. For these reasons, the DRC ansatz is well suited for benchmarking scalability in QML.

The DRC architecture relevant to this thesis is a short-depth, multi-qubit system, focusing on reproducing challenges in QML for the toy problem of learning a sinusoid - specifically a Fourier series. These DRC systems can encounter some key challenges: shallow architectures may exhibit exponentially many local minima [6], while scaling the number of qubits increases the chance to encounter barren plateaus [89]. There are three main components to enabling these types of circuits, shown as either a high-level schematic in Figure 3.1 or a detailed exemplar in Figure 4.1, with:

- Training single qubit gates: Composed of three single-qubit rotation gates (R_x, R_y, R_z) inputted with trainable parameters θ. The trainable parameters are periodic angles, $\theta \in \{0, 2\pi\}$, since the $R_{x,y,z}$ gates are single qubit rotation gates that apply the angle θ to rotate the qubit.

- Two-qubit entangling gates: Consisting of a ladder configuration of CNOT gates. Multiple layers of these entangling gates are added to enable a more strongly and efficiently entangled circuit [11]. In our circuit structure, a training block is considered as the combination of a layer of single qubit gates described above with a ladder of CNOT gates (with set sequences being alternated with different training blocks). An example of an extra training block is displayed in Figure 4.2, with the extra tunable hyper-parameter of number of training blocks N.

- Data uploading layer: Composed of a single rotation gate (R_x) inputted with classical data point x. The input data x is received via single-axis rotation gates, in our system, this is a single value of 1.

- Measurement: At the end of the circuit, on the first qubit of the system, the Z expectation value is recorded from the final quantum state. This is used within the loss function calculation for training the model.

In our implementation, training parameters θ_n are optimised to minimise the loss function between the circuit's output and the known target function. These architectural features make DRCs excel at regression tasks because repeatedly encoding the input data through layered quantum gates enables an enhanced ability to model complex continuous relationships [11, 90].

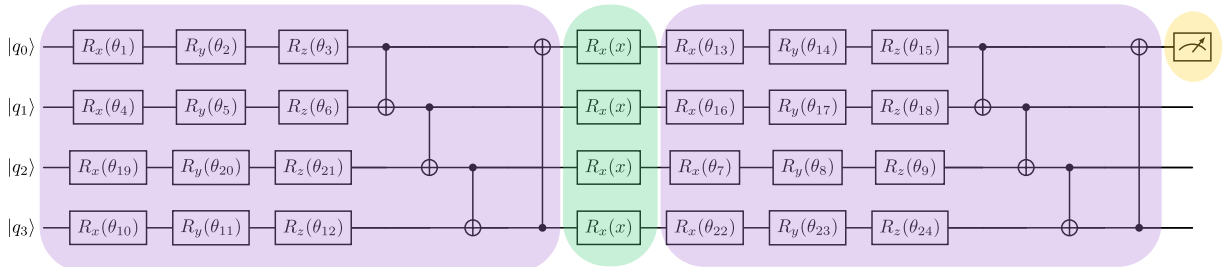

Figure 4.1: Circuit structure of a data re-uploading circuit, parameters θ_n are the trainable parameters, and x is a single classical data point. Entangling gates used are CNOT gates (dot is control, cross is target qubit), this is assembled in a ladder for maximal entanglement. Training layer section (highlighted in purple) comprises the set of R_x, R_y, R_z gates with θ followed by the ladder of CNOTs. The θ_n values are initialised randomly as a periodic value, then used for training after each iteration step. Data uploading layer (highlighted in green) comprises of R_x gates with input x. Measurement (highlighted in yellow) is at the end of the circuit only on qubit 0 ($|q_0\rangle$). This is a circuit with only one training block ($N = 1$). The training layer itself can be composed of multiple training blocks, illustrated in Figure 4.2.

With our knowledge of our chosen circuit architecture, we now introduce the toy problem of learning sinusoids, i.e. a truncated Fourier signal learned by a data-uploading circuit during supervised regression. The Fourier series provides a natural benchmark as a regression task, as it is a periodic function and analytically tractable, which allows for the complexity of this learning task to be systematically scaled with circuit size.

4.2 Fourier Series as a Learnable Target

The Fourier series is an effective choice for benchmarking quantum machine learning models for several key reasons:

- Expressivity: The structured pattern over multiple frequencies requires the model to learn both amplitude and phase relationships. This makes it a suitably complex problem for testing the capabilities of QML algorithms.

- Analytic tractability: The function is known with a well-defined analytic definition, making learning success easily measured against the target function.

- DRC compatibility: Previous literature has shown that DRCs with d number of qubits are expressive enough to represent d degree Fourier series, assuming sufficient circuit depth (training blocks) [11].

To evaluate whether DRCs can successfully learn high-dimensional regression problems, we use a complex-valued Fourier series as our target function. This target function is mathematically equivalent to a real Fourier series, and the output can produce a real-valued signal. A Fourier series formally represents a periodic function as a sum of frequency components,

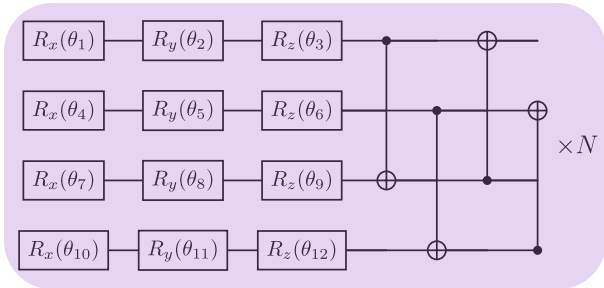

FIGURE 4.2: An example training layer with CNOT ladder of sequence 2. The training layer (highlighted in purple) is composed of many training blocks that alternate through sequences. This example starts at sequence 2 (CNOT ladder connects over one qubit cyclically), a normal training layer used within this work always begins at sequence 1 as seen in the above Figure 4.1. The variable N refers to the number of training blocks within a training layer, with 2 and 3 layers being tested within this thesis. This is important for understanding that this both adds extra parameters and entanglement based on the number of training blocks N included within the circuit.

defined as follows:

$$g(x) = \sum_{n=-d}^{d} c_n e^{-nix} \tag{4.1}$$

where d is the degree, and the coefficients in our system are:

$$c_0 = 0.0, \quad c_1 = c_2 = \cdots = c_n = 0.05 - 0.05i$$

These coefficients determine the amplitude and phase of each frequency component. For our regression tasks, the coefficients were chosen to ensure the output of the function is scaled and real-valued within the range $[-1, 1]$, aligning with the output range of quantum expectation values from measurement of the circuit. This range is valuable for our simple loss function of mean squared error (MSE) to be meaningful without requiring any post-processing.

To ensure that the DRC model is capable of exactly representing the target Fourier function, one can set the d degree of the Fourier series to equal either the number of qubits, or the number of layers [11]. We focus on the former regime, where for all simulations, the degree,

$$d \equiv n, \tag{4.2}$$

equals the number of qubits. We can then scale up the regression task easily by increasing the degree of the Fourier series and the number of qubits within the DRC. Capital N is a free parameter (we change it from $N = 2, 3$ in our empirical study)

This setup allows us to easily isolate and study optimisation challenges of a scaling system, as any failure to learn the target function must reflect issues such as barren plateaus, exponentially many local minima or optimiser inefficiency.

4.3 Usage of Adam as a Gradient-Based Optimiser

In this testbed, the Adam optimiser serves as a representative gradient-based method for training data re-uploading circuits. As introduced in Chapter 3, Adam combines stochastic

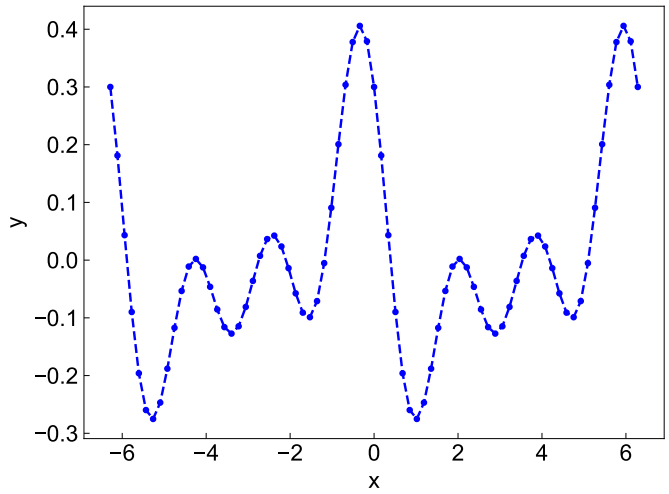

Figure 4.3: An example real-valued degree 3 Fourier series used as a regression target for a 3-qubit DRC. The function is constructed using constant complex coefficients $c_n = 0.05 - 0.05i$ for $n = 1, 2, 3...$, and $c_0 = 0.0$. These coefficients allow the series to be scaled to lie within the range $[-1, 1]$, aligning with quantum circuit expectation values. The resulting function is a smooth curve with periodic oscillations over multiple frequency components. DRCs with d many qubits are known to be capable of exactly representing a d degree Fourier series [11]. As such, this target function serves as an ideal benchmark for studying optimisation performance independent of model expressivity.

gradient descent strategies with adaptive moment estimation [25]. These features make it both effective and efficient, and as a result, it is widely used in both classical and quantum machine learning. The combination of adaptive learning rates and stochastic updates enables efficient learning and stable convergence, particularly in small or noiseless optimisation problems.

These two features of Adam can be summarised as follows: Mini-batch updates introduce stochasticity by computing gradients on random subsets of training data, rather than on the entire dataset at once. This variation is beneficial, as it can improve generalisation and help the optimiser avoid poor local minima [23]. Adaptive moment estimation adjusts the effective learning rate for each parameter individually by maintaining running averages of the gradient and its squared value. This allows Adam to automatically and adaptively scale updates based on the magnitude and variability in the gradients [25].

As an adaptive optimisation algorithm, Adam requires minimal hyper-parameter tuning to achieve accurate learning. The adaptivity of Adam is primarily controlled through a single hyper-parameter, the learning rate η. In our simulations, this value was fixed, allowing Adam to perform all internal adaptation automatically. This made Adam a practical choice for showcasing our testbed, as tuning this single hyper-parameter was sufficient to achieve good convergence behaviour. Adam's use of mini-batch stochasticity and adaptive learning rates enables robustness and generalisation across the optimisation landscape, making it computationally efficient for achieving accurate learning.

However, an important limitation of Adam is its sensitivity to small perturbations in gradient estimates. This occurs because Adam's adaptive update rules can amplify or mishandle noise in the gradients, leading to unreliable learning in noisy environments [91, 92]. This sensitivity contrasts with the quantum stochastic gradient descent (QSGD) method [38] discussed in Chapter 3, which is inherently robust to noise by design, but remains beyond the scope of this study.

In this thesis, Adam is used as the baseline optimiser for training DRCs in the QML testbed. We perform empirical simulations using the exact measurement regime based on density matrices as introduced in Section 2.1.5. In Chapter 6, we compare its performance with gradient-free optimisation methods under both exact and noisy finite-shot simulation settings, providing insight into its applications and limitations. We now proceed to characterise the learning scalability of DRCs trained with Adam, focusing on accuracy and computational resource requirements as circuit size increases.

4.4 Characterising Learning Scalability of QML Simulations

As previously established, data re-uploading circuits are capable of perfect learning of a Fourier series with the same degree as the number of qubits. In this section, we will evaluate how well the models can practically learn this function as the number of qubits increases.

4.4.1 Model Accuracy and Convergence Behaviour

Before introducing our training results, we briefly outline the simulation implementation. The data re-uploading circuit was constructed using the Qiskit Python library [93], and trained using the Adam optimiser. Adam's primary hyper-parameter is the learning rate η, which controls the step size during parameter updates [25]. Other parameters of the training setup include the number of optimisation steps, which determines how long training is performed, and batch size, which specifies how many samples are processed before each update [11]. This system also uses a mean squared error (MSE) loss function, which is a common loss function for supervised learning problems and was introduced in Section 3.1. This setup environment allowed for a simple process to be followed to run the supervised learning problem:

1. Initialise θ and define input x values for the circuit.
2. Construct the circuit based on the number of qubits n and training blocks N.
3. Execute the circuit and measure expectation values $\langle Z \rangle$.
4. Evaluate the loss function based on these trainable parameters θ and circuit output.
5. Input the loss function into the Adam optimiser initialised with learning rate η.
6. The optimiser updates the trainable parameters θ.

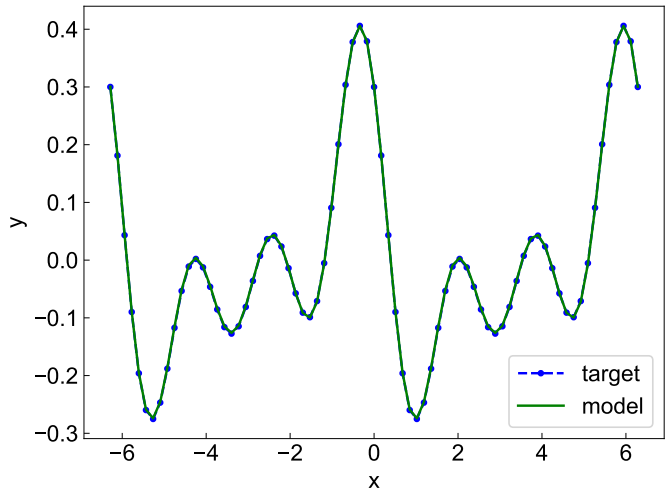

Figure 4.4: Trained 3-qubit data re-uploading circuit on a degree 3 Fourier series with near perfect learning ($R^2 = 0.9999$). The dashed blue line represents the target Fourier function, and the solid green line represents the trained model. This model was trained with 200 steps of the Adam optimiser with learning rate η set to 0.01 and a batch size of 25, with 2 training blocks ($N = 2$) used for this circuit. This shows that DRCs are capable of perfect learning in a practical simulation.

7. Repeat until the maximum number of steps is reached.

This overall process is later presented in detail as pseudocode in Algorithm 3 of Chapter 6, which outlines the interaction between the optimiser and the quantum circuit. These simulations were then executed on the UQ supercomputer - Bunya's high-performance computing (HPC) cluster [88].

The learning success of the model was measured using the following metrics: the loss function value and the coefficient of determination (R^2 value). These two metrics are directly correlated, but the R^2 value was the main metric for evaluating model accuracy. A perfect learning example is displayed in Figure 4.4.

After confirming that perfect learning is possible, we characterised the decay in performance of the supervised learning problem in DRCs as number of qubits increases, in the plot - Figure 4.5. As shown in Figure 4.5, DRCs trained with 2 to 6 qubits consistently achieve near perfect learning with high R^2 values (> 0.99) with an example of this perfect learning in Figure 4.4. However, performance significantly degrades for 7 qubit systems, where R^2 values average at 0.90. Since we know DRCs are capable of perfect learning, the drop in accuracy is not due to lack of expressivity, but rather an emerging optimisation problem.

To investigate the convergence behaviour of DRCs, the loss values and variance can be evaluated across training steps. The difference in loss functions for each system size is represented in Figure 4.6. In Figure 4.6, it is clear that the 7-qubit model struggles to converge, this qubit number being somewhat of a threshold for when perfect learning is no longer possible due to non-model expressivity issues. These results are consistent with known challenges in QML using variational quantum circuits, discussed in Section 3.1. The

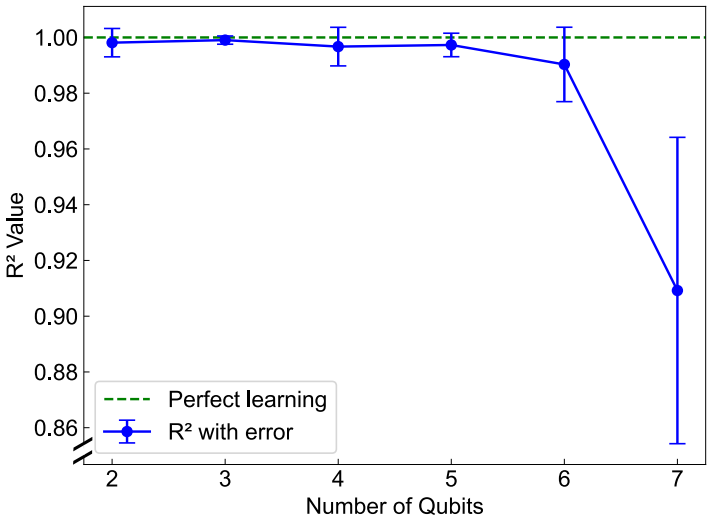

FIGURE 4.5: Comparison of model performance (R^2 Values) for an increasing number of qubits up to 7 qubits. The green dotted straight line indicates perfect learning at a R^2 value of 1.0. The blue data points are the averaged R^2 values of each system size with standard deviation error bars. All models were trained with 200 steps of the Adam optimiser with learning rate η set to 0.01 and a batch size of 25, each data point including 10 trials of the model. The number of training blocks used for these circuits is 2 ($N = 2$). DRCs achieve near perfect learning from 2 to 6 qubit systems, with the error bars including the perfect learning line. There is a sharp drop in performance at 7 qubits, indicating an issue in either the optimisation landscape or the optimiser itself.

stagnation in loss values and convergence could be due to a number of reasons, with the leading hypotheses being barren plateaus [5, 29], exponentially many local minima [6] or optimiser logic issues, or even lack of training steps. The high qubit count of 7 attributes to a high dimensional parameter space for the optimiser, where the loss landscape becomes increasingly flat and noisy - leading to the barren plateau problem. However, it's also possible that the shallow circuit - using one data re-uploading layer - leads to exponentially many local minima that interfere with convergence, as gradient-based optimisers like Adam are extremely susceptible to trap minima. This is a difficult question to answer, as it is not possible to simply look at the loss landscape itself as it is a high dimensional space, and if the problem is one with the optimiser itself, running simulations with more training steps or other hyper-parameter tweaks is resource consuming. This leads us to the next section, centring on the computational resource scaling of DRCs, with a focus on runtime, memory and number of parameters.

4.4.2 Computational Resource Scaling

In addition to learning accuracy, we characterised the resource costs of DRC simulations with scaling number of qubits. Alongside all the loss and R^2 data taken from the previous section's simulations, the runtime and memory data were recorded as metrics from Bunya's on board Jobstats job monitoring [88]. This allows for precise runtime and memory statistics

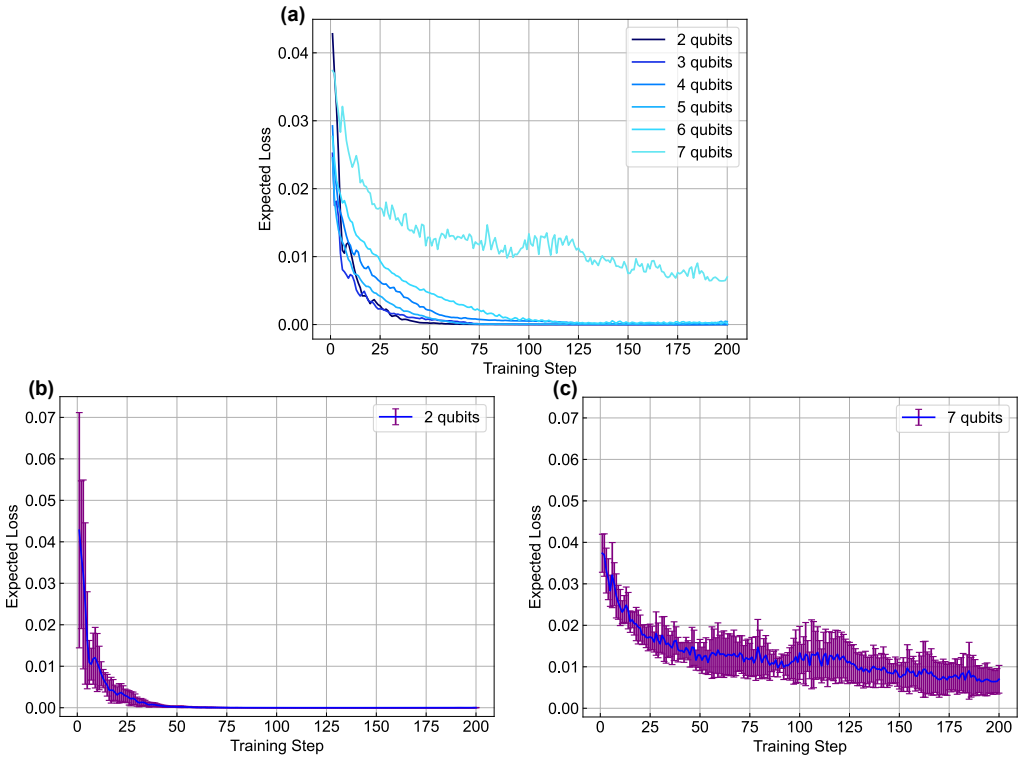

Figure 4.6: Training loss for data re-uploading circuits (DRCs) with increasing number of qubits. a) Expected loss over training steps for 2 to 7 qubit DRCs. Models with 2 to 6 qubits reliably converge to an accurate solution, while performance degrades for 7 qubit systems - not approaching near zero loss. b) Expected loss (blue) with variance error bars (purple) over 10 trials for a 2-qubit DRC. Though there is high variance at the beginning of training, the optimiser converges quickly with a low variance and loss. c) Expected loss versus training steps for the 7-qubit DRC. Despite DRCs being analytically confirmed to converge to a solution, the optimiser fails to consistently reduce the loss, with high variance throughout the training steps. The variance does not reach near zero, indicating the lack of convergence in 200 training steps. This non-zero variance suggests the model requires more training steps or indicating optimiser stagnation to a convergent solution.

as well as other important computational resources such as CPU & GPU usage as well as how many cores or gigabytes of memory were allocated to the job. For our purposes currently, only runtime and memory data are explicitly presented here. The runtime of simulations clearly increases non-linearly as number of qubits grows, with an exponential or polynomial trend [94, 95]. Though the assumption is that the runtime is an exponential trend, as the data was log-scaled with base e. This resulted in the data trend being a near straight line, and allowed for the linear trend to be extrapolated to 10 qubits. This projection estimates that a 9-qubit system simulation would take approximately 233.1 minutes to run a single trial. This however is a gross underestimation of the true runtime of one trial with 1325.8 minutes shown inFigure 4.8. This suggests a breakdown in simulation scalability when using the exact density matrix method, as the computational load cannot handle a 9-qubit Qiskit simulation, evidenced by the CPU-limitations running at 96% capacity. Other simulations

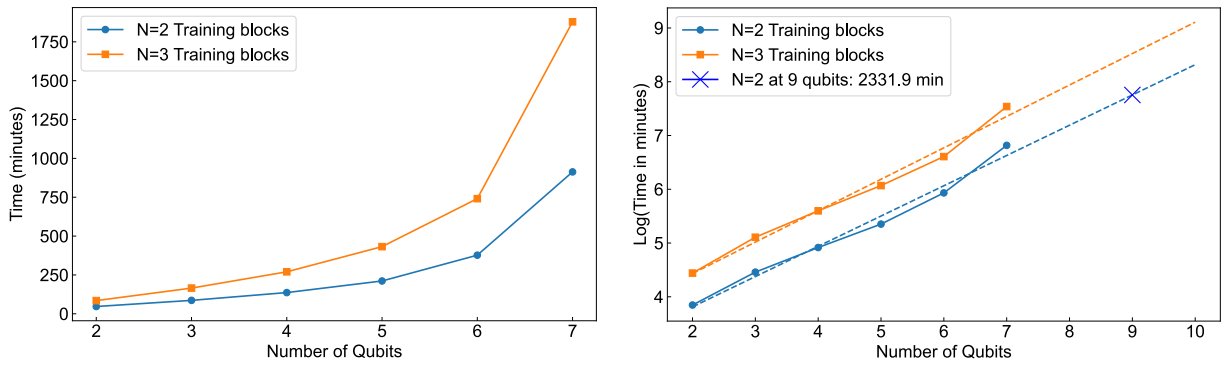

FIGURE 4.7: Simulation runtime as a function of number of qubits and circuit depth. (Left) Total training time for data re-uploading circuits (DRCs) with 2 and 3 training blocks for 10 trials of simulations. The runtime trend grows rapidly as the number of qubits increases, with deeper circuits, 3 layers, consistently requiring more time than 2 layers. At 7 qubits, the runtime growth becomes exponentially steep. (Right) Log-scaled version of the same data with a linear extrapolation to 10 qubits. The projected runtime for a 9-qubit DRC with 2 layers is 2331.9 minutes for 10 trials - estimating 233.1 for a single trial.

never reached this capacity, indicating a possible issue. With runtime being the key limiting factor of our DRC simulations, the memory usage of the system was also considered. The same simulations from 2 to 7 qubits had their memory data recorded as mentioned prior. This data is plotted within Figure 4.9.

The memory scaling data clearly shows inconsistent results, as the number of qubits seems uncorrelated with the memory usage. Memory usage in small scale Qiskit circuits is not large, and the data values average around 160 MB [93]. The runtime with the number of parameters is a useful metric, this being tested across 2 and 3 training block systems. Adding a training block is a simple way of increasing the number of parameters of the system as well as the number of entangling gates. The number of parameters is dependent on the number of entangling gates as it adds an extra set of rotation gates (R_x, R_y, R_z) totalling $p = 3 \times 2 \times n \times N$ many parameters, n being the number of qubits and N being the number of training blocks. As seen in Figure 4.9, there is a small correlation through the data in smaller qubit systems (2 to 3 qubits). However, it is obvious that the growing number of parameters increases runtime non-linearly [38]. This is due to the runtime of larger qubit systems being dominated by executing large qubit circuits, not processing the parameter space. The steeper trend for 3 training blocks could also be attributed to the larger number of entangling gates, with entangling correlations adding extra runtime.

This runtime issue is a growing problem and with current methods, it is unavoidable. However, this is where The concept of parallelisation can be used to lower the runtime of these complex simulations.

Parallelisation is the computational technique of dividing work evenly across computer processing unit (CPU) cores. This is usually implemented through running loops or other runtime consuming tasks simultaneously - significantly decreasing runtime. With the issue

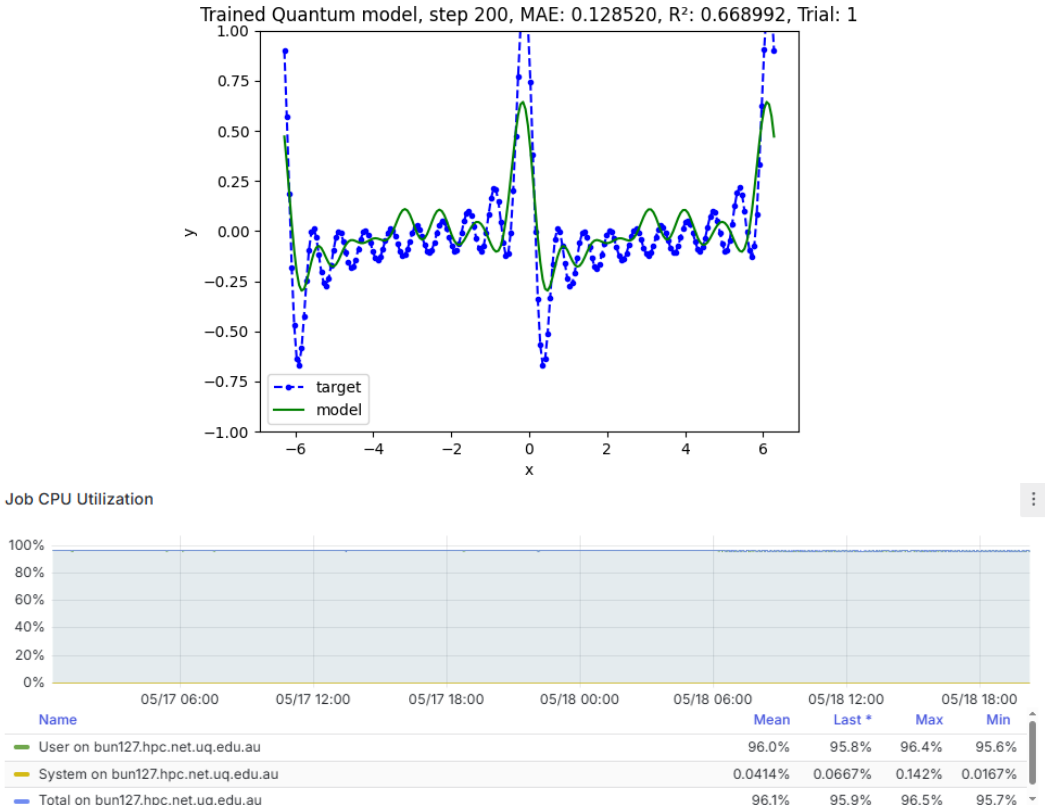

Figure 4.8: High qubit count simulation data. (Above) The results of one trial of the 9-qubit simulations, learning outcome is poor with a low R^2 value: ~ 0.67. This simulation data required 1325.8 minutes to achieve this learning outcome, this does not match the projected value of 233.1 minutes. Optimiser and circuit structure settings are kept the same as previously stated. (Below) Job statistics from this trial, displaying the CPU utilisation of the 9-qubit simulations. In this graphic, we can see that the CPU utilisation is at 96% on average throughout the simulation. This indicates that the simulation might have been CPU limited and that parallelisation is necessary.

of runtime clearly affecting our DRC simulations, parallelisation is considered a necessity for running higher qubit count simulations [96]. For the simulation set up, the simplest method of implementing parallelisation was through the loss function calculation. This is possible as each iterative step of the training process requires independent evaluations of the quantum circuit across different input points, allowing for simultaneous computation. The use of multiple CPU cores is implemented on HPC Bunya by allocating more than one CPU core to be accessed by our simulation script. The effectiveness of parallelisation was tested through small sample tests on different numbers of parallel cores. This is illustrated in Figure 4.10.

The empirical data recorded within Figure 4.10, shows that parallelisation is effective at reducing runtime for a simple DRC circuit, with at most a 46% reduction in runtime at 16 cores. Some denominations of cores perform poorly in comparison, namely, 2 cores

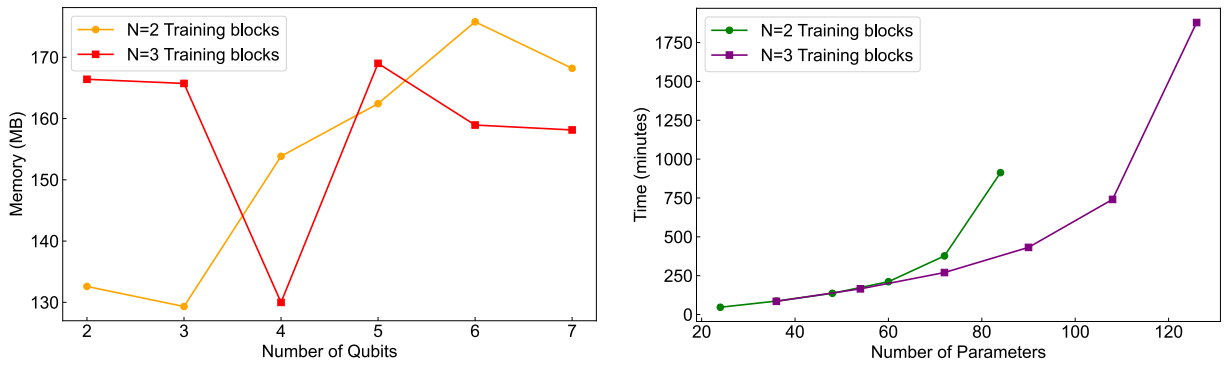

FIGURE 4.9: Other resource scaling of 2 to 7 qubit systems. (Left) Memory usage of the data re-uploading circuit (DRC) simulations in megabytes for growing system size. The memory usage of these simulations is low and remains manageable as qubits increase, averaging around 160 MB. This indicates that memory is not an exponential or polynomial scaling problem for Qiskit simulations [93]. (Right) Runtime versus the number of parameters of the DRC system. The number of parameters shows that parameterised gates make up the bulk of runtime in smaller simulations (2 to 3 qubits). However, as the system size increases, the runtime separates from direct correlation to the number parameters, evidence by the latter half of the plot. This is important to note, as runtime does not depend solely on number of parameters, but more on the number of qubits within the system [38, 94].

perform worse than the unparallelised case. This increase can be attributed to parallelisation overhead [97] and resource contention [98, 99]. Parallelisation overhead could impact performance for small scale tests, as the process of allocating the work to different cores might not outpace the benefit of simultaneous computation. Resource contention could also be at play, with the simple parallelisation implementation fighting over which cores are used. For example, a single core could be double allocated to two tasks, this easily attributing to the increase in runtime. Any other increases in runtime are thereby explained by one of the two effects (overhead or contention). Other factors to consider when employing parallelisation to the simulations include congestion [99] - as adding too many cores can slow down the runtime as only a small portion of work can be divided, leading to more parallelisation overhead than runtime improvement. The number of cores used can also be considered a computational resource, as requiring more CPUs to be allocated to one job can be considered resource intensive. Overall, parallelisation is an effective method of reducing runtime without needing structural changes to the circuit and preserving DRC learning accuracy.

So far we have built the software infrastructure with naive parallelisation capabilities to characterise the performance of DRCs using the gradient-based optimiser Adam. In the next section, we propose a new method that aims to reduce the cost of functional evaluations while optimising in a high dimensional parameter space. We test our proposed method using the software that we have validated for correctness in this section, and the baselines for runtime and memory will be used in future investigations.

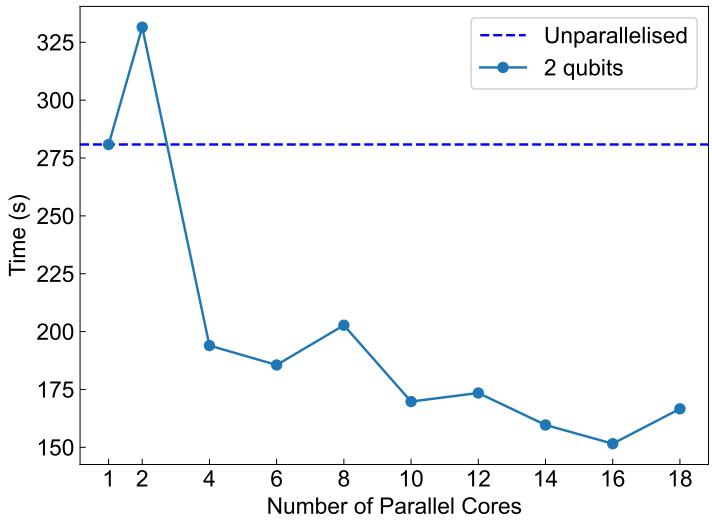

Figure 4.10: Parallelisation effectiveness in reducing simulation runtime as the number of parallel cores increases. The dashed blue line indicates the unparallelised baseline at 1 core, with a simulation runtime of about 280 seconds. The light blue data points indicate the runtime data of each simulation with added CPU cores. The simulations tested were 2-qubit 2 training block circuits, as these have the smallest runtime, allowing for this data to be recorded quickly. The data generally follows a downward trend, with some bumps in the data at certain cores. At 2 cores, the simulation has a longer runtime - this attributed to parallelisation overhead [97] and resource contention [98, 99]. 16 cores were empirically deemed to reduce the runtime the most with a 46% time reduction.

4.5 Towards Scalable Gradient-Free Optimisation

Our empirical results show that optimisation becomes unreliable at 7-qubits and beyond, despite data re-uploading circuits being theoretically expressive enough to learn the target Fourier series. We hypothesise that this is due to a difficult optimisation landscape such as the two main trainability issues discussed in Chapter 3. Gradient-based methods struggle in these landscapes, resulting in stalled learning and incomplete convergence, due to the unreliability of gradients in these regimes. However, as the name suggests, gradient-free algorithms do not rely on gradients of the loss function, this provides a small implicit resistance to difficult landscapes.

In this section, we primarily investigated Adam's learning accuracy and computational performance with scaling qubit circuits, these simulations serving as representative performance for gradient-based methods. To fairly complete this comparison between gradient-based and gradient-free optimisers, we must choose a representative gradient-free method. As existing methods such as Nelder-Mead, Powell and COBYLA are deterministic and were shown to be vulnerable to barren plateaus in Chapter 3, we turn to selecting a stochastic gradient-free method. Simulation using this method allows us to compare learning accuracy using our DRC testbed.

The next chapter focuses on the design of this stochastic gradient-free method utilised

for direct comparison between optimisers. We also introduce a theorised doubly stochastic gradient-free method inspired by QSGD [38] using finite-shot measurements, motivated by the promise of reduction of computational resources.

5

Direct Search for Variational Quantum Models

In this chapter, we first introduce a recent development in gradient-free optimisation literature, which is a direct search optimiser that stochastically chooses to optimise parameters in reduced subspaces. The description of the algorithm and its convergence properties are summarised from existing literature in Section 5.1. Subsequently, we extend this algorithm to loss functions for QML in Section 5.2.1 and to the finite-shot limit in Section 5.2.2.

The overall direction for this chapter was initially proposed by Riddhi Gupta. My contribution was to analyse the convergence guarantees of the direct search in reduced subspaces algorithm, and to investigate the extent to which these guarantees are impacted by quantum measurements in Section 5.2.2. I derived unbiased estimators of the loss under shot-based measurements and examined how sampling noise influences the implementation of the optimisation method.

5.1 Reduced Subspace Direct Search Algorithm

Direct search is a class of gradient-free optimisation algorithms designed to navigate a loss landscape through function evaluations alone, with no reliance on gradient information. These methods operate by comparing loss values at different points around the current position. A standard direct search protocol tests candidate points in random directions from the current position and moves to the best point found. For each successful step, the step size is increased to accelerate progress, while unsuccessful steps decrease the step size for exploration. This process can be visualised through Figure 5.1.

This basic probabilistic variant of direct search is a known algorithm, functioning through stochastic sampling of directions in the full parameter space. The novel reduced subspace direct search algorithm improves upon this by instead randomly choosing a reduced subspace at each iteration. Rather than picking random directions in the high dimensional parameter space, the optimiser projects into a reduced, randomly oriented subspace and searches within

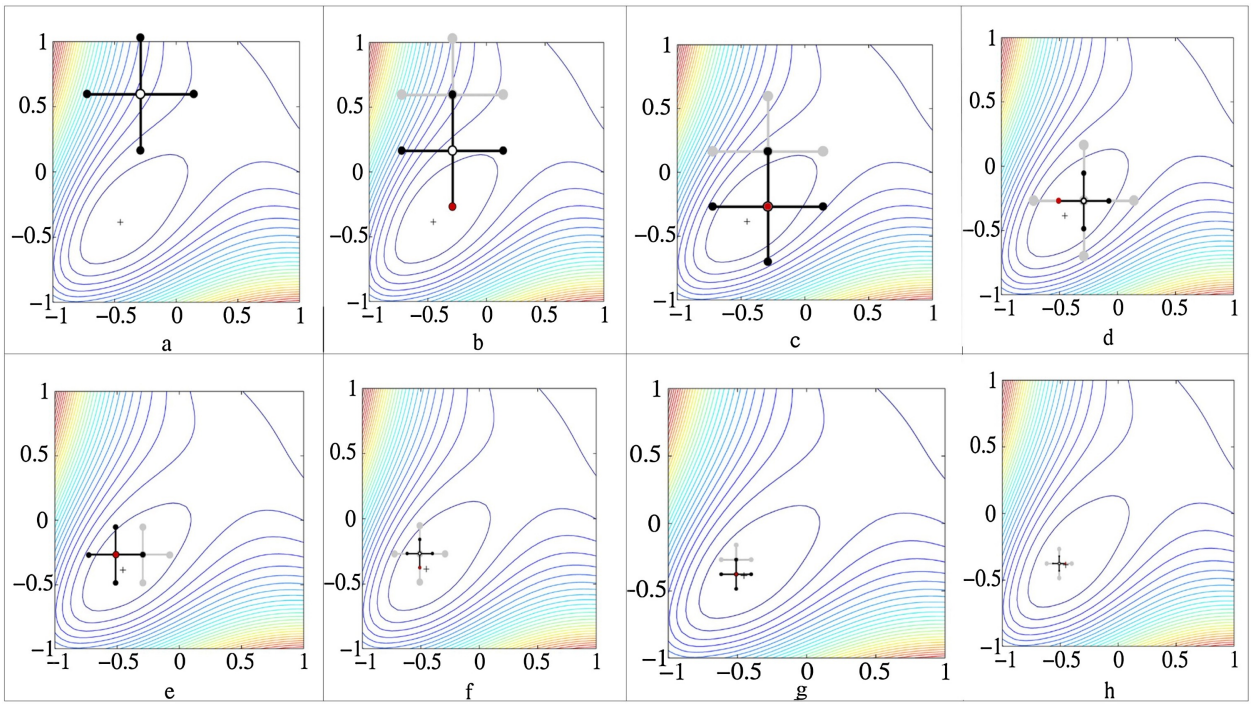

FIGURE 5.1: Visualisation of standard direct search optimisation moving on a topographical landscape from a) through to h). The small cross indicates a local/global minima of the landscape and the algorithm can move in the four cardinal directions to move towards it. The red point indicates where current position of the optimiser. This simple visualisation only reduces step size for unsuccessful steps (no step size increase on successful steps). (from Javed et al. Fig. 8 [100])

it. This approach naturally generates effectively random directions while operating in a smaller parameter space, which reduces computational cost.

Within the debut paper, classical optimisation studies showed that one-dimensional subspaces often yield optimal performance, as this maximises randomness for the search directions. In Chapter 6, we test whether this classical result holds for QML problems and how the use of the algorithm in QML enables its convergence effects. In this first subsection, we outline how the algorithm works through defining the parameters and notation as well as showcasing Algorithm 1.

5.1.1 Default Parameters and Notation

The direct search in reduced subspace method algorithm employs various symbols and notation to represent training parameters, constants and subspaces. These parameters and symbols are defined as such, aligning with standard terms in optimisation literature, adapted for a QML purpose in our work.

- The symbol θ refers to the parameters for the loss function, in the DRC case the angle of rotation for rotation gates R_x, R_y, R_z. Denoted with subscript θ_0 as the initial parameters and θ_k to refer to the kth iteration of the parameter angle.

- The symbol α refers to the step size variable which is adaptable. There is a maximum step size $\alpha_{\max} > 0$, and the step size is adaptively changed at the iteration of α_k.
- Constants c and γ control important qualities of the algorithm. The constant c changes the sufficient decrease condition of the algorithm, allowing for a larger or smaller sufficient decrease to be applied to the system. The constant γ dictates the change in step size, with γ_{dec} being the coefficient that the step size is decreased by, and γ_{inc} for increasing the step size.
- The random projection matrix is defined as $\mathbf{P}_k \in \mathbb{R}^{r \times p}$, where p represents the full number of parameters and r denotes the number of parameters in the reduced subspace. It maps the full parameter vector in $\mathbb{R}^p$ into the reduced r-dimensional subspace in $\mathbb{R}^r$, where directions $\mathbf{d}_k \in \mathbb{R}^r$ are mapped back to the full space as $\mathbf{P}_k^T \mathbf{d}_k$. Each $\mathbf{P}_k$ is drawn randomly at every iteration of the algorithm.
- The search directions are given by the set $\mathcal{D}_k$ within the reduced subspace. These directions are deterministic as a set of coordinate vectors and their negatives, and a chosen direction is denoted $\mathbf{d}_k$. When mapped back to the full parameter space via $\mathbf{P}_k^T$, these directions become randomly oriented due to the random projection.
- The loss function follows the same notation with $\mathcal{L}(\theta)$, in our implementation using the mean squared error (MSE) loss.

5.1.2 Algorithm Design

The design of the direct search algorithm from Roberts and Royer [83] is detailed in Algorithm 1; this pseudo-code outlines the core logic that allows the gradient-free method to function. The algorithm proceeds as follows for each iteration k: first, generate a random subspace $\mathbf{P}_k$ (line 3), then generate a set $\mathcal{D}_k$ of m directions within the reduced subspace (line 4). The algorithm polls these directions, checking if any $\mathbf{d}_k \in \mathcal{D}_k$ satisfies the sufficient decrease condition (line 5). If a successful direction is found, the parameters are updated along that direction (line 7) and the step size is increased (line 8). If no direction yields sufficient decrease, the parameters remain unchanged (line 10) and the step size is reduced (line 11). Convergence of this algorithm operates on the property of sufficient decrease, where the loss function decrease is characterised in terms of the step size α_k, with adaptive changes to the step size being necessary to advance the algorithm. The adaptive system for accepting a step builds inherent convergence into the algorithm, as polled directions are tested for sufficient decrease. If this condition is not met, the step size is decreased. This lowers the threshold for sufficient decrease and allows for progress to be made in areas close to the solution.

Adaptive Step size and Sufficient Decrease

The classical sufficient decrease acceptance rule and adaptive step size are vital in allowing this direct search method to adaptively control progress, preventing inefficient updates while ensuring steady convergence toward stationary points. The sufficient decrease condition is a strict rule used in optimisation algorithms to reject poor solutions [101]. It requires a decrease in loss proportional to the square of the step size, the sufficient decrease condition prevents the optimiser from accepting arbitrarily small steps and guarantees convergence to

Algorithm 1 Direct-search method in reduced parameter subspaces

Inputs: Initial parameters $\boldsymbol{\theta}_0 \in \mathbb{R}^p$, step size $\alpha_0 \in (0, \alpha_{\max}]$, maximum step size $\alpha_{\max} > 0$, constants $c > 0$, $0 < \gamma_{\text{dec}} < 1 < \gamma_{\text{inc}}$; number of directions $m \in \mathbb{N}$, reduced dimension $r \leq p$
for $k = 0, 1, 2, \ldots$ **do**
  Generate a random projection matrix $\mathbf{P}_k \in \mathbb{R}^{r \times p}$
  Generate a set $\mathcal{D}_k \subset \mathbb{R}^r$ of m search directions
  if there exists $\mathbf{d}_k \in \mathcal{D}_k$ such that

$$\mathcal{L}(\boldsymbol{\theta}_k + \alpha_k \mathbf{P}_k^T \mathbf{d}_k) < \mathcal{L}(\boldsymbol{\theta}_k) - \frac{c}{2}\alpha_k^2 \|\mathbf{P}_k^T \mathbf{d}_k\|^2$$

  then
    Update $\boldsymbol{\theta}_{k+1} := \boldsymbol{\theta}_k + \alpha_k \mathbf{P}_k^T \mathbf{d}_k$
    Update step size $\alpha_{k+1} := \min(\gamma_{\text{inc}} \alpha_k, \alpha_{\max})$
  else
    $\boldsymbol{\theta}_{k+1} := \boldsymbol{\theta}_k$
    $\alpha_{k+1} := \gamma_{\text{dec}} \alpha_k$
  end if
end for

a stationary point [102]. The sufficient decrease condition enables adaptive step size control, as the step size increases if the condition is met and decreases if it is not. This is implemented as follows. At iteration k, let the current step size be $\alpha_k > 0$, one chosen direction $\mathbf{d}_k \in \mathcal{D}_k$ and $\mathbf{P}_k$ is the random subspace projection. The next point is formed as a test point,

$$\theta_k^{test} = \theta_k + \alpha_k \mathbf{P}_k^T \mathbf{d}_k \tag{5.1}$$

This tested point is only accepted if it yields a sufficient decrease in the loss defined by this condition,

$$\mathcal{L}(\boldsymbol{\theta}_k + \alpha_k \mathbf{P}_k^T \mathbf{d}_k) < \mathcal{L}(\boldsymbol{\theta}_k) - \frac{c}{2}\alpha_k^2 \|\mathbf{P}_k^T \mathbf{d}_k\|^2 \tag{5.2}$$

where $c > 0$ is a constant for controlling the sufficient decrease condition requirement. Once the sufficient decrease condition is satisfied for at least one direction in the random direction subset, the next step towards convergence can be made. In a successful case, the loss parameter θ is updated to the accepted test parameter and the step size is increased,

$$\alpha_{k+1} = \gamma_{inc} \alpha_k \tag{5.3}$$

where $\gamma_{inc} > 1$ is a constant. If there is no tested direction that satisfies sufficient decrease, the next step in optimisation cannot be made. The unsuccessful case results in backtracking to the previous iteration and reducing the step size,

$$\alpha_{k+1} = \gamma_{dec} \alpha_k \tag{5.4}$$

where $0 < \gamma_{dec} < 1$ is a small constant. This adaptive step system in this direct search method mimics the classical trade-off between exploration and exploitation. Successful iterations

increase the step size - akin to exploitation as the algorithm rapidly moves in the most promising descent direction. In contrast, unsuccessful iterations must revert and decrease the step size, shifting focus to an exploration-type response, where small steps can focus on finding a promising descent direction. This adaptive process allows for dynamic movement along the optimisation landscape, alternating between aggressive convergence in favourable regions (e.g. deep slopes) and precise refinement.

The sufficient decrease condition prevents negligible improvement compared to step size, and the adaptive step size update balances the movement of the algorithm between exploration and exploitation.

Stopping Conditions

When considering convergence guarantees, the algorithm must terminate in a finite time using suitable stopping conditions. In this direct search method, the stopping condition is enacted as a practical judge of convergence to a stationary point. The conditions used by Roberts and Royer [83] are as follows,

1. Step size threshold: The algorithm is terminated once the step size is sufficiently small, defined as $\alpha_k \leq \alpha_{min}$ where $\alpha_{min} > 0$ is close to zero (set to 10^{-6} in [83]). This is a standard stopping condition in direct search methods, reflecting that no more meaningful progress can be made on the landscape at this small scale.

2. Evaluation budget E: As the dominant computational cost in optimisation algorithms is function evaluations, a maximum number of loss function evaluations is set to terminate the algorithm. This condition is resource-based rather than convergence-based. This is implemented as a practical and necessary condition, particularly in QML where each loss function calculation involves a quantum circuit evaluation. In later chapters, we refer to this term as the maximum evaluations hyper-parameter E (see Chapter 6).

These stopping criteria are necessary for the practical use of this algorithm, with 1. ensuring the algorithm can exercise guaranteed convergence, and 2. safeguarding high computational costs by preventing infinite loops and allowing robust benchmarking.

Within this framework, the reduced dimension r (subspace dimension) and the evaluation budget (maximum evaluations) emerge as the primary hyper-parameters we investigate in Chapter 6. The subspace dimension r controls the size of the search space at each iteration, while the evaluation budget constrains the total computational resources available. Our empirical analysis of hyper-parameters specifically in Section 6.2 characterises the effect of subspace dimension and maximum evaluations on optimisation performance and convergence in QML.

5.1.3 Convergence Criteria

We now summarise the convergence guarantees of the reduced subspace direct search algorithm, where random subspaces introduce probabilistic behaviour. The analysis generalises classical direct search convergence by introducing two probabilistic alignment properties that replace the deterministic positive-spanning assumption:

The first property is the well aligned subspace property, which ensures that each random projection preserves sufficient gradient information from the full parameter space. The second is the probabilistic descent property, which guarantees that at least one search direction within the reduced subspace provides sufficient decrease in the loss function. These two probabilistic properties allow for the algorithm to maintain the same classical convergence guarantees, despite introducing randomly oriented lower-dimensional spaces.

Before analysing convergence, several assumptions must hold:

1. The loss function is bounded below and is continuously differentiable, with Lipschitz continuous gradients.

2. Step sizes α_k remains bounded within a finite interval $(0, \alpha_{\max}]$, with $\alpha_{\max} > 0$.

3. Each projection matrix $\mathbf{P}_k \in \mathbb{R}^{r \times p}$ has full row rank and bounded operator norms, which ensure stable mappings between the reduced and full parameter spaces.

4. The number of polling directions in each reduced subspace is finite, which limits the number of function evaluations per iteration.

Probabilistic Well-aligned Property

In the reduced subspace described by algorithm 1, the polling directions are deterministically generated within the lower r-dimensional subspace. However, because this subspace is itself randomly chosen through the random projection matrix $\mathbf{P}_k$, its orientation varies between iterations k. As a result, the polling directions $\mathbf{P}_k^T \mathbf{d}_k$ are effectively random in $\mathbb{R}^p$, even though they are deterministically structured within $\mathbb{R}^r$.

To ensure convergence, $\mathbf{P}_k$ must be considered well-aligned with the gradient of the objective function ensuring that the chosen subspace is capable of representing a significant portion of the descent direction. This property is defined as $(\nu, \sigma, P_{\max})$-well aligned as long as it satisfies three inequalities: (1) it guarantees that the projected gradient retains an ν-fraction of the full gradient norm; (2) it bounds the operator norm of $\mathbf{P}_k$ above by the constant $P_{\max}$; and (3) it ensures that $\mathbf{P}_k$ has full row rank and its smallest non-zero singular value is at least σ:

$$\|\mathbf{P}_k \nabla \mathcal{L}(\theta_k)\| \geq \nu \|\nabla \mathcal{L}(\theta_k)\| \tag{5.5}$$

$$\|\mathbf{P}_k\| \leq P_{\max} \tag{5.6}$$

$$\sigma_{\min}(\mathbf{P}_k) \geq \sigma \tag{5.7}$$

These conditions are trivially satisfied when $\nu = \sigma = P_{\max} = 1$ and $\mathbf{P}_k$ is the identity, but not necessarily when the subspace is reduced or when the projected matrix is random. Because the deterministic requirements cannot be guaranteed in random subspaces, we must impose them with a prescribed probability q_w. This probabilistic property is hence described as the $(\nu, \sigma, P_{\max}, q_w)$-well aligned property, ensuring these conditions will hold with a probability of at least q, given the algorithm's past history $\mathcal{F}_{k-1}$:

$$\mathbb{P}(\mathbf{P}_k \text{ is } (\nu, \sigma, P_{\max})\text{-well aligned} \mid \mathcal{F}_{k-1}) \geq q_w, \tag{5.8}$$

where $\mathcal{F}_{k-1}$ denotes the information available to the algorithm up to the last iteration $k-1$ - ensuring the condition holds when the subspaces are generated adaptively. Intuitively, this means that probability q dictates how often the algorithm draws subspaces that contain a valid downhill direction. This probabilistic well alignment guarantees that the random subspace preserves efficiency for convergent descent to occur.

Probabilistic Descent

One of the core properties relevant to the convergence of the direct search in reduced subspaces is the concept of probabilistic descent. Classical deterministic direct search methods guarantee convergence by ensuring at each iteration, the polling set $\mathcal{D}_k$ forms a positive spanning set (PSS) [103]. This polling set includes a set of directions that guarantee at least one direction will point downhill if the current point is not optimal. As previously discussed above, the polling directions are not drawn from the full parameter space $\mathbb{R}^p$, but rather deterministically drawn from the r-dimensional subspace. The resulting polling set $\mathcal{D}_k$ is random because the orientation of the subspace - defined by the random projection matrix $\mathbf{P}_k$ - changes between iterations. Consequently, even a deterministic subspace direction set becomes effectively random in the full parameter space after the projection $\mathbf{P}_k^T\mathbf{d}_k$. Therefore, the classical PSS assumption no longer holds, and a probabilistic condition must be introduced to ensure convergence.

Following the framework of Roberts and Royer [83], the concept of probabilistic descent serves as a probabilistic analogue. In the deterministic case, a direction set $\mathcal{D}_k$ satisfies the $(\kappa, D_{\max})$-descent property at θ_k if,

$$cm(\mathcal{D}_k, -\mathbf{P}_k\nabla\mathcal{L}(\theta_k)) = \max_{d\in\mathcal{D}_k} \frac{-d^T\mathbf{P}_k\nabla\mathcal{L}(\theta_k)}{\|d\|\|\mathbf{P}_k\nabla\mathcal{L}(\theta_k)\|} \geq \kappa, \tag{5.9}$$

Here, cm denotes the cosine measure - a value that measures the worst-case alignment between the set $\mathcal{D}_k$ and any non-zero gradient $(-\nabla\mathcal{L}(\theta_k))$. Larger cosine measures indicate that the polling set contains at least one well aligned direction to the gradient, which enables potential descent.

In the reduced subspace setting, as discussed prior, this condition must be probabilistic due to the random generation of subspace and hence direction set. The probabilistic framework ensures that same alignment holds with some probability p_d. This property is formally defined as $(\kappa, D_{\max}, p_d)$-descent,

$$\mathbb{P}(\mathcal{D}_k \text{ is } (\kappa, D_{\max})\text{-descent} \mid \mathcal{F}_{k-1/2}) \geq p_d \tag{5.10}$$

where $\mathcal{F}_{k-1}$ denotes the past history of the algorithm up to iteration $k-1$.

This implies that at each iteration, there is a probability p that the polling set $\mathcal{D}_k$ contains at least one direction that sufficiently aligned with the descent direction $-\nabla\mathcal{L}(\theta_k)$ by a factor of at least κ. In simple terms, even though directions are drawn from random subspaces, the direct search method retains a consistent chance of finding the next step to reduce the loss function.

Together, both the $(\nu, \sigma, P_{\max}, q_w)$-well-aligned property and the $(\kappa, D_{\max}, p_d)$-descent property form the core probabilistic guarantees for convergence in direct search in reduced

subspaces. By utilising these conditions, we incur a non-zero probability at each iteration to become well-aligned with the descent direction, thereby guaranteeing convergence of the algorithm to a minima.

Convergence Guarantees

As explored above, the direct search method in reduced subspaces has convergence guarantees that rely on the two probabilistic properties: the well aligned subspace and the probabilistic descent conditions. Together with the sufficient decrease acceptance rule, these properties enable the algorithm to ensure almost-sure convergence under mild assumptions.

Almost-sure convergence is a term mainly used for probabilistic methods as a type of guaranteed convergence. This means that if the algorithm were to run an infinite number of times, the probability of non-convergence is zero. With this property, a single run of the algorithm is subject to random behaviour, but the method is theoretically robust for most iterations [104].

We can confirm almost-sure convergence using the previously mentioned $(\kappa, D_{\max}, p_d)$-descent property and the $(\nu, \sigma, P_{\max}, q_w)$-well aligned property of the random subspace. In these reduced subspaces, the $(\nu, \sigma, P_{\max}, q_w)$-well aligned property ensures that the random projection $\mathbf{P}_k \in \mathbb{R}^{r\times n}$ is well aligned with the gradient with probability $q_w > 0$, while the corresponding subspace directions $\mathcal{D}_k$ also satisfy the $(\kappa, D_{\max}, p_d)$-descent property with probability $p_d > 0$. When these probabilistic properties simultaneously hold and the joint probability satisfies $p_d q_w > p_0$, where $p_0 \in (0, 1)$ is a threshold constant depending on the algorithmic parameters $(c, \gamma_{\text{inc}}, \gamma_{\text{dec}}, \alpha_{\max})$ as defined previously. When this condition holds, the well aligned and descent capable subspaces occur often enough to guarantee convergence. This allows for the iterative set of parameter variables $\{\theta_k\}$ generated to almost surely converge to a stationary point. This is represented as the probability of 1 [103],

$$\mathbb{P}\left(\liminf_{k\to\infty} \|\nabla\mathcal{L}(\theta_k)\| = 0\right) = 1 \tag{5.11}$$

We assume that the loss function is bounded below and continuously differentiable with Lipschitz continuous gradients. This lim inf guarantee is an asymptotic property, meaning that the algorithm will keep finding points with arbitrarily small gradients and without considering computational stopping criteria. The number of function evaluations in the algorithm required to reach a ϵ-approximately stationary [22] point can be represented as a time complexity bounded above by

$$\mathcal{O}(p\epsilon^{-2}), \tag{5.12}$$

where p is the dimension of the space and ϵ is a tolerance threshold with ϵ-stationary point being a practical approximation of where the loss gradient is zero. This time complexity arises from the random direction on a unit sphere being well-aligned with the descent direction with probability proportional to $\frac{1}{p}$. This implies that $\mathcal{O}(p)$ random samples must be taken to identify the descent direction for every iteration, and the ϵ^{-2} scaling where finding a point θ_k with $||\nabla\mathcal{L}(\theta_k)|| \leq \epsilon$ requires $\mathcal{O}(\epsilon^{-2})$ (gradient descent has the same worst case). Intuitively, the complexity scaling shows that more directions are needed to find the descent direction in higher dimensional problems, but due to the adaptive step size mechanism, once

the direction is found, the convergence is comparable to first-order (e.g. gradient descent) methods.

5.2 Direct-Search for Quantum Models in the Finite-Shot Limit

With the necessary background on convergence guarantees of the direct search in reduced subspaces algorithm, we can now extend the analysis to the estimation of loss values using outputs of quantum models. We consider two loss functions applicable for variational quantum circuits: direct measurement loss for variational quantum eigensolvers (VQEs), and mean-squared loss for supervised QML.

5.2.1 Loss functions for quantum models

For variational quantum eigensolvers (VQEs), the loss function can be expressed as an expectation value of a quantum observable O:

$$\mathcal{L}(\theta) = \langle O \rangle_\theta \tag{5.13}$$

While exact simulation methods exist, practical quantum measurements rely on finite circuit executions otherwise known as shots, making expectation value estimates inherently stochastic. This practical result is a random variable defined as $\tilde{o}^{(S)}(\theta)$, obtained by averaging S measurement shots. By construction, the expectation value of this is an unbiased estimator of the true expectation value:

$$\mathbb{E}_S[\tilde{o}^{(S)}(\theta)] = \langle O \rangle_\theta = \mathcal{L}(\theta) \tag{5.14}$$

where $\mathbb{E}_S$ denotes the expectation over measurement shots. This simple loss case allows for $\tilde{o}^{(S)}(\theta)$ to be an unbiased estimator of the true expectation value, obtained by averaging over S measurement shots.

In contrast, variational QML problems often use a nonlinear loss function - namely the quadratic mean squared error (MSE). Here the estimator $\tilde{o}^{(S)}(\theta)$ can no longer be unbiased for the loss. With MSE loss we can instead construct an unbiased estimator for the target value y. MSE loss can be therefore represented as:

$$\mathcal{L}_{MSE}(\theta) = (\langle O \rangle_\theta - y)^2 = (\mathbb{E}_S[\tilde{o}^{(S)}(\theta)] - y)^2 \tag{5.15}$$

To represent the loss function itself as an unbiased estimator, we must consider the expectation value of the MSE loss. To construct an unbiased estimator for the MSE loss itself, we consider its expectation value over the data set. We define the data set $\mathcal{D}$ as such:

$$(x_j, y_j) \sim \mathcal{D} \quad \text{for } \mathcal{D} := \{x_j, y_j\}_{j=1}^M \tag{5.16}$$

Over this data set $\mathcal{D}$, the expectation value represented by $\mathbb{E}_\mathcal{D}$ refers to the empirical average over the M sampled data points (x_j, y_j). Hence, the true loss function of our problem can

be represented as an expectation value of the empirical loss (MSE loss).

$$\mathcal{L}_{true} = \mathbb{E}_{\mathcal{D}}[\hat{\mathcal{L}}] \qquad (= \frac{1}{M}\sum_{j=1}^{M} \mathcal{L}_{MSE}(\theta)) \tag{5.17}$$

For some (x_j, y_j), the empirical MSE loss is:

$$\hat{\mathcal{L}}(\theta, x_j, y_j) = (\langle O \rangle_{\theta, x_j} - y_j)^2 \tag{5.18}$$

Under the expectation value over the set $\mathcal{D}$, we can use the previously constructed unbiased estimator from equation 5.14, and substitute this into $\mathbb{E}_{\mathcal{D}}[\hat{\mathcal{L}}]$.

$$\mathbb{E}_{\mathcal{D}}[\hat{\mathcal{L}}] = \mathbb{E}_{\mathcal{D}}[(\mathbb{E}_S[\tilde{o}^{(S)}(\theta)] - y_j)^2] \tag{5.19}$$

In our current setting of perfectly learnable DRCs, the expectation value itself aims to perfectly equal the target data y_j as such:

$$y_j = \langle O \rangle_{\theta, x_j} \tag{5.20}$$

This allows for the unbiased estimator to be further evaluated:

$$\mathcal{L}_{true} = \mathbb{E}_{\mathcal{D}}[\hat{\mathcal{L}}] = \mathbb{E}_{\mathcal{D}}[(\mathbb{E}_S[\tilde{o}^{(S)}(\theta)] - \langle O \rangle_{\theta, x_j})^2] \tag{5.21}$$

However, though this further evaluation resembles the variance of the true expectation value of the quantum operator O, the two expectation values are different. These variables are distinct as $\mathbb{E}_S$ refers to the expectation value over the number of measurement shots, while $\mathbb{E}_{\mathcal{D}}$ refers to the expectation over the full data set. As such, this final result displays the data-averaged mean squared deviation between the estimator on the observable and the true expectation value evaluated at each x_j.

5.2.2 Direct search in finite-shot limit

In QML, circuit evaluations are costly, and therefore using an unbiased estimator for loss for finite (or even single) shots allows for a reduction in measurement resources. The unbiased estimators of loss introduced in the previous section justify applying the direct search method not just in the theoretical infinite-shot limit but also in the practical finite-shot regime typically associated with real hardware experiments. Finite-shot loss estimation, including the single-shot limit $S \to 1$ has been shown to preserve trainability for gradient-based methods in Sweke et al. [38]. Here, we examine whether similar reasoning may extend to gradient-free optimisation via direct-search, potentially enabling learning in the single-shot limit.

The unbiased finite-shot estimator in Equation (5.21) is valid for all finite-shot numbers, including the case where only a single measurement shot ($S = 1$) is used in expectation. For the case of the direct search algorithm, introducing finite shots may influence the convergence guarantees discussed previously by introducing an additional source of stochasticity to the loss evaluations.

We conjecture that even in the finite-shot setting, the introduction of the unbiased finite-shot estimator of the true loss is unlikely to alter the probabilistic nature of the algorithm's convergence behaviour. The core properties behind convergence in reduced subspaces – namely the well-aligned and probabilistic descent properties – are inherently stochastic, depending on the random generation of subspaces and the corresponding distribution of directions in the full parameter space. The validity of these properties should therefore be retained under unbiased loss estimation, since both the random projection matrix and direction sampling processes are independent of measurement noise introduced by the finite-shot regime.

The main effect of the unbiased finite-shot estimator for the true loss is expected to manifest through the sufficient decrease condition in Equation (5.2). In the deterministic infinite-shot limit, this test deterministically dictates which optimiser steps are accepted based on exact loss values. Under finite-shot noise, the sufficient decrease comparison becomes stochastic because the loss values are estimated. Consequently, individual iterations may occasionally accept or reject steps incorrectly due to estimator variance. However, by the construction of the unbiased estimator, we can assume zero-mean noise, implying that the expected sufficient decrease condition will still hold over many iterations.

Importantly, the unbiased finite-shot estimator of loss values only affects the sufficient decrease condition which governs the adaptive step size, but not the core convergence properties. Convergence depends primarily on the probabilistic descent and well aligned properties, together with standard smoothness assumptions. With the loss estimator constructed to satisfy $\mathbb{E}_{\mathcal{D}}[\hat{\mathcal{L}}] = \mathcal{L}(\theta)$, and assuming bounded variance, we conjecture that the convergence guarantees defined in Roberts and Royer [83] – specifically the result in Equation (5.11) – are likely to extend to the functionally doubly stochastic setting.

In summary, we conjecture that the direct search algorithm converges in expectation to a stationary point under unbiased stochastic loss evaluations. This suggests a viable extension of the theoretical guarantees of direct search in reduced subspaces to loss functions involving a stochastic, finite ensemble of quantum measurements.

However, an open question remains regarding the impact of finite shots on the time complexity of convergence (Equation (5.12)) and the definition of an approximate stationary point. Finite-shot noise limits the resolution of the accessible loss landscape, which could affect what is considered an ϵ-approximate stationary point. Specifically, the effect of sampling noise could manifest within the norm of the gradient $||\nabla\mathcal{L}||$, potentially influencing step acceptance as well as progress in minimising the loss. While the asymptotic guarantee of almost-sure convergence in Equation (5.11) may be preserved in expectation, the practical number of shots required to reach a tolerance for convergence could differ significantly from the infinite-shot case. While a detailed analysis of convergence behaviour is beyond the scope of this project, we can demonstrate empirically that the direct search method can be applied using finite-shot simulations. In the next chapter (Chapter 6), we present proof-of-concept simulations using small circuits to demonstrate that the method is feasible in practice.

6

Direct Search Simulations for Data Re-uploading Circuits

This chapter combines the direct search algorithm introduced in Chapter 5 with variational quantum data re-uploading circuits discussed in Chapter 4, providing an empirical evaluation of how these methods perform in QML tasks. We focus on understanding the effect of direct search hyper-parameters when applied to parameterised quantum circuits.

In this chapter, I implemented the direct search algorithm for multi-qubit data re-uploading circuits by writing new Python functions that use existing direct search libraries [83] to interface with the simulation framework developed in Chapter 4. I designed and executed all simulation experiments on the HPC Bunya cluster [88], and proposed the methodological approach used to ensure a fair comparison between Adam and direct search. The discussion of the implications of direct search for QML was carried out in collaboration with my supervisor, Riddhi Gupta.

To ensure that the key comparison between direct search and Adam is robust, the first two sections address how to configure Algorithm 1 for a specific application, including the choice of hyper-parameters, especially in the absence of existing literature. This is achieved through empirical simulations that illustrate how these hyper-parameters affect optimisation performance, highlighting their influence on convergence, and identifying optimal settings for running simulations on HPC Bunya [88].

Next, we examine the impact of finite-shot measurements and empirically test the doubly stochastic variant of direct search. This section investigates how sampling noise affects the convergence of direct search and compares the same implementation with Adam, providing insight into each method's robustness to noise under realistic conditions.

Finally, the last section discusses how direct search and Adam are expected to perform when encountering the main trainability challenges of VQAs, such as barren plateaus and exponentially many local minima, as described in Chapter 3. This discussion provides insight

into the relative strengths and limitations of each optimiser, which guides the selection of strategies for practical quantum machine learning applications.

6.1 Methodology for Configuring Algorithm 1

The approach of this thesis is to combine quantum simulations of data re-uploading circuits with classical optimisation algorithms to complete a machine learning task, this specifically being a regression task. All simulations of quantum variational circuits relied on the Qiskit framework [93] which is a Python-based library. The original direct search method corresponding to Algorithm 1 has an accompanying Python library `directsearch` developed by Roberts and Royer [83].

The general framework of this training process was previously outlined in Section 4.4.1, and here we expand on the algorithm in detail.

The full variational quantum algorithm pseudocode developed to support this project is presented in Algorithm 3. Each iteration begins by executing the parameterised DRC to obtain an output, either through an exact density matrix simulation or through a finite number of measurement shots, for the purpose of evaluating the mean squared loss function. In the latter case, the resulting loss is estimated using the unbiased estimator introduced in Equation (5.21). This loss function is then passed to the classical optimiser to update the circuit parameters for the next step. In our implementation, the framework was designed to be modular, allowing the classical optimiser to be selected as either Adam or direct search. To ensure consistent application of these two methods, both were implemented using the same step-wise structure.

In practice, the `directsearch` package from Ref. [83] was utilised to initialise a DirectSearch optimiser instance, which was then executed in a step-wise manner through a wrapper function, as shown in Algorithm 2. Each step of the optimiser performs an internal optimisation following the procedure outlined in Algorithm 1. This involves evaluating the loss function multiple times over a randomly projected subspace, up to the specified evaluation budget, where each evaluation may correspond to either an accepted or rejected step. The loss function itself interfaces directly with Qiskit via the quantum circuit simulator, where each evaluation constructs and executes the parameterised DRC to compute the expectation value and the MSE loss. Once the evaluation budget is reached, the optimiser returns an updated set of parameters, where it is used as the starting point for the next step. This step-wise wrapper serves to connect the classical optimisation algorithm to the quantum circuit simulation. This step-wise optimisation structure over also applies to the Adam optimiser, as illustrated in Algorithm 3 through the choice of optimiser.

The number of optimisation steps is terminated using a fixed number of steps. This corresponds to a hard evaluation budget, and in our implementation learning is stopped at 1000 steps, which is sufficient for the majority of potentially convergent solutions to converge in practice. This choice aligns naturally with the discussion in Chapter 5, where the evaluation budget is treated as a relevant stopping criteria for the direct search algorithm. A second stopping criterion is introduced when the target accuracy is reached. Specifically, once the model achieves a threshold $R^2 = 0.99$ value, the optimisation is terminated early and the model is considered converged, as the learning task has been accurately fit. This

Algorithm 2 Reduced Subspace Direct Search Step Implementation

Require: subspace dimension r, maximum evaluations E, initial parameters θ_0, loss function $\mathcal{L}(\theta)$

```
Initialise DirectSearch optimiser with (r, E)
function STEP(L, θ_0)
    Perform subspace projection of dimension r (Gaussian random subspace)
    Evaluate L(θ) at E random directions (deterministic polling directions)
    Select parameter updates that minimises loss L(θ)
    return updated parameters θ
end function
```

threshold was chosen empirically, as values around $R^2 \approx 0.99$ visually correspond to very near-perfect learning for the data considered, without requiring the optimiser to spend additional iterations in refining the model. However, this slightly limits our ability to observe the optimiser's behaviour in the fine-adjustment stage. This early stopping condition prevents necessary evaluations once a sufficiently accurate model has been learned, which helps reduce overall computational cost. Unlike in Chapter 5, we do not use the step-size-based stopping criteria in our implementation. This is because our implementation of the direct search loop resets the internal step size of the optimiser every iteration, functioning effectively as a simple form of restarting, which improves robustness but means that the step-size convergence cannot be meaningfully tracked.

The remaining subsections focus on how the logic of the direct search algorithm (Algorithm 1) is called to minimise the loss function. Of the many free parameters given in Algorithm 1, this thesis finds that two hyper-parameters are critical to performance, these being the subspace dimension r and maximum evaluations E. The subsections below outline a methodology by which these hyper-parameters of Algorithm 1 are characterised for variational quantum circuits - both in terms of their impact on performance and the number of functional evaluations.

6.1.1 Subspace dimension

The subspace dimension r determines the size of the projected r-dimensional space onto the full parameter space of dimension p. As outlined in Section 5.1, each iteration randomly samples an r-dimensional of the full parameter space and performs optimisation within this projection. Here, p denotes the total number of trainable parameters, which corresponds to the dimensionality of the full parameter space. This projected subspace enables the core logic of the direct search method in reduced subspaces, with the subspace dimension dictating the reduction in parameter space.

Values assigned to the subspace dimension hyper-parameter must be lower than the number of trainable parameters in the system to be considered the reduced subspace method. At a subspace dimension r equal to the number of parameters p, the algorithm collapses to a deterministic full parameter space direct search, as the probabilistic exploration property is linked to the subspace. This concept can be easily visualised as a three-dimensional parameter space $\mathbb{R}^p$ projected to a two-dimensional subspace $\mathbb{R}^r$, where the subspace is

Algorithm 3 Data Re-uploading Circuit Testbed Implementation

Require: Optimiser $\in$ {`Adam`, `DirectSearch`}, Model $\in$ {Finite Shots, Density Matrix}, shots S, degree d, qubits n, layers N, learning rate η, subspace dimension r, max evaluations E

Generate Fourier dataset $\{(x_j, y_j)\}$ of degree d and random initial parameters θ_0
function CREATECIRCUIT(n, N)
    Initialise n-qubit parameterised circuit with Qiskit
    Apply N trainable layers interleaved with entanglement
    Apply data re-uploading encoding
    return DRC circuit
end function
function QUANTUMMODEL(θ, x_j)
    CreateCircuit and bind parameters θ and input x_j to the circuit
    if Model = Density Matrix **then**
        Simulate reduced density matrix ρ_{red} to compute $\langle Z \rangle$
    else (Model = Finite Shots)
        Sample circuit with S shots to estimate $\langle Z \rangle$
    end if
    return expectation value $\langle Z \rangle$
end function
function LOSS(θ)
    $\hat{y}_j \leftarrow$ QuantumModel(θ, x_j)
    return mean squared error loss $\mathcal{L}(\theta) = \frac{1}{M} \sum_j (y_j - \hat{y}_j)^2$
end function
function TRAIN(θ_0)
    if Optimiser = `Adam` **then**
        Initialise Adam optimiser with learning rate η
    else (Optimiser = `DirectSearch`)
        Initialise Direct Search optimiser with subspace dimension r and maximum evaluations E
    end if
    for each training step **do**
        $\theta \leftarrow$ Optimiser.step($\mathcal{L}, \theta$)
        if convergence criteria met $R^2 = 0.99$ **then**
            break
        end if
    end for
    return trained weights θ^*
end function
for each trial **do**
    Initialise random weights θ_0
    $(\theta^*, \text{logs}) \leftarrow$ Train(θ_0)
    Compute final R^2
    Save results
end for

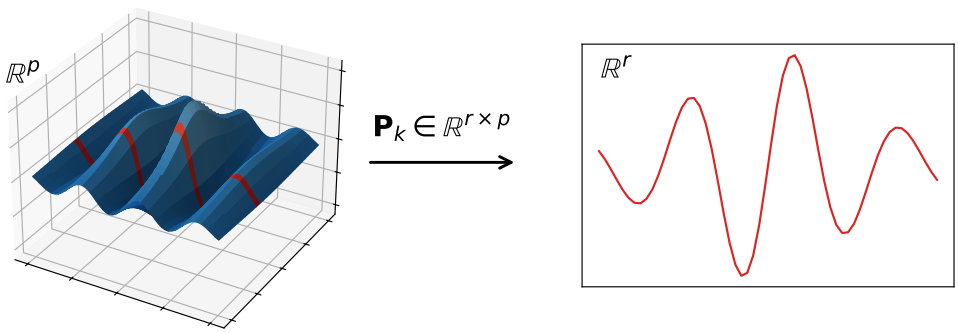

Figure 6.1: Visualisation of full parameter space in three dimensions, projected to a two-dimensional subspace to show the reduction of parameter space optimised over.

one slice of the three-dimensional function. This visualisation in Figure 6.1 shows what a subspace dimension of 2 would look like if the full parameter space was of dimension 3. In a practical setting, the difference between dimensions is much larger, and allows the random alignment of projection matrices to allow for probabilistic descent as discussed in Section 5.1.3. While the subspace dimension r controls how much of the parameter space is explored in each step, the maximum evaluations hyper-parameter E determines how thoroughly the given subspace is explored during optimisation.

6.1.2 Maximum evaluations

The maximum evaluations E hyper-parameter defines the upper limit on the number of loss function evaluations that the direct search optimiser can perform during training. In the context of the library `directsearch`, this is used as a computational budget to ensure that the optimisation does not utilise more than the assigned budget.

However, in the modified code used in this thesis to enable a step-wise implementation compatible with variational quantum circuits (Algorithm 2) , the maximum evaluations hyper-parameter E now caps the number of loss function evaluations per optimisation step. This means that in the maximum evaluations budget set - the optimiser essentially conducted self contained optimisation loops in each step, terminating at the maximum number each time, as discussed above. From this point onward, the terms maximum evaluations and maximum evaluations per step are used interchangeably to refer to the same quantity, denoted by E.

My modified application of direct search using the maximum evaluations hyper-parameter comes with advantages and disadvantages. The main advantage is that it introduces an implicit restart mechanism, where each step begins with a reset step size, allowing the optimiser to escape local minima such as those described in Chapter 3. This is a known mitigation method introduced in Section 3.1.4, using the larger initialisation step size to escape poor minima. The main disadvantage of my modified algorithm is that step-wise progress between these restarts is lost. This means that evaluations sometimes must be spent returning to the step size previously achieved and re-approaching the prior good region. In

practice, the computational cost of this was observed to be low, as the algorithm quickly reduces its step size, halving after every successful step taken.

In summary, the approach used for this hyper-parameter is slightly different from the original work of Roberts and Royer [83]. In the original implementation, maximum evaluations acted as a total computational budget, whereas in this usage the number of maximum evaluations behaves more like a number of loss evaluations per step. This is useful as we can pinpoint the number of functional evaluations used and adjust it directly. Adjusting the maximum evaluations hyper-parameter E effectively increases the amount of optimisation per step. This is particularly important in larger parameter spaces, where more random projections are needed to make meaningful progress. If the number of maximum evaluations per step is too small, the optimiser can fail to find directions that provide improvements, resulting in little or no advancement in the optimisation landscape. By increasing the maximum evaluations, more steps are likely to contribute meaningful improvement, making the optimisation more effective.

6.2 Empirical Evaluation of Hyper-parameters

In this section, we present empirical results testing the effects of direct search hyper-parameters on our designed QML problem. To investigate how this applies in a quantum setting, we can examine how each hyper-parameter influences learning behaviour, convergence, and computational efficiency. Specifically, we focus on understanding how tuning the subspace dimension and the maximum evaluations can improve optimisation performance.

When considering simulations for testing the effect of hyper-parameters, we specifically chose a mid-range qubit number circuit to tune hyper-parameters with scaling in mind. This model was a 5-qubit circuit using the direct search in reduced subspaces method. Each trial enforced a R^2 convergence threshold of 0.99, deeming this as a converging solution, where a hard cap on the number of steps was set at 1000. For scale, most converging solutions on a 5-qubit model were below 100 steps.

6.2.1 Effect of Subspace Dimension on Learning Behaviour

The subspace dimension r plays an important role in determining how effectively the reduced subspace direct search method can explore the parameter space of the training problem. The size of this subspace controls the amount of information available to the optimiser in a single step – smaller values of r encourage stochastic exploration and simplify the search, while larger values increase deterministic behaviour. In our simulations testing this hyper-parameter, we expected the best results at dimensions much smaller than 10; however, we also considered implementing this method at a much larger subspace dimension to see how the algorithm performs when approaching a deterministic search.

In order to study the effect of subspace dimension size on accuracy, the maximum evaluations E were set to a middle value of 50. Specifically, the subspace dimensions that were tested ranged from 1 to 10 and also included 30 and 60. This selection allowed us to analyse the behaviour of low dimensional subspaces (below 10) as well as consider half (30) and the entire parameter space (60) of a 5-qubit circuit, which has $p = 60$ total trainable parameters.

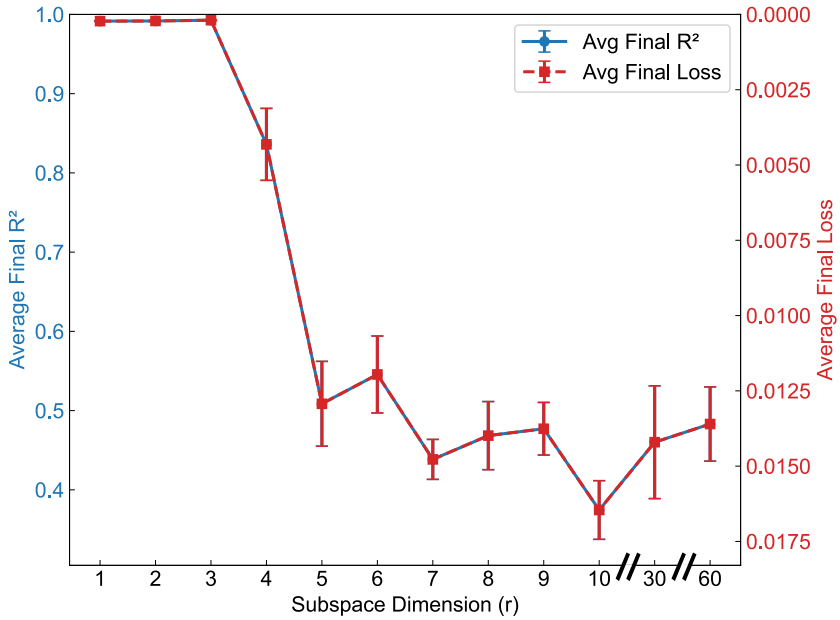

Figure 6.2: Average final R^2 value (blue) and loss (red) as a function of the subspace dimension r using a fixed number of maximum evaluations E set at 50, on a 5-qubit model. Each point shows the mean value across 5 trials, with error bars representing the standard error of the mean (SEM). The model remains accurate up to a dimension of 3, dropping sharply at dimensions above this. Larger dimensions are less effective, as the optimisation becomes more deterministic and loses the stochastic direction selection that enables exploration in the reduced subspace, leading to reduced accuracy because the solution cannot converge

This result, shown in Figure 6.2 demonstrates the importance of subspace dimension. At low subspace dimensions, the projected subspace is more likely to align sampled directions randomly, enabling the probabilistic descent property and allowing convergence. As the subspace dimension increases, however, the optimisation becomes increasingly deterministic as it approaches the full parameter space size. This occurs because higher dimensional subspaces have a reduced ability to align directions randomly, which limits stochastic exploration and causes the search to fail to reach optimal regions. This behaviour aligns with the findings of Roberts and Royer [83], who observed that one-dimensional subspaces were optimal for sampling random directions in classical applications. This theoretical result was introduced earlier in Section 5.1, where we discussed how reduced one-dimensional subspaces maximise randomness in direction sampling and lead to faster convergence. Our results support this observation, showing that lower dimensional subspaces can achieve accurate learning in a QML setting.

6.2.2 Scaling with Maximum Evaluations

The maximum evaluations E hyper-parameter influences how much optimisation occurs in each step, directly influencing how much progress the optimiser can make towards convergence. Larger values of E allow for deeper exploration within each step, while small values

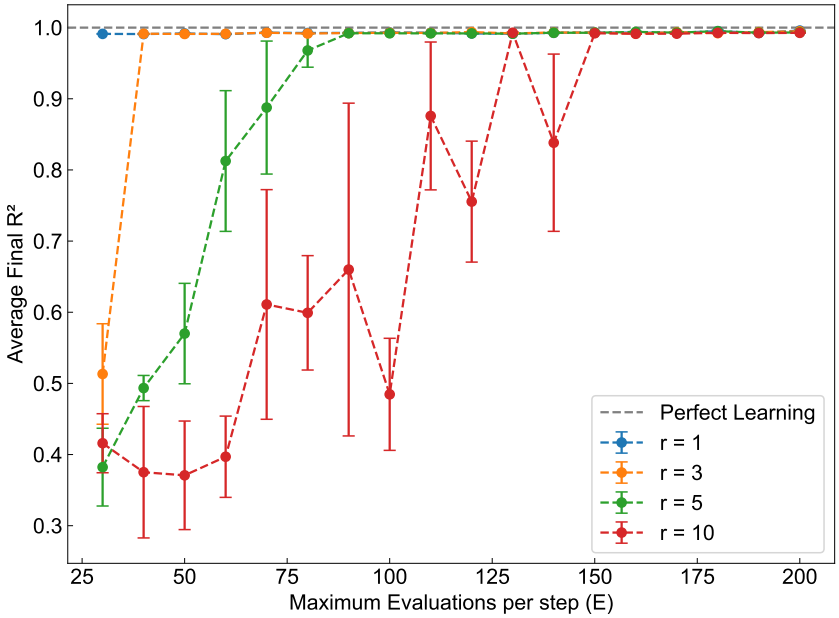

FIGURE 6.3: Average final R^2 value as a function of the maximum evaluations per step E for different subspace dimensions r in different colours. Each line represents the performance of a specific subspace dimension r, e.g $r = 1$ (blue), $r = 10$ (red). The grey dashed line indicates perfect learning at $R^2 = 1$. The data collected was averaged over multiple trials and the error bars denote the standard error of the mean (SEM). The plot shows that increasing the maximum evaluations per step allows for each subspace dimension to converge, as more evaluations allow the optimiser to search more robustly. This increase is vital for larger subspace dimensions since subspaces such as $r = 10$ are much larger than $r = 1$ and require more evaluations to make meaningful progress in the optimisation landscape. Similarly to Figure 6.2, larger dimensions are less efficient as they can only converge under a high number of maximum evaluations. In this plot, the subspace dimension of $r = 1$ is capable of achieving a convergent solution with the least number of maximum evaluations per step.

result in faster but shallower updates. Both regimes are valid, with the choice primarily influencing the total number of optimisation steps required.

To investigate the effect of the maximum evaluations hyper-parameter, we conducted simulations on the same 5-qubit model discussed in Section 6.2, considering a spread of subspace dimensions as the maximum evaluations per steps increases. We tested subspace dimensions $r = 1, 3, 5, 10$ ranging from 30 to 200 maximum evaluations per step. This spread allowed us to study the effect of maximum evaluations per step, and see if high-dimensional subspaces can converge if given enough evaluations to explore their parameter spaces. The resulting plot in Figure 6.3 shows the effect of scaling maximum evaluations per step. At a low number of maximum evaluations per step E and a high-dimensional subspace (e.g $r = 10$), the optimiser struggles to locate a meaningful descent direction before reaching its evaluation budget, leading to poor convergence. This trend is seen with $r = 5$ with 30 to 80 evaluations, with the budget allowing for meaningful optimisation progress after this. Contrasting the result from Figure 6.2, we see that higher-dimensional subspaces are

capable of convergence, given a large number of maximum evaluations. This aligns with our expectations as these subspaces are larger and would require more evaluations per step to traverse the optimisation landscape and find a solution. Each of these subspaces does eventually converge given a valid number of evaluations, but a subspace dimension of 1 can converge with less than 30 evaluations per step.

However, this plot only quantifies learning performance in terms of R^2 accuracy relative to the evaluation budget per step. It does not reflect how many steps were required for convergence overall. Since each subspace dimension is capable of converging, this motivates further comparison based on true computational efficiency, to find which configuration achieves convergence with the least computational cost.

6.2.3 Computational Costs

Before considering computational cost, we assess the interaction of the two hyper-parameters - subspace dimension r and maximum evaluations per step. We construct a heat map showing the average final loss values obtained across combinations of the two hyper-parameters. From a subspace dimension of 1 to 10, and maximum evaluations per step from 30 to 200, resulting in a multicoloured grid. Each grid square represents a specific subspace dimension r with a number of maximum evaluations per step. This approach allows us to visualise the full effect of these hyper-parameters on convergence using direct search in reduced subspaces. This visualisation is valuable as it captures the full trend, which was not completely obvious in previous plots. It allows us to definitively identify convergent and non-convergent regimes, and provides insight into expected performance at high qubit numbers.

The heat maps in Figure 6.4 highlights the discrete regions of convergence performance. The full scale map shows no smooth or continuous dependence on either hyper-parameter, instead convergence only occurs in distinct convergent and non-convergent regions. This discrete separation reflects the direct search algorithm's learning technique - once a sufficient number of maximum evaluations are used, the optimiser approaches the solution quickly. But when maximum evaluations per step are below this required limit, the optimiser struggles to converge. This behaviour is observed for the high dimensional subspaces with low maximum evaluations, specifically referring to the multi-coloured region in the left panel of Figure 6.4. The zoomed-in plot on the right highlights this clear distinction with the reduced scale. The low loss value scale seen in the right plot, shows that each point in the convergent region achieves comparable loss values, indicating accurate learning across the whole 'good' region. However, any variance in this pink section is uninformative as trials enforced a R^2 (directly correlated to loss) convergence threshold, which pauses any training when a value of 0.99 is reached. At this scale, this confirms there is no transition region, i.e., sections where the optimiser almost converges.

Now with this insight into the interaction between the two hyper-parameters, we can look into aspects of computational efficiency. In this context, we are specifically interested in the number of loss function evaluations required for convergence. This metric is important as the most computational resources are used in evaluating the loss function, where the quantum circuit must be measured each time, increasing the runtime and memory cost of optimisation. Since the previous analysis showed which regions lead to convergent learning,

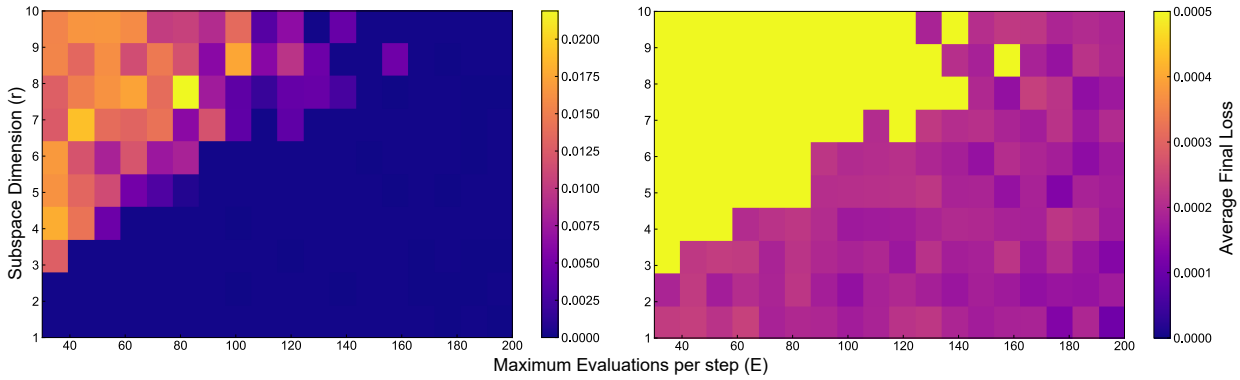

FIGURE 6.4: Heat maps of average final loss value as a function of subspace dimension r and maximum evaluations per step E. The left map shows full scale, while the right map shows reduced scale to focus on low loss achieving data. (Left) Full scale map shows a discrete convergent region (dark blue) and a non-convergent region (multi-coloured), this distinguishing between accurate and poor learning. The converging section of hyper-parameters has a very low loss value. (Right) To see the variation in these low loss values, the right map shows a smaller scale. The yellow section is that same poor region, where the pink section is the convergent region. Due to the R^2 convergence threshold, the difference between points is not informative. But this shows that the whole pink section shows low loss, meaning valid accurate learning and convergence.

we can observe those same regions to differentiate which set of hyper-parameters is both effective and efficient.

The same data and trials were used from the previous analysis, with the following analysis characterising the number of loss function evaluations. To calculate this metric, the total number of steps performed are multiplied with the maximum evaluations per step. Plotting this quantity across the hyper-parameters allows us to observe if differences in efficiency exist between subspace dimension and maximum evaluations per step. We expect there to be some correlation with maximum evaluations per step, however when the system is given a higher number of maximum evaluations per step, the system is capable of converging in a lower number of steps.

These effects are evident in the resulting heat maps in Figure 6.5, where the trade-offs between subspace dimension and maximum evaluations are shown. The left heat map mirrors Figure 6.4 showing where the convergent and non-convergent zones are located, but from the large scale the difference between converging solutions is not fully visible. In this dark blue region, we zoom in with the right map, this showing a more continuous distribution between subspaces. It is clear that the maximum evaluations per step does not affect the number of loss function evaluations as much as the subspace dimension does. This is clear by the multi-coloured gradient from top to bottom, where lower subspace dimensions achieve a number of loss function evaluations below 3000, whereas high dimension spaces take more than double. The effect of maximum evaluations per step on efficiency is not substantial, with the number of steps the optimiser naturally balancing-out performance across this hyper-parameter.

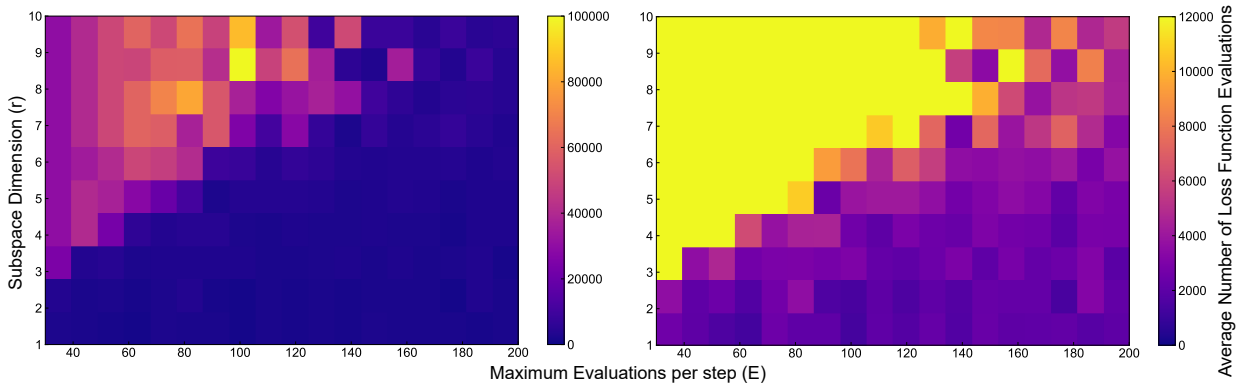

Figure 6.5: Heat maps showing the average number of loss function evaluations by the hyper-parameters, subspace dimension r and maximum evaluations per step E. This metric is calculated by multiplying the maximum evaluations per step with the number of steps the optimiser takes. (Left) The full scale map shows similar discrete sections like in Figure 6.4, where these two regions correlate to converging (dark blue) and non-converging (multi-coloured) solutions. (Right) By using a reduced scale, we can see that the number of functional calls grows with increasing subspace dimension. This suggests that a one-dimensional subspace is the most computationally efficient for QML.

Overall, from the heat maps in Figure 6.4 and Figure 6.5, there is a clear trade-off between the two hyper-parameters. From characterising loss, we observed that the direct search optimiser either converges or does not, depending on the hyper-parameters set. But importantly, that any subspace dimension is capable of converging given a sufficient number of maximum evaluations per step. After characterising the number of loss function evaluations, we concluded that larger subspace dimensions are substantially less efficient, requiring more than double the number of loss function evaluations than low-dimensional subspaces. So with these two factors considered, the most efficient and accurate subspace dimension to use is a one-dimensional space. Remarkably, these empirical results for one-dimensional direct search in quantum variational circuits in this thesis reconfirm the empirical observations for classical machine learning applications of one-dimensional direct search originally considered in Ref. [83] as discussed earlier in Section 5.1.

6.3 Comparison of Gradient-free and Gradient-based Methods

Now having characterised the behaviour of hyper-parameters for direct search in reduced subspaces (Algorithms 1 and 2), we can now compare its performance to the gradient-based optimiser tested previously, Adam. This comparison is one of the main empirical results of this thesis and serves to highlight the difference in trainability of the same quantum variational circuit as a result of these two different optimisation techniques.

To enable a fair comparison between the two optimisers, we ensured that at least three

hyper-parameter configurations were tested for both Adam and direct search, with the complete table of settings listed in Table 6.1. Since Adam was already used in Chapter 4, we had prior observations of its sensitivity to the learning rate. Initial testing found that learning rates on the log scale such as 0.001 and 0.0001 produced little or no learning progress, while rates approaching 1.0 destabilised learning. This motivated the selection of the three values $\eta \in \{0.3, 0.1, 0.01\}$, where $\eta = 0.3$ provided faster aggressive learning, $\eta = 0.1$ was the stable and effective value used in Chapter 4, and $\eta = 0.01$ allowed for slower and more cautious learning.

In contrast, the direct search requires tuning both the reduced subspace dimension r and maximum evaluation budget per step E. Based on the hyper-parameter study in Section 6.1, we defined three representative configurations: a fast setting, using $r = 1$ with a low evaluation budget, a balanced setting using $r = 3$ with a moderate evaluation budget, and a safe setting using $r = 1$ with a high evaluation budget to prioritise convergence reliability. These configurations were used as baseline settings, with maximum evaluations E scaled upward with qubit number. For larger systems, the number of maximum evaluations per step E was doubled or tripled depending on performance, leading to the broader range of settings shown in Table 6.1 for the 7-10 qubit simulations. Each setting stated in Table 6.1 corresponds directly to a data group appearing in the comparison box plot in Figure 6.6.

Qubits	Direct Search		Adam	
	Settings (r, E)	No. Trials	Settings (η)	No. Trials
2	(1,100), **(3,150)**, (1,200)	5, **5**, 5	0.3, **0.1**, 0.01	5, **5**, 5
3	**(1,100)**, (3,150), (1,200)	**5**, 5, 5	**0.3**, 0.1, 0.01	**5**, 5, 5
4	**(1,100)**, (3,150), (1,200)	**5**, 5, 5	0.3, **0.1**, 0.01	5, **5**, 5
5	**(1,100)**, (3,150), (1,200)	**5**, 5, 5	0.3, **0.1**, 0.01	5, **5**, 5
6	**(1,100)**, (3,150), (1,200)	**5**, 5, 5	0.3, **0.1**, 0.01	5, **5**, 5
7	(1,100), (3,150), (1,200), (3,300), **(1,400)**	5, 5, 20, 15, **15**	0.3, **0.1**, 0.01	5, **5**, 5
8	(1,200), (1,300), (3,300), (1,400), (3,400), (1,500), (1,1000), (3,1500), **(1,2000)**	5, 1, 5, 4, 1, 1, 4, 3, **2**	0.3, **0.1**, 0.01	1, **1**, 1
9	(1,200), (1,300), (3,300), (1,400), (3,400), (1,500), (1,1000), **(3,1500)**	2, 1, 1, 1, 1, 1, 1, **1**	0.3, 0.1, **0.01**	1, 1, **1**
10	(1,300), (3,400), **(1,500)**	1, 1, **1**	0.3, **0.1**, 0.01	1, **1**, 1

TABLE 6.1: Direct Search and Adam optimiser settings versus qubit count. Settings are shown for each qubit, with the respective number of trials per setting separated by commas. The direct search settings are defined as (subspace dimension r, maximum evaluations E), and Adam's single value setting is learning rate η. The best performing setting is set in bold, with the corresponding value found in Table 6.2.

The main comparison of these two optimisers was an accuracy comparison where we considered the peak or best-step R^2 value reached by the optimiser as qubit number increases. With this we can compare whether direct search in reduced subspaces is a better scaling algorithm, if it is more capable of near-perfect learning as the number of qubits increases.

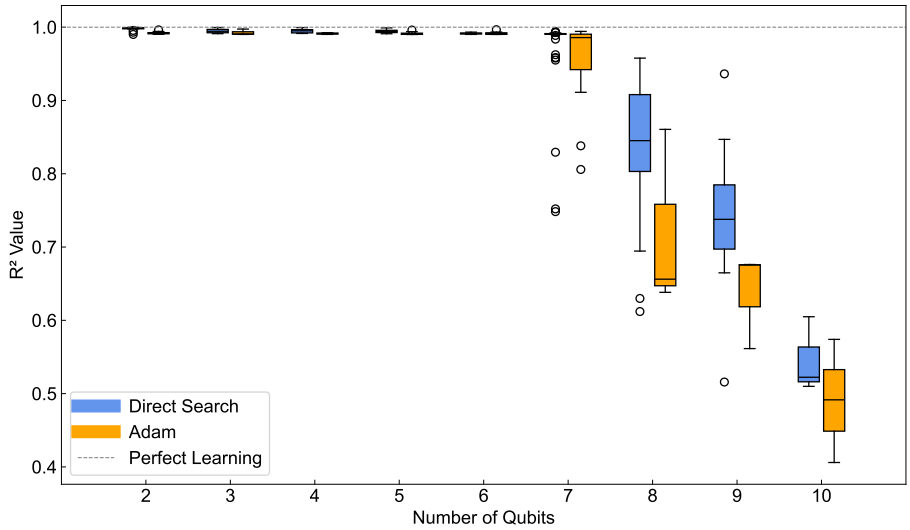

Figure 6.6: Comparison of learning performance between direct search in reduced subspaces (blue) and Adam (orange) optimisers across increasing qubit numbers. Each box represents the distribution of the best-step R^2 values averaged from multiple learning trials. The dashed grey line indicates perfect learning at $R^2 = 1$. The hollow circles indicate outliers, these mostly including poor optimiser settings - set too low initially being supplemented with extra data trials. At low qubit numbers from 2 to 6, both optimisers reliably achieve near-perfect learning, indicated by the proximity to the perfect learning line. The key difference between optimisers begins at 7 qubits and above, with the direct search algorithm capable of near perfect learning, while Adam has a wider distribution. Beyond 7 qubits, both optimisers are unreliable in accurately learning, with even maximum values not achieving perfect learning, however direct search is capable of learning slightly more accurately than Adam.

To properly represent the performance of the different settings for the two optimisers, we constructed a box-plot. In this box-plot the distribution of results can be shown, directly comparing the performance of different settings with the two optimisers.

The results of this comparison are shown in Figure 6.6, where the distribution of R^2 values is plotted for each optimiser for increasing qubit size circuits. For smaller circuit sizes (2-6), both optimisers can reliably achieve near perfect learning across all hyper-parameter settings. This is expected, as convergence in these problem sizes is easily achievable, and less sensitive to hyper-parameter settings.

Each box in Figure 6.6 represents the distribution of the best-step R^2 values obtained across all hyper-parameter tested at a given qubit number. In other words, for a fixed system size, a single box combines results from multiple configurations, as listed in Table 6.1, rather than the statistical variance of repeated runs of one setting. The spread in each box instead represents the variability across different hyper-parameter settings which are averaged over many trials. The boxes each have features standard to box plots, where the horizontal line in the middle denotes the median value of the distribution, the whiskers represent the minimum and maximum non-outlier values, and the hollow circles indicate outliers. The best performing settings and their corresponding R^2 values are summarised in Table 6.2.

Qubits	Direct Search		Adam	
	$R^2 \pm$ SEM	Setting (r, E)	$R^2 \pm$ SEM	Setting (η)
2	0.999 ± 0.0003	(3, 150)	0.993 ± 0.0008	0.1
3	0.997 ± 0.0007	(1, 200)	0.994 ± 0.0011	0.3
4	0.996 ± 0.0011	(1, 200)	0.992 ± 0.0003	0.1
5	0.995 ± 0.0011	(1, 200)	0.992 ± 0.0009	0.1
6	0.992 ± 0.0004	(1, 200)	0.993 ± 0.0010	0.1
7	0.991 ± 0.0005	(1, 400)	0.977 ± 0.0126	0.1
8	0.954 ± 0.0026	(1, 2000)	0.861 ± N/A	0.1
9	0.936 ± N/A	(3, 1500)	0.676 ± N/A	0.01
10	0.605 ± N/A	(1, 500)	0.574 ± N/A	0.1

TABLE 6.2: Average best-step R^2 performance versus qubit count for Direct Search and Adam optimisers, with the SEM shown as ±. The associated setting for the achieved R^2 value is alongside it, direct search settings are defined as (subspace dimension r, maximum evaluations E), and Adam's single value setting is learning rate η.

However, as the qubit number increases, convergence becomes less reliable and the spread in R^2 values broadens. This trend is also reflected numerically in Table 6.2, which lists the best R^2 value and its associated setting for each qubit number. This is behaviour is evident at 7 qubits. For Adam, the best performing setting no longer reaches near perfect learning $R^2 = 0.99$, with the peak value only reaching $R^2 = 0.977 \pm 0.0126$ (see Table 6.2), and poorer settings dropping to around $R^2 \sim 0.9$. This indicates that the range of learning rates tested are insufficient for Adam to near perfectly learn the $n = 7$ circuit. In contrast, direct search maintains a tight spread with all settings within near perfect learning, with only low evaluation budget configurations producing outliers. This indicates that direct search remains comparatively stable across settings, while Adam becomes increasingly sensitive and less effective at this scale. The presence of outliers for direct search still indicates that careful hyper-parameter tuning remains important.

At high qubit numbers (> 7), accurate learning and convergence is no longer achievable. Both optimisers show a decline in learning accuracy, with increasingly wider spreads in R^2 values, indicating that even with different hyper-parameter settings, near perfect learning is no longer possible. This is expected, as the circuit complexity grows and the optimisation problem becomes significantly harder with qubit number, which also results in a rapid increase in computational costs.

From 8 to 10 qubits, several trials remained unfinished (steps < 1000) due to computational runtime limits, which appears in Table 6.1 as settings with only a single trial. This arises as optimisation runtime scales exponentially with qubit number as discussed in Section 4.4.2, due to the exponential growth of the Hilbert space. Additionally, these simulations used the exact method for calculating measurements using density matrices, which requires storing matrices of size $2^n \times 2^n$, leading to rapidly increasing memory usage that becomes limiting beyond 8 qubits. These high resource costs made testing different hyper-parameters difficult. As a result, systematic analysis of different settings, as well as repeating trials to generate statistically significant results could not be realised.

Despite these constraints, direct search remains more capable of achieving accurate learning, with both a higher median performance and a higher peak R^2 value than Adam, as illustrated in Figure 6.6 and Table 6.2. This stronger performance can be attributed to the way direct search can allocate more maximum evaluations per optimisation step. In the box plot, the best performing configurations were often trials with a high number of maximum evaluations per step E, which is confirmed in Table 6.2. This suggests that increasing the maximum evaluations per step effectively allows each optimisation step to explore the landscape more thoroughly before a restart occurs. The restart mechanism further helps prevent the optimiser from remaining trapped in poor solutions. Together, the added depth of the search from increasing maximum evaluations E allows for valuable progress to be made in each step translating into improved convergence. We hypothesise that with a sufficiently large number of maximum evaluations E, that the direct search algorithm is capable of achieving near perfect learning on these large qubit models.

Adam's performance in this high-qubit region is fundamentally limited by the structure of the optimiser itself. As we know, Adam has only one primary hyper-parameter, the learning rate η. While Adam adaptively rescales and updates this learning rate during optimisation, as discussed in Section 4.3, there are no additional mechanisms that allow the optimiser to explore the loss landscape more thoroughly when optimisation becomes difficult. The learning rate range used here was chosen on prior successful performance on smaller qubit systems. As mentioned earlier, every trial was run for a fixed optimisation budget of 1000 steps. However, under these equal conditions, Adam does not reliably reach the near perfect convergence threshold of $R^2 = 0.99$ beyond 7-qubits, as shown in Table 6.2. This indicates that this reduced performance is not simply due to hyper-parameter choice, but rather reflects a limitation of Adam in this optimisation regime.

While direct search appears to outperform Adam, these results highlight a key limitation of our classical simulations of a quantum algorithm - the exponential increase in computational cost with qubit number. As these large qubit models require many loss function evaluation, each quantum circuit simulation requires storing large density matrices. As a result, both the runtime and memory costs for simulation increase exponentially. To make these simulations of large qubit circuits feasible, we must transition to using approximations for the circuit measurements by using finite quantum measurement, or shots. This method introduces sampling noise, but significantly reduces computational cost per iteration, specifically in the memory sense. In the following section, we explore the effect of finite shots through using the doubly stochastic direct search method described in Section 5.2, designed to reduce computational resource usage while retaining convergence.

6.4 Finite Shots and Doubly Stochastic Optimisation

Circuit measurements in quantum simulations can be expensive as qubit count grows, especially when using an exact density matrix method. As discussed in Chapter 5, an alternative method used to reduce the computational cost of circuit measurements is to use finite measurement shots. By using a large number of shots, we can use Equation (2.21) to accurately estimate expectation values. This significantly reduces computational and memory load in large qubit number simulations as the exact method must construct large density matrix

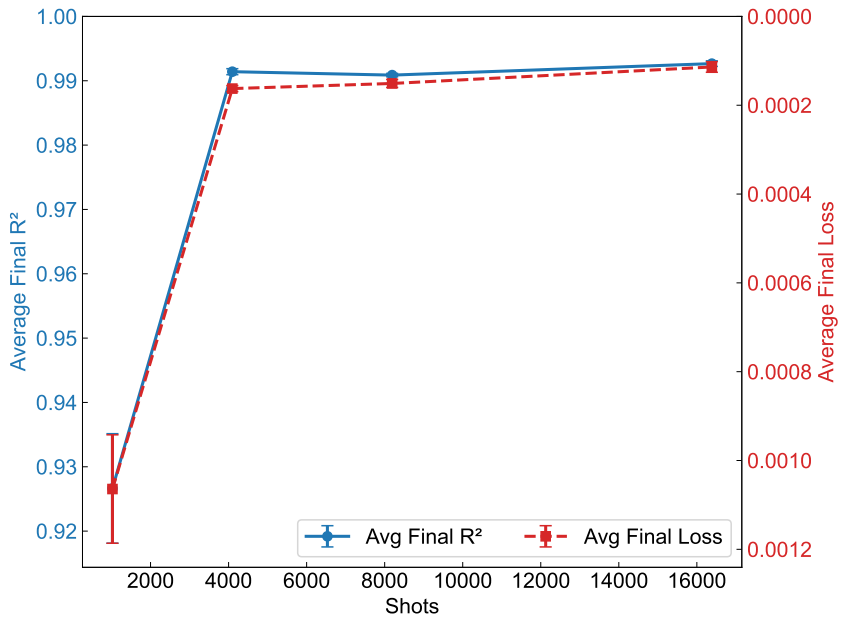

FIGURE 6.7: Average final R^2 value (blue) and loss (red) as the number of shots increases on a 3-qubit model at 50 maximum evaluations per step E. Each point represents the mean value across 5 trials with error bars of the standard error of the mean (SEM). The number of shots tested included $S = 1024, 4096, 8192, 16384$. The R^2 and loss lines do not match up perfectly due to the sampling noise in the system causing small fluctuations that misalign the correlation. The model can accurately learn at 4096 shots and above, showing that the doubly stochastic model is feasible and robust to shot noise.

representations of the circuit output. One key limitation of using the shot-based method is the introduction of shot sampling noise, which arises due to the approximation. Some optimisation methods have been shown to benefit from this additional noise [38]; this concept is explored in Section 5.2, where we introduce the doubly stochastic direct search approach.

6.4.1 Doubly Stochastic Direct Search

Doubly stochastic direct search extends the direct search in reduced subspaces method to the finite-shot regime. By utilising two sources of randomness: one from the random subspace sampling inherent to the algorithm, and the other from the stochasticity introduced by estimating the loss function using a finite number of shots. As detailed in Section 5.2, this adaptation preserves the theoretical convergence guarantees associated with direct search in reduced subspaces, while aiming to reduce computational costs associated with large qubit circuit simulations.

To demonstrate the practicality of the doubly stochastic direct search method, we perform a proof-of-concept test on a small scale circuit (3-qubits) to verify that accurate learning is possible in the finite-shot regime.

The results in Figure 6.7 demonstrate that the doubly stochastic direct search method can be successfully implemented. With a finite number of shots, this method is capable

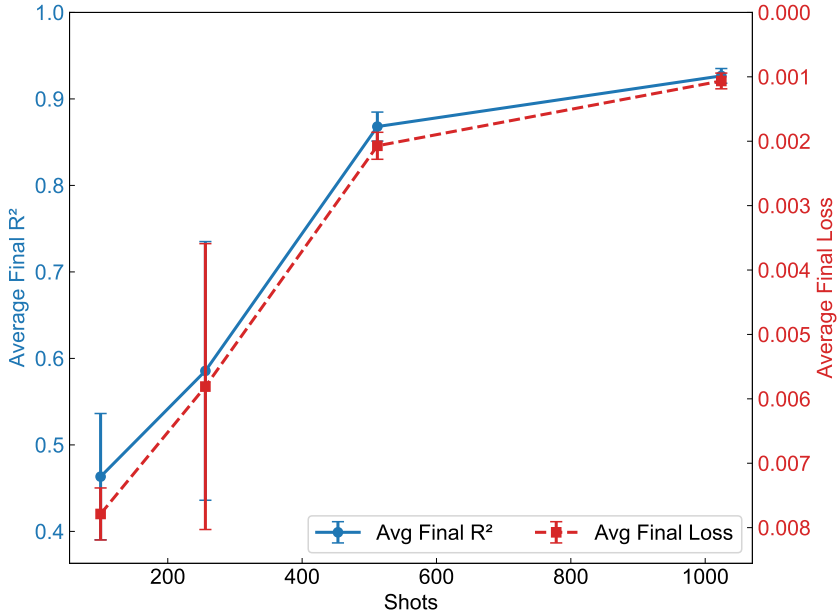

Figure 6.8: Average final R^2 value (blue) and loss (red) for a 3-qubit model at 50 maximum evaluations per step E. Each point represents the mean value across 3 trials, with SEM error bars. Data points include $S = 100, 256, 512, 1024$, we overlap $S = 1024$ from Figure 6.7 for comparison. The results show that direct search can still perform learning at low shot numbers, though performance degrades as sampling noise increases.

of achieving accurate learning. While lower shot numbers lead to large fluctuations in loss due to sampling noise, increasing the number of shots stabilises the optimiser and allows it to reliably reach near perfect learning. The 3-qubit model that was used here shows that $S = 4096$ shots is sufficient to ensure convergence at this scale. This confirms that forming an unbiased estimator of the loss using finite shots does not break the convergence guarantees of the direct search algorithm in practice.

To examine how the optimiser behaves when the number of measurement shots is very small, we additionally tested shot numbers $S = 1, 100, 256, 512$, extending below the range used in the main comparison.

The results in Figure 6.8 show that the optimiser can still make progress at lower shot counts, but performance degrades as sampling noise increases. We observe that the final R^2 increases steadily from $S = 100$ to $S = 512$, with a slower improvement towards $S = 1024$. This indicates that even relatively low shot numbers can still provide some degree of learning. However, when testing $S = 1$, the optimiser failed to learn in any capacity, with a final value of approximately $R^2 \approx -30$, indicating that the loss is completely dominated by sampling noise at this limit. While we initially hypothesised that single-shot measurements were possible for this method, the current implementation does not support this stable optimisation in this regime.

Overall, these results show that the doubly stochastic direct search is robust to moderate levels of sampling noise. However, there is a practical lower limit on the number of shots

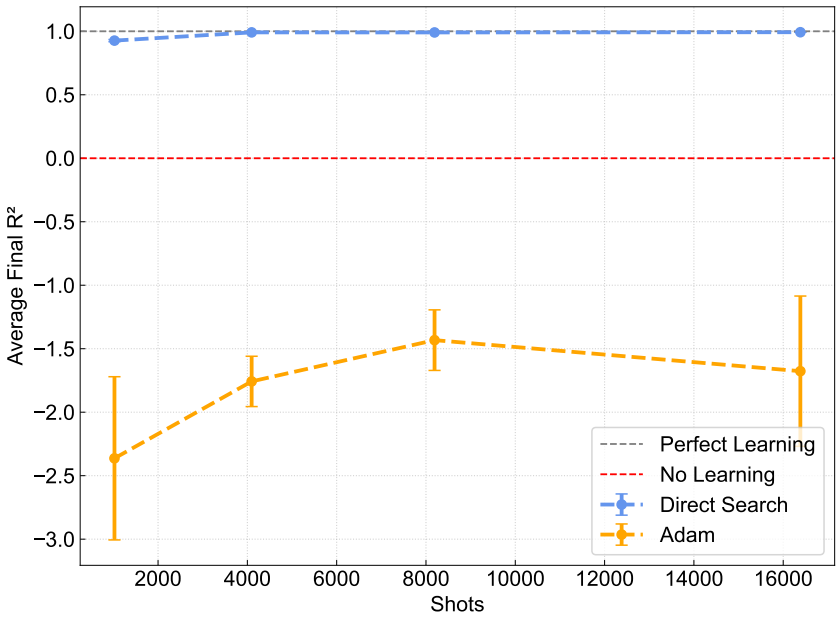

FIGURE 6.9: Comparison of doubly stochastic direct search (blue) and a shot-based variant of Adam (orange) using finite shots on a 3-qubit model. The average final R^2 is shown across increasing measurement shots, with a dashed red line to indicate no learning at $R^2 = 0$ and a grey perfect learning line at $R^2 = 1$. Adam fails to converge under finite shots, with a negative R^2 value across all shot counts due to the high variance in gradient estimates induced by shot noise. The error bars (SEM) also indicate high variance and instability in the Adam solution.

required to achieve accurate learning.

These results provide a proof-of-concept that doubly stochastic direct search can be implemented while preserving its convergence guarantees. Since this method relies on finite shots, it reduces computational costs while maintaining learning accuracy. This confirms that the doubly stochastic method is robust to sampling noise and can exploit this stochasticity to converge to an accurate solution.

6.4.2 Limitations of Gradient-Based Optimisers Under Shot Noise

To mirror our earlier comparison between direct search and Adam, we next investigate how gradient-based optimisers behave under shot noise. As outlined previously in Section 4.3, Adam is an inherently stochastic method through its use of mini-batch updates and adaptive moment estimates. However, despite this adaptive behaviour, Adam remains highly sensitive to small perturbations in gradients [91, 92]. When gradients are estimated from noisy measurements using a finite number of shots, rather than computed as exact expectation values, the noise can distort the descent direction, interfering with convergence behaviour.

We therefore directly compared the effect of shot noise on both Adam and doubly stochastic direct search method. When attempting to implement Adam by using shot-based loss

estimates, the optimiser becomes unstable. In this case, the noise in the estimated loss gradients can lead to improper descent directions, preventing consistent convergence and often causing the optimiser to move in the opposite direction to the global minimum.

As shown in Figure 6.9, the comparison between the doubly stochastic direct search and the shot-based variant of Adam illustrates their differing robustness under shot noise. Adam fails to converge producing negative R^2 values due to noisy gradients skewing the descent direction. This demonstrates that although the Adam optimiser is inherently stochastic, it cannot be made doubly stochastic due to the variance introduced by shot noise breaking its convergence behaviour.

In contrast, doubly stochastic direct search can utilise the extra form of stochasticity from finite shots without breaking convergence, because it relies on evaluating loss values rather than estimating gradients. This allows the optimiser to incorporate shot noise as part of the search mechanism, rather than being disrupted by it. This highlights its capability as a gradient-free optimiser, and its possible application in high qubit number simulations, where gradient-based methods like Adam may become impractical.

6.5 Does Algorithm 1 Overcome QML Trainability Issues?

An important question following the work presented in this thesis is whether the direct search in reduced subspaces method can mitigate the main trainability challenges in variational QML, namely barren plateaus and exponentially many local minima. Throughout this thesis, we have discussed the potential of gradient-free methods to avoid the stalled learning effects caused by difficult optimisation landscapes. Here, we consider which design choices of the direct search in reduced subspaces algorithm might contribute to robustness against these issues.

For barren plateaus, gradient-free methods do not rely on gradients and therefore are not immediately and obviously affected by vanishing gradients. However, in a landscape where most loss values have plateaued, the algorithm could repeatedly evaluate regions of similar loss values without seeing an improvement. The reduced subspace approach may perhaps mitigate this, through the exploration of a subset of parameters at each iteration, it is possible that some directions still can provide non-zero descent, which allows for some incremental learning. This reasoning is necessarily speculative, as our tested circuits are relatively small and shallow, and there is no definitive metric in our empirical testbed to confirm the presence of barren plateaus.

As discussed in Section 3.1.4, one mitigation strategy available for barren plateaus is the construction of a local cost/loss function. In our implementation of the quantum data re-uploading circuit, we measure only the first qubit, constructing a local loss function, as illustrated by Figure 3.3. The use of a local loss function means that the optimisation landscape is less likely to exhibit a barren plateau. A related but distinct concept of locality arises from the classical optimiser procedure itself. Specifically, by restricting optimisation to a reduced parameter subspace, the algorithm only explores a subset of directions at a time. In this sense, the optimiser experiences a local loss landscape, not because the loss function

is local, but because we only explore low-dimensional local sections of the landscape.

Through direct search using random subspaces, each iteration explores a slice of the parameter space, which may make the loss landscape appear local. This lowers the dimensionality of the optimisation problem and potentially creates a more favourable optimisation landscape due to the impossibility of over-parameterisation within the sampled subspace. However, this does not directly address the underlying cause of barren plateaus of the vanishing gradients in deep or wide circuits. As noted above, there is no definitive metric in our current empirical testbeds to quantify the presence of barren plateaus, so any observations remain speculative.

For the issue of exponentially many local minima, we hypothesise that the direct search implementation could be more robust than gradient-based methods. This robustness enables the optimiser to potentially hop out of poor minima until a sufficiently good minimum is found. In our protocol, the random subspace sampling reduces the likelihood of becoming trapped, as only a subset of parameters is optimised at each iteration. As discussed in Section 3.1.4, multiple restarts are a common mitigation strategy for these landscapes. In our implementation, the step-size reset acts as an implicit restart mechanism, allowing the search to periodically refresh and potentially escape poor local minima. In contrast, gradient-based methods typically require an explicit restart strategy to be implemented, as steepest descent offers no inherent mechanism to escape poor local minima.

It is important to emphasise that while our study focused on data re-uploading circuits trained with a mean squared error (MSE) loss, the reduced subspace direct search framework is not specific to this model. This means that the optimisation strategy can be easily extended to broader families of quantum variational algorithms by changing the loss function. For example, in variational eigensolving, the loss is constructed directly from measured outcomes [45]. In this way, the optimisation procedure generalises naturally beyond the predictive setting considered here.

Our empirical results provide early evidence that the reduced subspace method shows promise for mitigating some aspects of trainability challenges in these applications. However, further theoretical analysis is required to determine the influence of random reduced parameter subspaces on VQA trainability in future work.

7
Conclusions and Next Steps

Variational quantum algorithms (VQAs) are hybrid quantum-classical frameworks in which a parameterised quantum circuit is trained using a classical optimiser to solve quantum machine learning tasks. These techniques are popular in designing current QML models. While most existing VQA literature relies on gradient-based optimisation methods, such as stochastic gradient descent (SGD) and Adam, this thesis investigated a gradient-free optimisation algorithm for VQAs.

In this work, we studied a novel gradient-free optimisation method - the direct search in reduced subspaces algorithm [83]. The algorithm combines probabilistic updates with a reduction in the parameter space dimension. At each iteration, randomly chosen subspaces allow the optimiser to explore smaller, lower dimension regions of parameter space, which reduces problem complexity. Because the search directions are aligned with random subspaces, the method retains stochastic search directions which improve exploration and convergence properties through probabilistic descent. We further extended this direct search method to the finite-shot limit by proposing a doubly stochastic variant that combined random subspace sampling with finite stochastic measurements. This provided a framework for gradient-free optimisation that remains robust in the finite-shot regime.

The second novel contribution of this thesis was the adaptation and application of the direct search in reduced subspaces algorithm to quantum models for the first time, with a specific focus on universal QML models for classification and regression tasks. We implemented the method within a QML testbed, analysing its performance across different hyper-parameters, determining how each affects optimisation behaviour. Our results showed that the reduced subspace direct search algorithm can match or outperform Adam in terms of learning accuracy for modest qubit numbers. Moreover, in finite-shot regimes where gradient estimates become unreliable, direct search maintained convergence and demonstrated resilience to noise, where Adam failed to train effectively. These findings indicated that direct search is a promising candidate for scalable and noise-robust optimisation in variational

quantum models.

Remarkably, our results for quantum variational circuits reflect the same trend observed in classical machine learning applications of direct search. Namely, that using a reduced random subspace of dimension $r = 1$ consistently produced the best learning performance. This behaviour mirrored the classical machine learning result found in Ref. [83], where one-dimensional subspaces were shown to be optimal for sampling random directions. A natural next step is to understand why this behaviour persists in a quantum setting, and whether it remains optimal under different operational conditions.

A second compelling direction is to determine if our doubly-stochastic direct search method retains performance if loss functions can only access a single-shot quantum measurement, instead of a finite set of measurements. The doubly-stochastic quantum gradient optimiser introduced in Ref. [38] has shown promising results in the single-shot regime. We plan to further develop and establish whether our proposed doubly-stochastic direct search can achieve accurate learning in the single-shot limit. Both our optimiser and that of Ref. [38] rely on two sources of randomness: stochasticity in loss value/gradient calculations, and additional randomness from finite quantum measurements. We aim to compare our protocol to the doubly stochastic gradient descent variant [38] and evaluate the differences between doubly stochastic gradient-free and gradient-based methods in terms of convergence, robustness and computational efficiency.

Several additional avenues for future work naturally arise from this study. One is to determine whether this class of algorithms can mitigate the effects of trainability challenges such as barren plateaus, while still achieving accurate learning. Another is to characterise how the computational resource requirements scale relative to standard gradient-based methods. Addressing these aspects will help guide the development of scalable, stochastic and gradient-free optimisation strategies for practical quantum machine learning applications, and may help establish gradient-free methods as a viable alternative to gradient-based training in future VQA research.

References

[1] M. Cerezo, A. Arrasmith, R. Babbush, S. C. Benjamin, S. Endo, K. Fujii, J. R. McClean, K. Mitarai, X. Yuan, L. Cincio, and P. J. Coles. *Variational Quantum Algorithms*. Nature Reviews Physics **3**(9), 625 (2021). ArXiv:2012.09265 [quant-ph], URL `http://arxiv.org/abs/2012.09265`.

[2] M. Schuld. *Supervised quantum machine learning models are kernel methods* (2021). ArXiv:2101.11020 [quant-ph], URL `http://arxiv.org/abs/2101.11020`.

[3] M. Hibat-Allah, M. Mauri, J. Carrasquilla, and A. Perdomo-Ortiz. *A framework for demonstrating practical quantum advantage: comparing quantum against classical generative models*. Communications Physics **7**(1), 68 (2024). Publisher: Nature Publishing Group, URL `https://www.nature.com/articles/s42005-024-01552-6`.

[4] V. Havlicek, A. D. Córcoles, K. Temme, A. W. Harrow, A. Kandala, J. M. Chow, and J. M. Gambetta. *Supervised learning with quantum enhanced feature spaces*. Nature **567**(7747), 209 (2019). ArXiv:1804.11326 [quant-ph], URL `http://arxiv.org/abs/1804.11326`.

[5] J. R. McClean, S. Boixo, V. N. Smelyanskiy, R. Babbush, and H. Neven. *Barren plateaus in quantum neural network training landscapes*. Nature Communications **9**(1), 4812 (2018). URL `https://www.nature.com/articles/s41467-018-07090-4`.

[6] E. R. Anschuetz and B. T. Kiani. *Quantum variational algorithms are swamped with traps*. Nature Communications **13**(1), 7760 (2022). Publisher: Nature Publishing Group, URL `https://www.nature.com/articles/s41467-022-35364-5`.

[7] S. Wang, E. Fontana, M. Cerezo, K. Sharma, A. Sone, L. Cincio, and P. J. Coles. *Noise-induced barren plateaus in variational quantum algorithms*. Nature Communications **12**(1), 6961 (2021). Publisher: Nature Publishing Group, URL `https://www.nature.com/articles/s41467-021-27045-6`.

[8] S. Sim, P. D. Johnson, and A. Aspuru-Guzik. *Expressibility and entangling capability of parameterized quantum circuits for hybrid quantum-classical algorithms*. Advanced Quantum Technologies **2**(12), 1900070 (2019). ArXiv:1905.10876 [quant-ph], URL `http://arxiv.org/abs/1905.10876`.

[9] S. Thudumu, J. Fisher, and H. Du. *Supervised Quantum Machine Learning: A Future Outlook from Qubits to Enterprise Applications* (2025). ArXiv:2505.24765 [quant-ph] version: 1, URL `http://arxiv.org/abs/2505.24765`.

[10] A. Arrasmith, M. Cerezo, P. Czarnik, L. Cincio, and P. J. Coles. *Effect of barren plateaus on gradient-free optimization*. Quantum **5**, 558 (2021). Publisher: Verein zur Förderung des Open Access Publizierens in den Quantenwissenschaften, URL `https://quantum-journal.org/papers/q-2021-10-05-558/`.

[11] M. Schuld, R. Sweke, and J. J. Meyer. *Effect of data encoding on the expressive power of variational quantum-machine-learning models*. Physical Review A **103**(3), 032430 (2021). URL `https://link.aps.org/doi/10.1103/PhysRevA.103.032430`.

[12] C. v. Rossum, S. Shrapnel, and R. Gupta. *Exploiting biased noise in variational quantum models* (2025). ArXiv:2510.24050 [quant-ph], URL http://arxiv.org/abs/2510.24050.

[13] M. A. Nielsen and I. L. Chuang. *Quantum Computation and Quantum Information: 10th Anniversary Edition* (2010). ISBN: 9780511976667 Publisher: Cambridge University Press, URL https://www.cambridge.org/highereducation/books/quantum-computation-and-quantum-information/01E10196D0A682A6AEFFEA52D53BE9AE.

[14] M. Schuld and F. Petruccione. *Supervised Learning with Quantum Computers*. Quantum Science and Technology (Springer International Publishing, Cham, 2018). URL https://link.springer.com/10.1007/978-3-319-96424-9.

[15] J. J. Sakurai and J. Napolitano. *Modern Quantum Mechanics* (Cambridge University Press, 2020), 3 ed. URL https://www.cambridge.org/core/product/identifier/9781108587280/type/book.

[16] *Bloch sphere* (2025). Page Version ID: 1319916543, URL https://en.wikipedia.org/w/index.php?title=Bloch_sphere&oldid=1319916543.

[17] A. J, A. Adedoyin, J. Ambrosiano, P. Anisimov, W. Casper, G. Chennupati, C. Coffrin, H. Djidjev, D. Gunter, S. Karra, N. Lemons, S. Lin, A. Malyzhenkov, D. Mascarenas, S. Mniszewski, B. Nadiga, D. O'Malley, D. Oyen, S. Pakin, L. Prasad, R. Roberts, P. Romero, N. Santhi, N. Sinitsyn, P. J. Swart, J. G. Wendelberger, B. Yoon, R. Zamora, W. Zhu, S. Eidenbenz, A. Bärtschi, P. J. Coles, M. Vuffray, and A. Y. Lokhov. *Quantum Algorithm Implementations for Beginners*. ACM Transactions on Quantum Computing **3**(4), 1 (2022). ArXiv:1804.03719 [cs], URL http://arxiv.org/abs/1804.03719.

[18] *Bell state* (2025). Page Version ID: 1314388682, URL https://en.wikipedia.org/w/index.php?title=Bell_state&oldid=1314388682.

[19] I. Goodfellow, Y. Bengio, and A. Courville. *Deep Learning* (MIT Press, 2016). URL http://www.deeplearningbook.org.

[20] C. M. Bishop. *Pattern recognition and machine learning*. Information science and statistics (Springer, New York, 2006).

[21] S. P. Boyd and L. Vandenberghe. *Convex optimization* (Cambridge University Press, Cambridge New York Melbourne New Delhi Singapore, 2023), version 29 ed.

[22] Y. Carmon, O. Hinder, J. C. Duchi, and A. Sidford. *"Convex Until Proven Guilty": Dimension-Free Acceleration of Gradient Descent on Non-Convex Functions* (2017). ArXiv:1705.02766 [math], URL http://arxiv.org/abs/1705.02766.

[23] S. Ruder. *An overview of gradient descent optimization algorithms* (2017). ArXiv:1609.04747 [cs], URL http://arxiv.org/abs/1609.04747.

[24] A. Dereventsov, C. G. Webster, and J. D. D. Jr. *An adaptive stochastic gradient-free approach for high-dimensional blackbox optimization* (2022). ArXiv:2006.10887 [math], URL http://arxiv.org/abs/2006.10887.

[25] D. P. Kingma and J. Ba. *Adam: A Method for Stochastic Optimization* (2017). ArXiv:1412.6980 [cs], URL http://arxiv.org/abs/1412.6980.

[26] T. L. Patti, J. Kossaifi, A. Anandkumar, and S. F. Yelin. *Variational quantum optimization with multibasis encodings*. Physical Review Research **4**(3), 033142 (2022). URL https://link.aps.org/doi/10.1103/PhysRevResearch.4.033142.

[27] Z. Holmes, K. Sharma, M. Cerezo, and P. J. Coles. *Connecting ansatz expressibility to gradient magnitudes and barren plateaus.* PRX Quantum **3**(1), 010313 (2022). ArXiv:2101.02138 [quant-ph], URL http://arxiv.org/abs/2101.02138.

[28] M. Larocca, N. Ju, D. García-Martín, P. J. Coles, and M. Cerezo. *Theory of overparametrization in quantum neural networks.* Nature Computational Science **3**(6), 542 (2023). ArXiv:2109.11676 [quant-ph], URL http://arxiv.org/abs/2109.11676.

[29] M. Cerezo, A. Sone, T. Volkoff, L. Cincio, and P. J. Coles. *Cost function dependent barren plateaus in shallow parametrized quantum circuits.* Nature Communications **12**(1), 1791 (2021). Publisher: Nature Publishing Group, URL https://www.nature.com/articles/s41467-021-21728-w.

[30] M. Wiedmann, M. Periyasamy, and D. D. Scherer. *Fourier Analysis of Variational Quantum Circuits for Supervised Learning* (2024). ArXiv:2411.03450 [cs], URL http://arxiv.org/abs/2411.03450.

[31] K. Mitarai, M. Negoro, M. Kitagawa, and K. Fujii. *Quantum Circuit Learning.* Physical Review A **98**(3), 032309 (2018). ArXiv:1803.00745 [quant-ph], URL http://arxiv.org/abs/1803.00745.

[32] Y. Sun, Z. Wu, Y. Ma, and V. Tresp. *Quantum Architecture Search with Unsupervised Representation Learning* (2025). ArXiv:2401.11576 [quant-ph], URL http://arxiv.org/abs/2401.11576.

[33] M. Ghasemi and D. Ebrahimi. *Introduction to Reinforcement Learning* (2024). ArXiv:2408.07712 [cs], URL http://arxiv.org/abs/2408.07712.

[34] Y. Peng, X. Li, Z. Zhang, S. Y.-C. Chen, Z. Liang, and Y. Wang. *Breaking Through Barren Plateaus: Reinforcement Learning Initializations for Deep Variational Quantum Circuits* (2025). ArXiv:2508.18514 [cs], URL http://arxiv.org/abs/2508.18514.

[35] A. Kundu. *Reinforcement learning-assisted quantum architecture search for variational quantum algorithms* (2024). ArXiv:2402.13754 [quant-ph], URL http://arxiv.org/abs/2402.13754.

[36] X. Guo, T. Muta, and J. Zhao. *Quantum Circuit Ansatz: Patterns of Abstraction and Reuse of Quantum Algorithm Design* (2024). ArXiv:2405.05021 [cs], URL http://arxiv.org/abs/2405.05021.

[37] X. You and X. Wu. *Exponentially Many Local Minima in Quantum Neural Networks* (2021). ArXiv:2110.02479 [quant-ph], URL http://arxiv.org/abs/2110.02479.

[38] R. Sweke, F. Wilde, J. Meyer, M. Schuld, P. K. Faehrmann, B. Meynard-Piganeau, and J. Eisert. *Stochastic gradient descent for hybrid quantum-classical optimization.* Quantum **4**, 314 (2020). Publisher: Verein zur Förderung des Open Access Publizierens in den Quantenwissenschaften, URL https://quantum-journal.org/papers/q-2020-08-31-314/.

[39] O. Lockwood. *An Empirical Review of Optimization Techniques for Quantum Variational Circuits* (2022). ArXiv:2202.01389 [quant-ph], URL http://arxiv.org/abs/2202.01389.

[40] J. A. Nelder and R. Mead. *A Simplex Method for Function Minimization.* The Computer Journal **7**(4), 308 (1965). URL https://academic.oup.com/comjnl/article-lookup/doi/10.1093/comjnl/7.4.308.

[41] M. J. D. Powell. *A Direct Search Optimization Method That Models the Objective and Constraint Functions by Linear Interpolation.* In S. Gomez and J.-P. Hennart, eds., *Advances in Optimization and Numerical Analysis*, pp. 51–67 (Springer Netherlands, Dordrecht, 1994). URL http://link.springer.com/10.1007/978-94-015-8330-5_4.

[42] C. Cartis, L. Roberts, and O. Sheridan-Methven. *Escaping local minima with derivative-free methods: a numerical investigation.* Optimization **71**(8), 2343 (2022). ArXiv:1812.11343 [math], URL http://arxiv.org/abs/1812.11343.

[43] S. Al-Abri, T. X. Lin, M. Tao, and F. Zhang. *A Derivative-Free Optimization Method With Application to Functions With Exploding and Vanishing Gradients.* IEEE Control Systems Letters **5**(2), 587 (2021). URL https://ieeexplore.ieee.org/document/9123920/.

[44] C. Audet and M. Kokkolaras. *Blackbox and derivative-free optimization: theory, algorithms and applications.* Optimization and Engineering **17**(1), 1 (2016). URL https://doi.org/10.1007/s11081-016-9307-4.

[45] T. Shirakawa, J. Robledo-Moreno, T. Itoko, V. Tripathi, K. Ueda, Y. Kawashima, L. Broers, W. Kirby, H. Pathak, H. Paik, M. Tsuji, Y. Kodama, M. Sato, C. Evangelinos, S. Seelam, R. Walkup, S. Yunoki, M. Motta, P. Jurcevic, H. Horii, and A. Mezzacapo. *Closed-loop calculations of electronic structure on a quantum processor and a classical supercomputer at full scale* (2025). ArXiv:2511.00224 [quant-ph], URL http://arxiv.org/abs/2511.00224.

[46] A. Macaluso. *Quantum Supervised Learning.* KI - Künstliche Intelligenz **38**(4), 277 (2024). ArXiv:2407.17161 [cs], URL http://arxiv.org/abs/2407.17161.

[47] G. B. Fotopoulos, P. Popovich, and N. H. Papadopoulos. *Review Non-convex Optimization Method for Machine Learning* (2024). ArXiv:2410.02017 [cs], URL http://arxiv.org/abs/2410.02017.

[48] C. Jin, R. Ge, P. Netrapalli, S. M. Kakade, and M. I. Jordan. *How to Escape Saddle Points Efficiently* (2017). ArXiv:1703.00887 [cs], URL http://arxiv.org/abs/1703.00887.

[49] L. Bittel and M. Kliesch. *Training variational quantum algorithms is NP-hard.* Physical Review Letters **127**(12), 120502 (2021). ArXiv:2101.07267 [quant-ph], URL http://arxiv.org/abs/2101.07267.

[50] M. Cerezo, G. Verdon, H.-Y. Huang, L. Cincio, and P. J. Coles. *Challenges and Opportunities in Quantum Machine Learning.* Nature Computational Science **2**(9), 567 (2022). ArXiv:2303.09491 [quant-ph], URL http://arxiv.org/abs/2303.09491.

[51] Y. Yao and Y. Hasegawa. *Avoiding Barren Plateaus with Entanglement.* Physical Review A **111**(2), 022426 (2025). ArXiv:2406.03748 [quant-ph], URL http://arxiv.org/abs/2406.03748.

[52] A. Arrasmith, Z. Holmes, M. Cerezo, and P. J. Coles. *Equivalence of quantum barren plateaus to cost concentration and narrow gorges.* Quantum Science and Technology **7**(4), 045015 (2022). Publisher: IOP Publishing, URL https://dx.doi.org/10.1088/2058-9565/ac7d06.

[53] M. Larocca, S. Thanasilp, S. Wang, K. Sharma, J. Biamonte, P. J. Coles, L. Cincio, J. R. McClean, Z. Holmes, and M. Cerezo. *A Review of Barren Plateaus in Variational Quantum Computing* (2024). ArXiv:2405.00781 [quant-ph] version: 1, URL http://arxiv.org/abs/2405.00781.

[54] E. R. Anschuetz. *Critical Points in Quantum Generative Models* (2023). ArXiv:2109.06957 [quant-ph], URL http://arxiv.org/abs/2109.06957.

[55] E. R. Anschuetz. *A Unified Theory of Quantum Neural Network Loss Landscapes* (2025). ArXiv:2408.11901 [quant-ph], URL http://arxiv.org/abs/2408.11901.

[56] A. Pesah, M. Cerezo, S. Wang, T. Volkoff, A. T. Sornborger, and P. J. Coles. *Absence of Barren Plateaus in Quantum Convolutional Neural Networks.* Physical Review X **11**(4), 041011 (2021). ArXiv:2011.02966 [quant-ph], URL http://arxiv.org/abs/2011.02966.

[57] K. Zhang, M.-H. Hsieh, L. Liu, and D. Tao. *Toward Trainability of Quantum Neural Networks* (2020). ArXiv:2011.06258 [quant-ph], URL http://arxiv.org/abs/2011.06258.

[58] A. Skolik, J. R. McClean, M. Mohseni, P. van der Smagt, and M. Leib. *Layerwise learning for quantum neural networks.* Quantum Machine Intelligence **3**(1), 5 (2021). URL https://doi.org/10.1007/s42484-020-00036-4.

[59] E. Grant, L. Wossnig, M. Ostaszewski, and M. Benedetti. *An initialization strategy for addressing barren plateaus in parametrized quantum circuits.* Quantum **3**, 214 (2019). ArXiv:1903.05076 [quant-ph], URL http://arxiv.org/abs/1903.05076.

[60] D. S. Bagaev, M. A. Gavreev, A. S. Mastiukova, A. K. Fedorov, and N. A. Nemkov. *Regularizing quantum loss landscapes by noise injection* (2025). ArXiv:2505.08759 [quant-ph] version: 1, URL http://arxiv.org/abs/2505.08759.

[61] M. Wang, B. Fang, A. Li, and P. Nair. *Efficient QAOA Optimization using Directed Restarts and Graph Lookup.* In *Proceedings of the 2023 International Workshop on Quantum Classical Cooperative*, pp. 5–8 (ACM, Orlando FL USA, 2023). URL https://dl.acm.org/doi/10.1145/3588983.3596680.

[62] M. Wang, P. Das, and P. J. Nair. *Qoncord: A Multi-Device Job Scheduling Framework for Variational Quantum Algorithms* (2024). ArXiv:2409.12432 [quant-ph] version: 1, URL http://arxiv.org/abs/2409.12432.

[63] J. Napp. *Quantifying the barren plateau phenomenon for a model of unstructured variational ansätze* (2022). ArXiv:2203.06174 [quant-ph], URL http://arxiv.org/abs/2203.06174.

[64] I. Cong, S. Choi, and M. D. Lukin. *Quantum convolutional neural networks.* Nature Physics **15**(12), 1273 (2019). Publisher: Nature Publishing Group, URL https://www.nature.com/articles/s41567-019-0648-8.

[65] A. Mari, T. R. Bromley, and N. Killoran. *Estimating the gradient and higher-order derivatives on quantum hardware.* Physical Review A **103**(1), 012405 (2021). ArXiv:2008.06517 [quant-ph], URL http://arxiv.org/abs/2008.06517.

[66] M. Schuld, V. Bergholm, C. Gogolin, J. Izaac, and N. Killoran. *Evaluating analytic gradients on quantum hardware.* Physical Review A **99**(3), 032331 (2019). ArXiv:1811.11184 [quant-ph], URL http://arxiv.org/abs/1811.11184.

[67] L. Bottou, F. E. Curtis, and J. Nocedal. *Optimization Methods for Large-Scale Machine Learning* (2018). ArXiv:1606.04838 [stat], URL http://arxiv.org/abs/1606.04838.

[68] A. Arrasmith, L. Cincio, R. D. Somma, and P. J. Coles. *Operator Sampling for Shot-frugal Optimization in Variational Algorithms* (2020). ArXiv:2004.06252 [quant-ph], URL http://arxiv.org/abs/2004.06252.

[69] T. G. Kolda, R. M. Lewis, and V. Torczon. *Optimization by Direct Search: New Perspectives on Some Classical and Modern Methods.* SIAM Review **45**(3), 385 (2003). URL http://epubs.siam.org/doi/10.1137/S0036144502428889.

[70] D. Wang, D. Tan, and L. Liu. *Particle swarm optimization algorithm: an overview.* Soft Computing **22**(2), 387 (2018). URL http://link.springer.com/10.1007/s00500-016-2474-6.

[71] A. Benítez-Buenache and Q. Portell-Montserrat. *Bayesian Parameterized Quantum Circuit Optimization (BPQCO): A task and hardware-dependent approach* (2024). ArXiv:2404.11253 [quant-ph] version: 1, URL http://arxiv.org/abs/2404.11253.

[72] O. I. Siddiqui, N. Innan, A. Marchisio, M. Bennai, and M. Shafique. *Quantum Bayesian Networks for Machine Learning in Oil-Spill Detection* (2024). ArXiv:2412.19843 [quant-ph] version: 1, URL http://arxiv.org/abs/2412.19843.

[73] M. H. Wright. *Direct Search Methods: Once Scorned, Now Respectable*.* Numerical Analysis 1995: Proceedings of the 1995 Dundee Biennial Conference in Numerical Analysis pp. 191–208 (1996). URL https://bemlar.ism.ac.jp/zhuang/Refs/Refs/wright1995numana.pdf.

[74] M. J. D. Powell. *An efficient method for finding the minimum of a function of several variables without calculating derivatives.* The Computer Journal **7**(2), 155 (1964). URL https://doi.org/10.1093/comjnl/7.2.155.

[75] A. Pellow-Jarman, I. Sinayskiy, A. Pillay, and F. Petruccione. *A Comparison of Various Classical Optimizers for a Variational Quantum Linear Solver.* Quantum Information Processing **20**(6), 202 (2021). ArXiv:2106.08682 [quant-ph], URL http://arxiv.org/abs/2106.08682.

[76] T. H. Rowan. *Functional stability analysis of numerical algorithms.* Ph.D. thesis, The University of Texas (1990). URL https://www.proquest.com/openview/7dc600542f24d644de536b3ec0fbcdd7/1?pq-origsite=gscholar&cbl=18750&diss=y.

[77] D. Byatt. *CONVERGENT VARIANTS OF THE NELDER-MEAD ALGORITHM.* Ph.D. thesis, University of Canterbury (2000). URL https://ir.canterbury.ac.nz/server/api/core/bitstreams/2301895c-8155-4c49-9f03-a1f455a823c5/content.

[78] V. S. Vassiliadis and R. Conejeros. *Powell Method.* In *Encyclopedia of Optimization*, pp. 2001–2003 (Springer, Boston, MA, 2001). URL https://link.springer.com/rwe/10.1007/0-306-48332-7_393.

[79] M. J. D. Powell. *A view of algorithms for optimization without derivatives* (2007). URL https://www.damtp.cam.ac.uk/user/na/NA_papers/NA2007_03.pdf.

[80] V. J. Torczon. *Multi-Directional Search: A Direct Search Algorithm for Parallel Machines* (1989).

[81] R. M. Lewis, V. Torczon, and M. W. Trosset. *Direct search methods: then and now.* Journal of Computational and Applied Mathematics **124**(1-2), 191 (2000). URL https://linkinghub.elsevier.com/retrieve/pii/S0377042700004234.

[82] R. M. Gower, N. Loizou, X. Qian, A. Sailanbayev, E. Shulgin, and P. Richtarik. *SGD: General Analysis and Improved Rates* (2019). ArXiv:1901.09401 [cs], URL http://arxiv.org/abs/1901.09401.

[83] L. Roberts and C. W. Royer. *Direct search based on probabilistic descent in reduced spaces* (2023). ArXiv:2204.01275 [math], URL http://arxiv.org/abs/2204.01275.

[84] W. Hare, L. Roberts, and C. W. Royer. *Expected decrease for derivative-free algorithms using random subspaces* (2023). ArXiv:2308.04734 [math], URL http://arxiv.org/abs/2308.04734.

[85] K. J. Dzahini, F. Rinaldi, C. W. Royer, and D. Zeffiro. *Direct-search methods in the year 2025: Theoretical guarantees and algorithmic paradigms* (2025). ArXiv:2403.05322 [math], URL http://arxiv.org/abs/2403.05322.

[86] J. C. Lagarias, J. A. Reeds, M. H. Wright, and P. E. Wright. *Convergence Properties of the Nelder–Mead Simplex Method in Low Dimensions.* SIAM Journal on Optimization **9**(1), 112 (1998). URL http://epubs.siam.org/doi/10.1137/S1052623496303470.

[87] C. Audet and W. Hare. *Derivative-Free and Blackbox Optimization.* Springer Series in Operations Research and Financial Engineering (Springer International Publishing, Cham, 2017). URL http://link.springer.com/10.1007/978-3-319-68913-5.

[88] Dell Technologies Pty Ltd. *Bunya* (2024). URL https://rcc.uq.edu.au/systems/high-performance-computing/bunya.

[89] N. A. Nemkov. *Barren plateaus swamped with traps.* Physical Review A **111**(1) (2025).

[90] A. Pérez-Salinas, A. Cervera-Lierta, E. Gil-Fuster, and J. I. Latorre. *Data re-uploading for a universal quantum classifier*. Quantum **4**, 226 (2020). ArXiv:1907.02085 [quant-ph], URL http://arxiv.org/abs/1907.02085.

[91] L. Balles and P. Hennig. *Dissecting Adam: The Sign, Magnitude and Variance of Stochastic Gradients* (2020). ArXiv:1705.07774 [cs], URL http://arxiv.org/abs/1705.07774.

[92] P. Zhou, J. Feng, C. Ma, C. Xiong, S. Hoi, and W. E. *Towards Theoretically Understanding Why SGD Generalizes Better Than ADAM in Deep Learning* (2021). ArXiv:2010.05627 [cs], URL http://arxiv.org/abs/2010.05627.

[93] A. Javadi-Abhari, M. Treinish, K. Krsulich, C. J. Wood, J. Lishman, J. Gacon, S. Martiel, P. D. Nation, L. S. Bishop, A. W. Cross, B. R. Johnson, and J. M. Gambetta. *Quantum computing with Qiskit* (2024). ArXiv:2405.08810 [quant-ph], URL http://arxiv.org/abs/2405.08810.

[94] I. L. Markov and Y. Shi. *Simulating quantum computation by contracting tensor networks*. SIAM Journal on Computing **38**(3), 963 (2008). ArXiv:quant-ph/0511069, URL http://arxiv.org/abs/quant-ph/0511069.

[95] J. Preskill. *Quantum Computing in the NISQ era and beyond*. Quantum **2**, 79 (2018). ArXiv:1801.00862 [quant-ph], URL http://arxiv.org/abs/1801.00862.

[96] S. Boixo, S. V. Isakov, V. N. Smelyanskiy, and H. Neven. *Simulation of low-depth quantum circuits as complex undirected graphical models* (2018). ArXiv:1712.05384 [quant-ph], URL http://arxiv.org/abs/1712.05384.

[97] U. Shrawankar and M. Joshi. *Overhead Management in Multi-Core Environment* (2022). ArXiv:2202.06744 [cs], URL http://arxiv.org/abs/2202.06744.

[98] R. Hood, H. Jin, P. Mehrotra, J. Chang, J. Djomehri, S. Gavali, D. Jespersen, K. Taylor, and R. Biswas. *Performance impact of resource contention in multicore systems*. In *2010 IEEE International Symposium on Parallel & Distributed Processing (IPDPS)*, pp. 1–12 (IEEE, Atlanta, GA, USA, 2010). URL https://ieeexplore.ieee.org/document/5470399/.

[99] A. Fedorova, S. Blagodurov, and S. Zhuravlev. *Managing contention for shared resources on multicore processors*. Communications of the ACM **53**(2), 49 (2010). URL https://dl.acm.org/doi/10.1145/1646353.1646371.

[100] M. Y. Javed, A. F. Murtaza, Q. Ling, S. Qamar, and M. M. Gulzar. *A novel MPPT design using generalized pattern search for partial shading*. Energy and Buildings **133**, 59 (2016). URL https://www.sciencedirect.com/science/article/pii/S0378778816309252.

[101] J. J. Moré and D. J. Thuente. *Line search algorithms with guaranteed sufficient decrease*. ACM Transactions on Mathematical Software **20**(3), 286 (1994). URL https://dl.acm.org/doi/10.1145/192115.192132.

[102] F. Shang, Y. Liu, K. Zhou, J. Cheng, K. K. W. Ng, and Y. Yoshida. *Guaranteed Sufficient Decrease for Stochastic Variance Reduced Gradient Optimization* (2018). ArXiv:1802.09933 [stat], URL http://arxiv.org/abs/1802.09933.

[103] S. Gratton, C. Royer, L. Vicente, and Z. Zhang. *Direct Search Based on Probabilistic Descent*. SIAM Journal on Optimization **vol. 25**(n° 3), pp. 1515 (2015). Publisher: Society for Industrial and Applied Mathematics, URL https://hal.science/hal-01523690.

[104] T. E. B. E. Kadi and O. Saadi. *On the Almost Sure Convergence of the Stochastic Three Points Algorithm* (2025). ArXiv:2501.13886 [math], URL http://arxiv.org/abs/2501.13886.